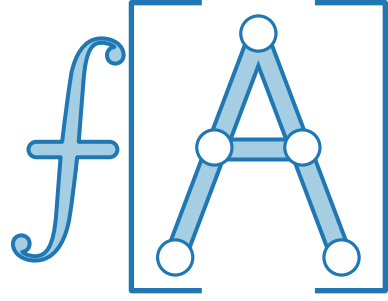
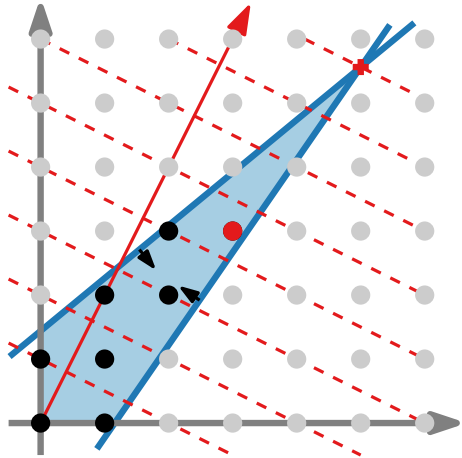




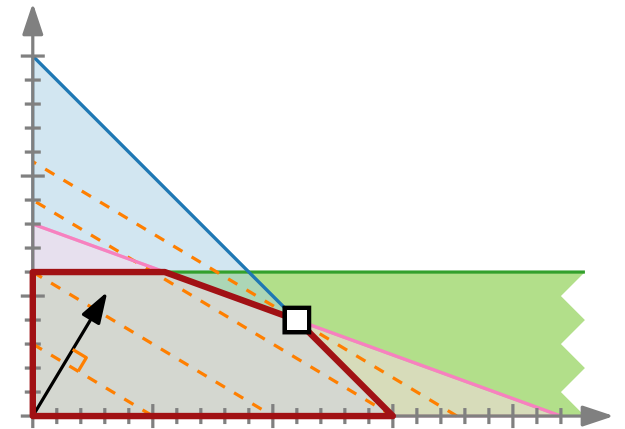
Fortgeschrittene Algorithmen

22. Vorlesung ILP & MaxSAT Solver

Teil I: Problemstellung

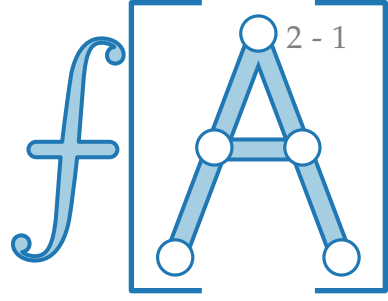


$$\begin{pmatrix} \text{blue} & \text{green} & \text{teal} \\ 0 & \dots & 0 & 1 & 0 & \dots & 0 \\ \text{red} & \text{pink} & \text{purple} \end{pmatrix}$$



Gewinnmaximierung

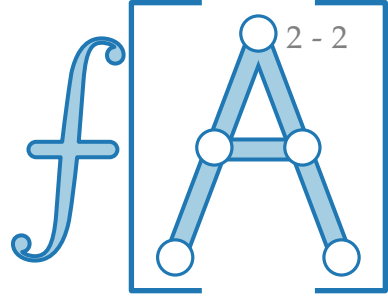
Sie sind Chef einer kleinen Firma, die zwei Produkte P_1 und P_2 herstellt.



Gewinnmaximierung

Sie sind Chef einer kleinen Firma, die zwei Produkte P_1 und P_2 herstellt. Produzieren Sie x_1 Einheiten P_1 und x_2 Einheiten P_2 , so beträgt Ihr Gewinn in €

$$G(x_1, x_2) = 30x_1 + 50x_2$$



Gewinnmaximierung

Sie sind Chef einer kleinen Firma, die zwei Produkte P_1 und P_2 herstellt. Produzieren Sie x_1 Einheiten P_1 und x_2 Einheiten P_2 , so beträgt Ihr Gewinn in €

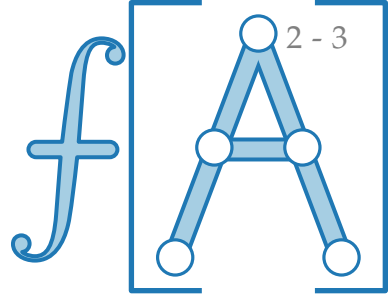
$$G(x_1, x_2) = 30x_1 + 50x_2$$

Drei Maschinen M_A , M_B und M_C produzieren die dafür jeweils notwendigen Einzelteile.

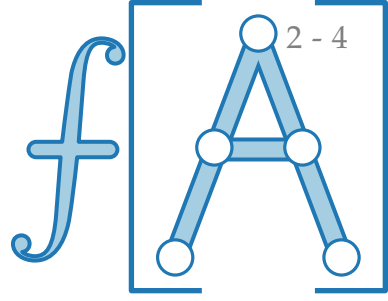
M_A :

M_B :

M_C :



Gewinnmaximierung



Sie sind Chef einer kleinen Firma, die zwei Produkte P_1 und P_2 herstellt. Produzieren Sie x_1 Einheiten P_1 und x_2 Einheiten P_2 , so beträgt Ihr Gewinn in €

$$G(x_1, x_2) = 30x_1 + 50x_2$$

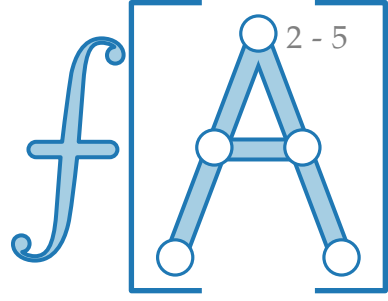
Drei Maschinen M_A , M_B und M_C produzieren die dafür jeweils notwendigen Einzelteile. Dabei brauchen sie eine bestimmte Zeit pro Teil und haben jeweils eine maximale Arbeitszeit, die die Produktion der Einzelteile einschränkt:

$$M_A: \quad 4x_1 + 11x_2 \leq 880$$

$$M_B: \quad x_1 + x_2 \leq 150$$

$$M_C: \quad x_2 \leq 60$$

Gewinnmaximierung



Sie sind Chef einer kleinen Firma, die zwei Produkte P_1 und P_2 herstellt. Produzieren Sie x_1 Einheiten P_1 und x_2 Einheiten P_2 , so beträgt Ihr Gewinn in €

$$G(x_1, x_2) = 30x_1 + 50x_2$$

Drei Maschinen M_A , M_B und M_C produzieren die dafür jeweils notwendigen Einzelteile. Dabei brauchen sie eine bestimmte Zeit pro Teil und haben jeweils eine maximale Arbeitszeit, die die Produktion der Einzelteile einschränkt:

$$M_A: \quad 4x_1 + 11x_2 \leq 880$$

$$M_B: \quad x_1 + x_2 \leq 150$$

$$M_C: \quad x_2 \leq 60$$

Welche Wahl von (x_1, x_2) maximiert den Gewinn?

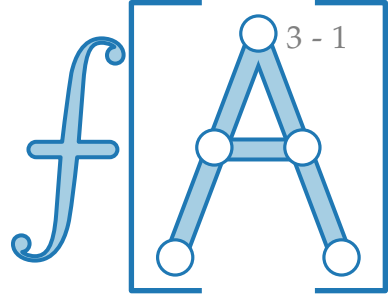
Lösung

Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



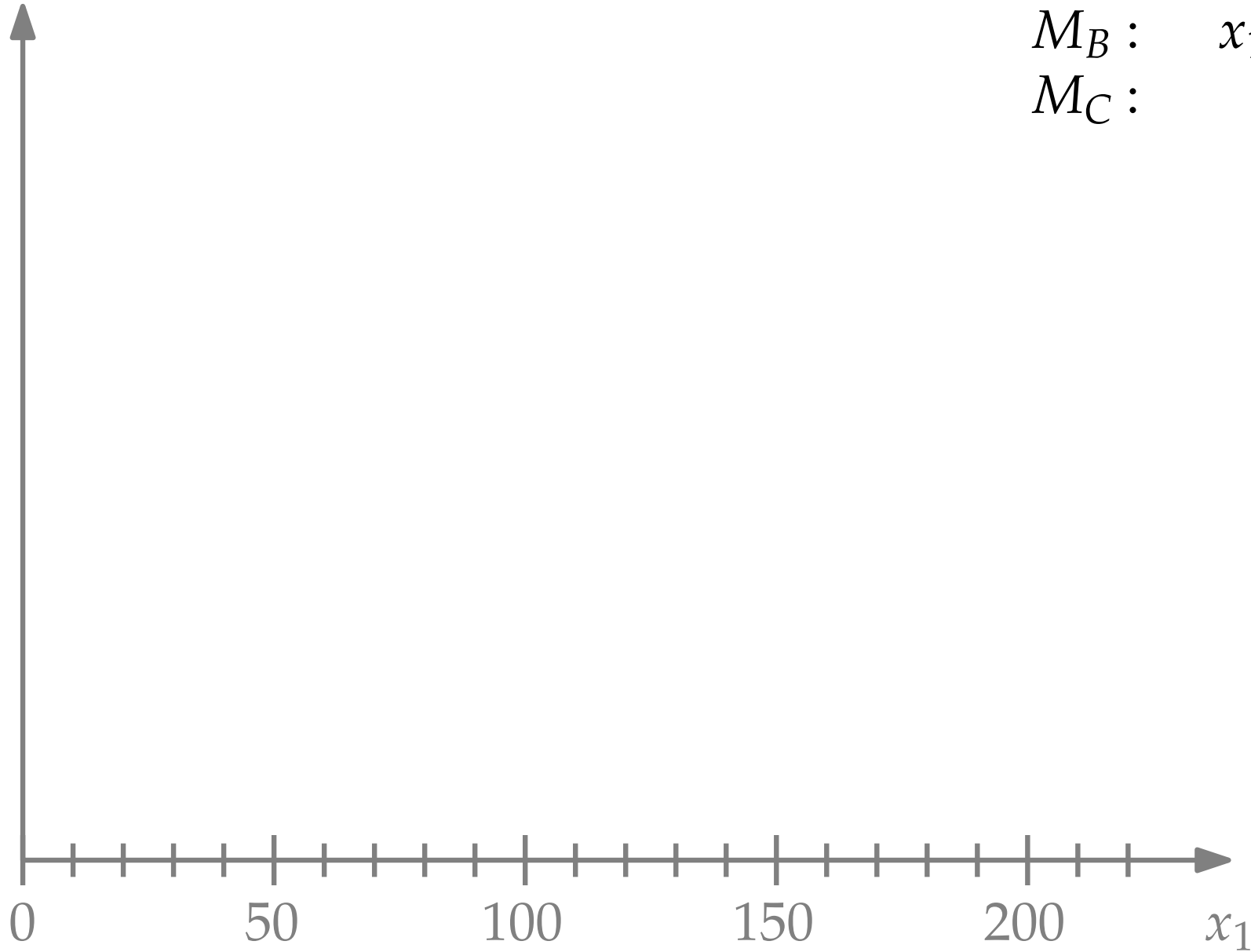
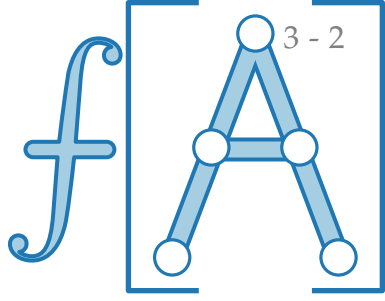
Lösung

Lineare Beschränkungen:

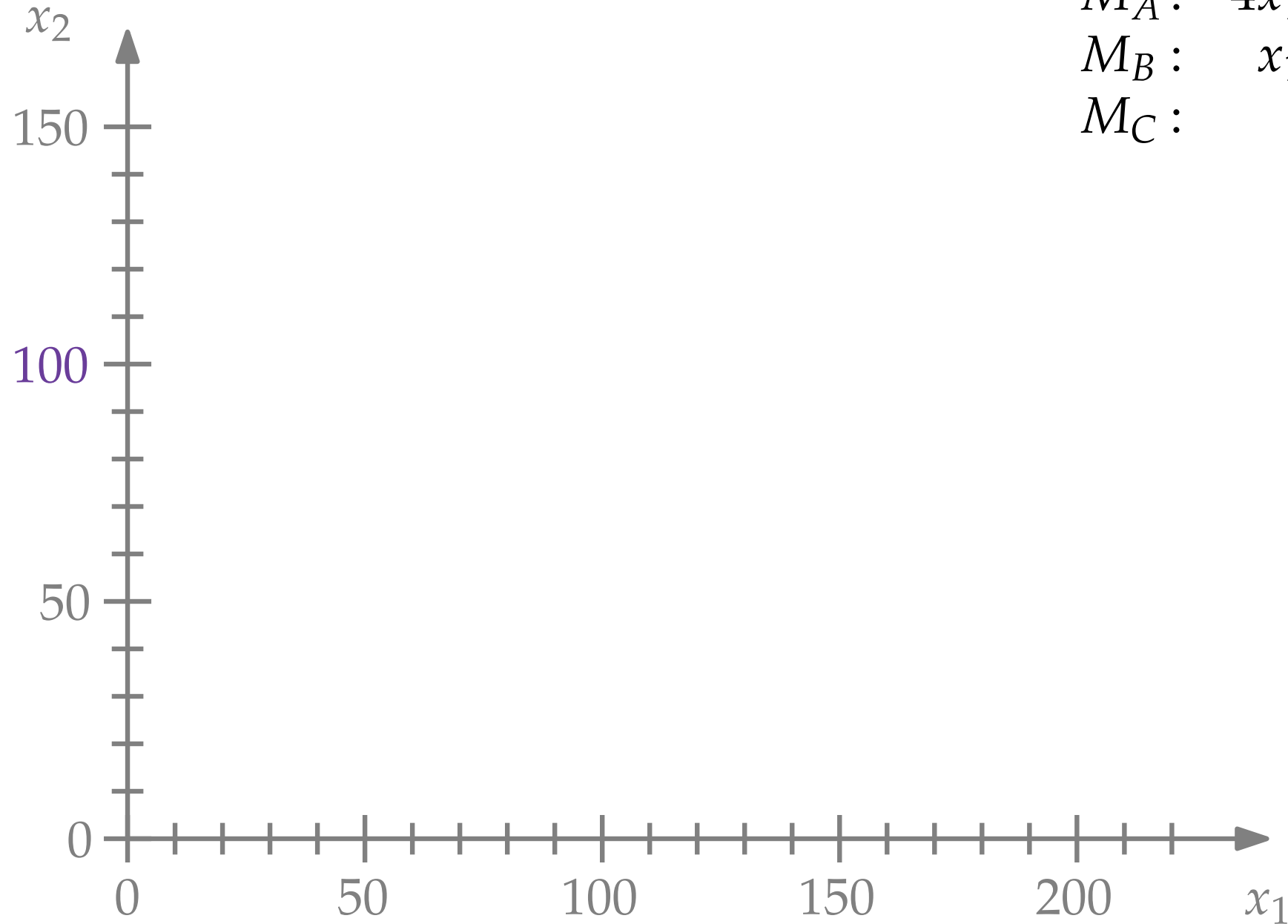
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung

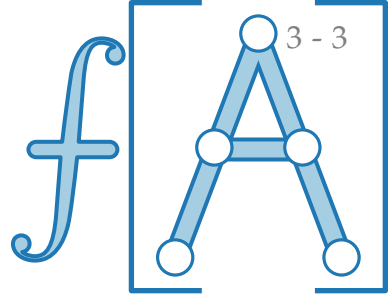


Lineare Beschränkungen:

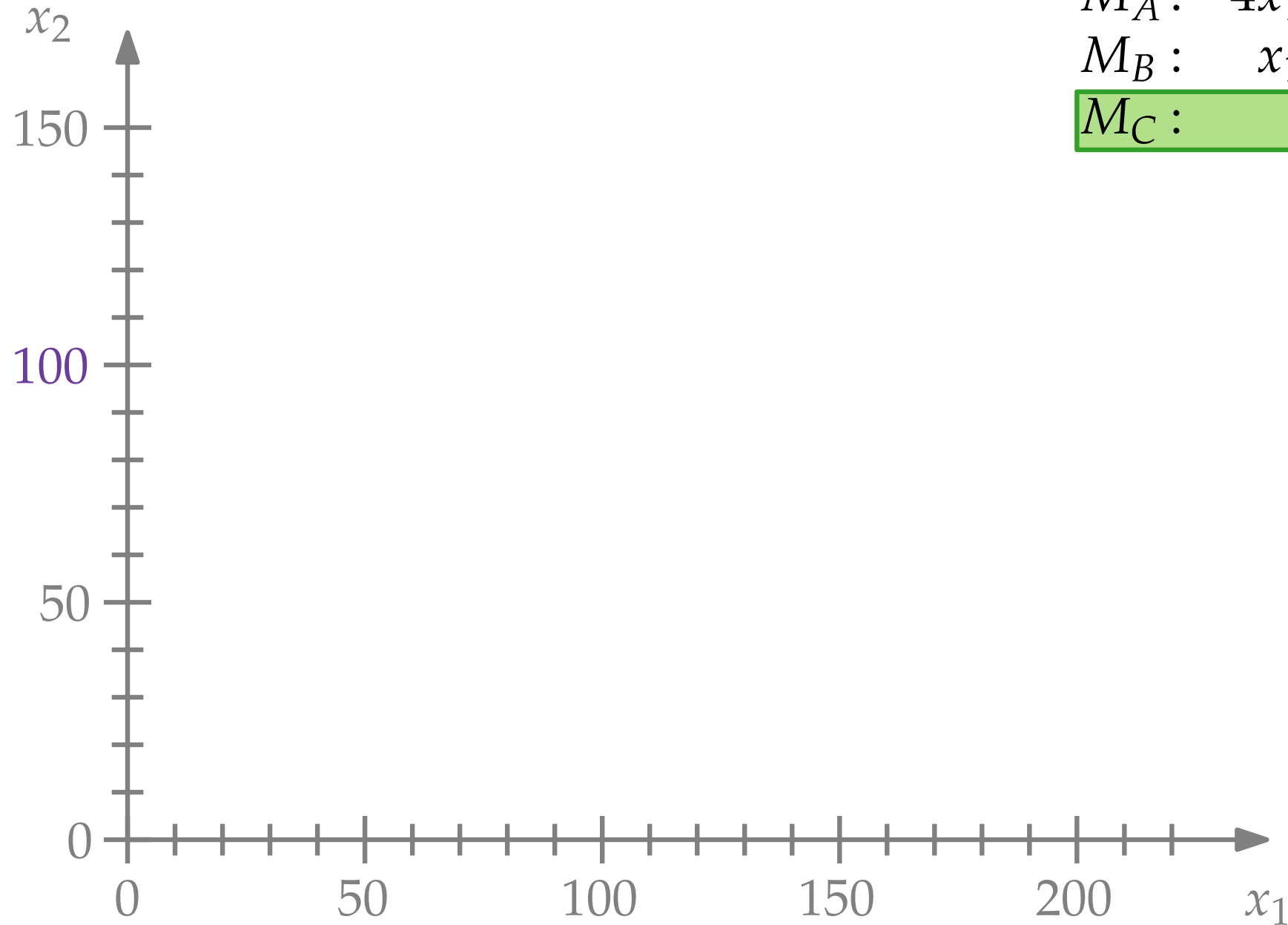
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung

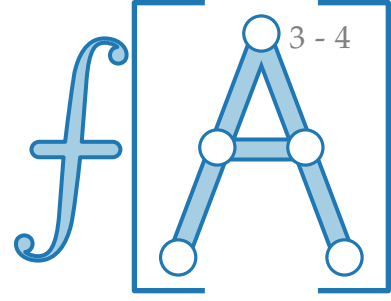


Lineare Beschränkungen:

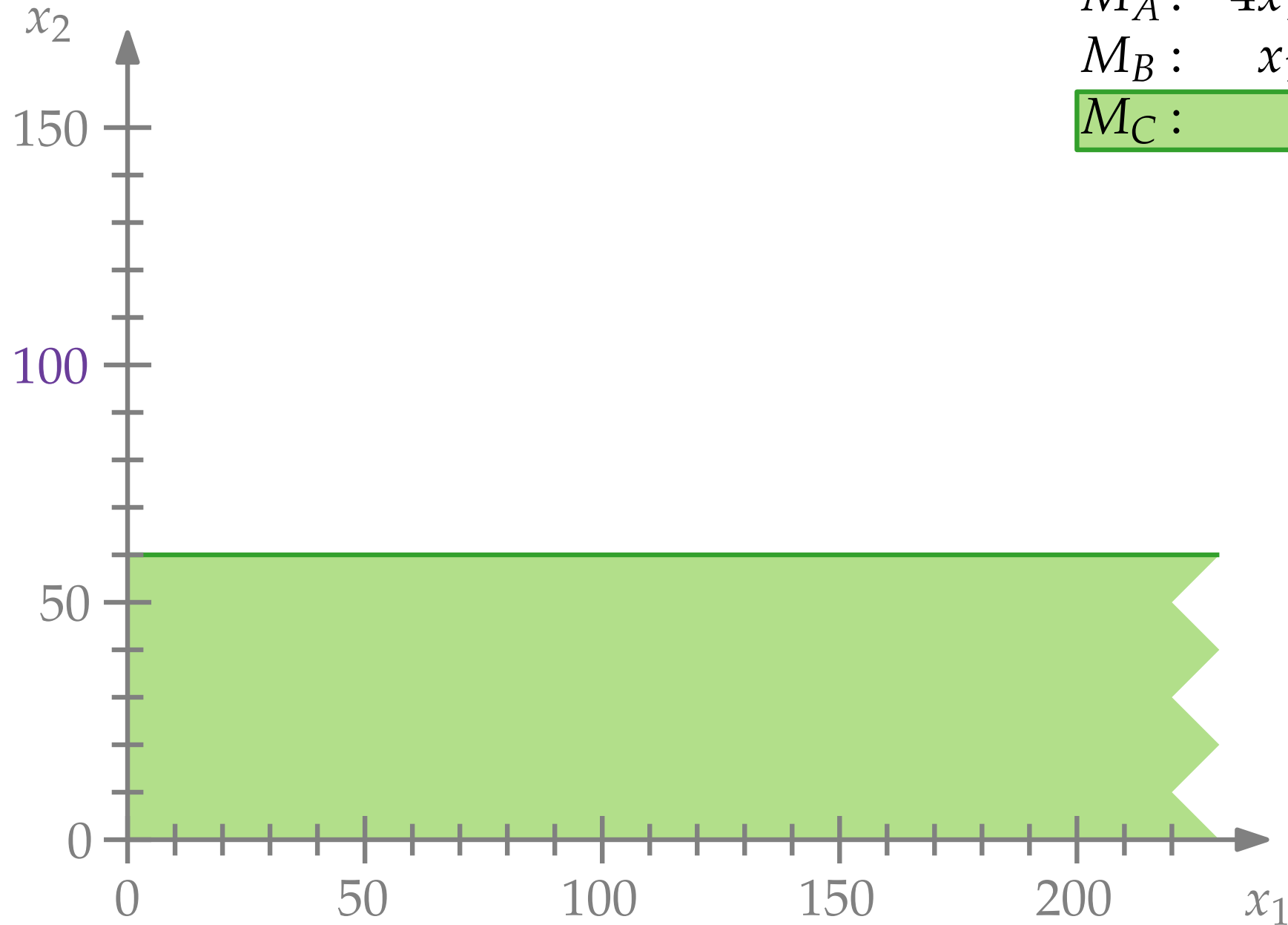
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung

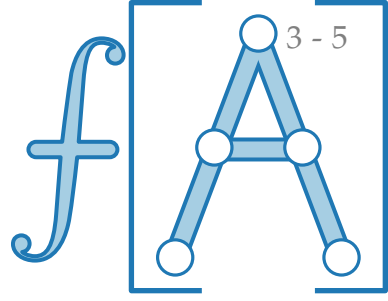


Lineare Beschränkungen:

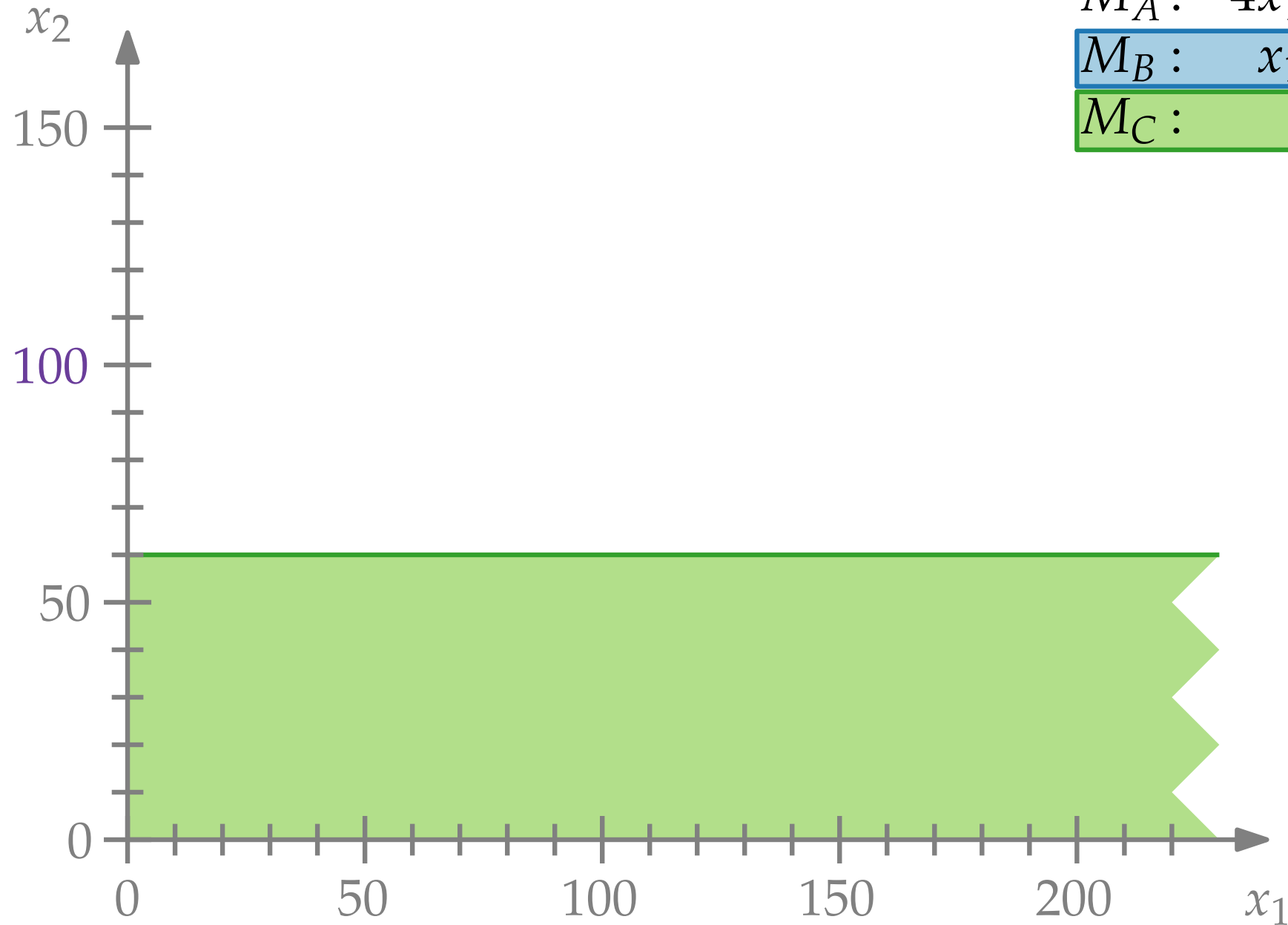
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung

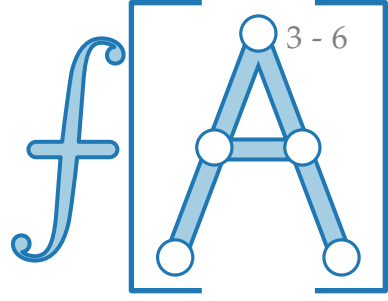


Lineare Beschränkungen:

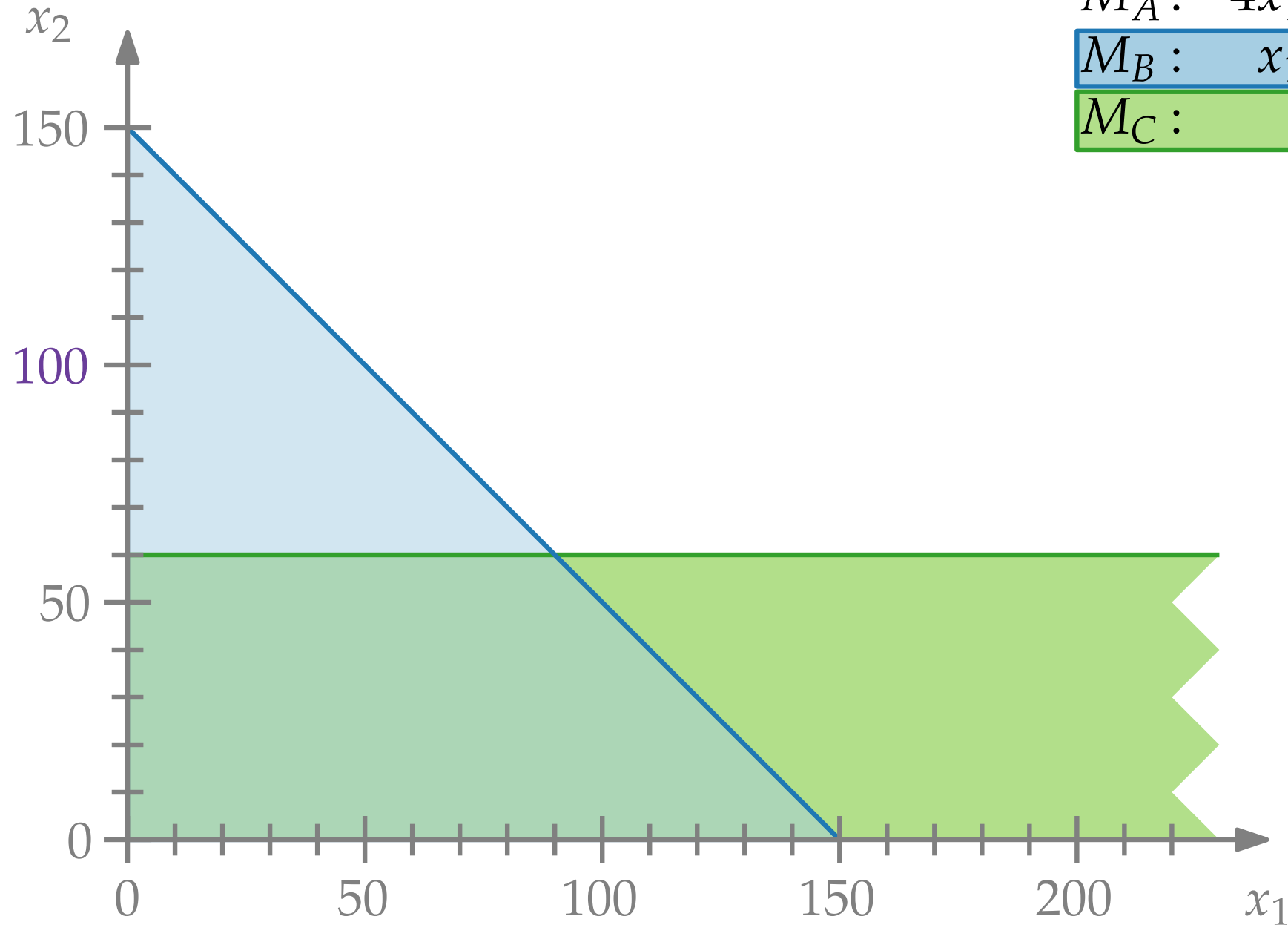
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung

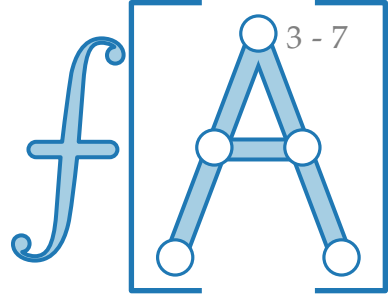


Lineare Beschränkungen:

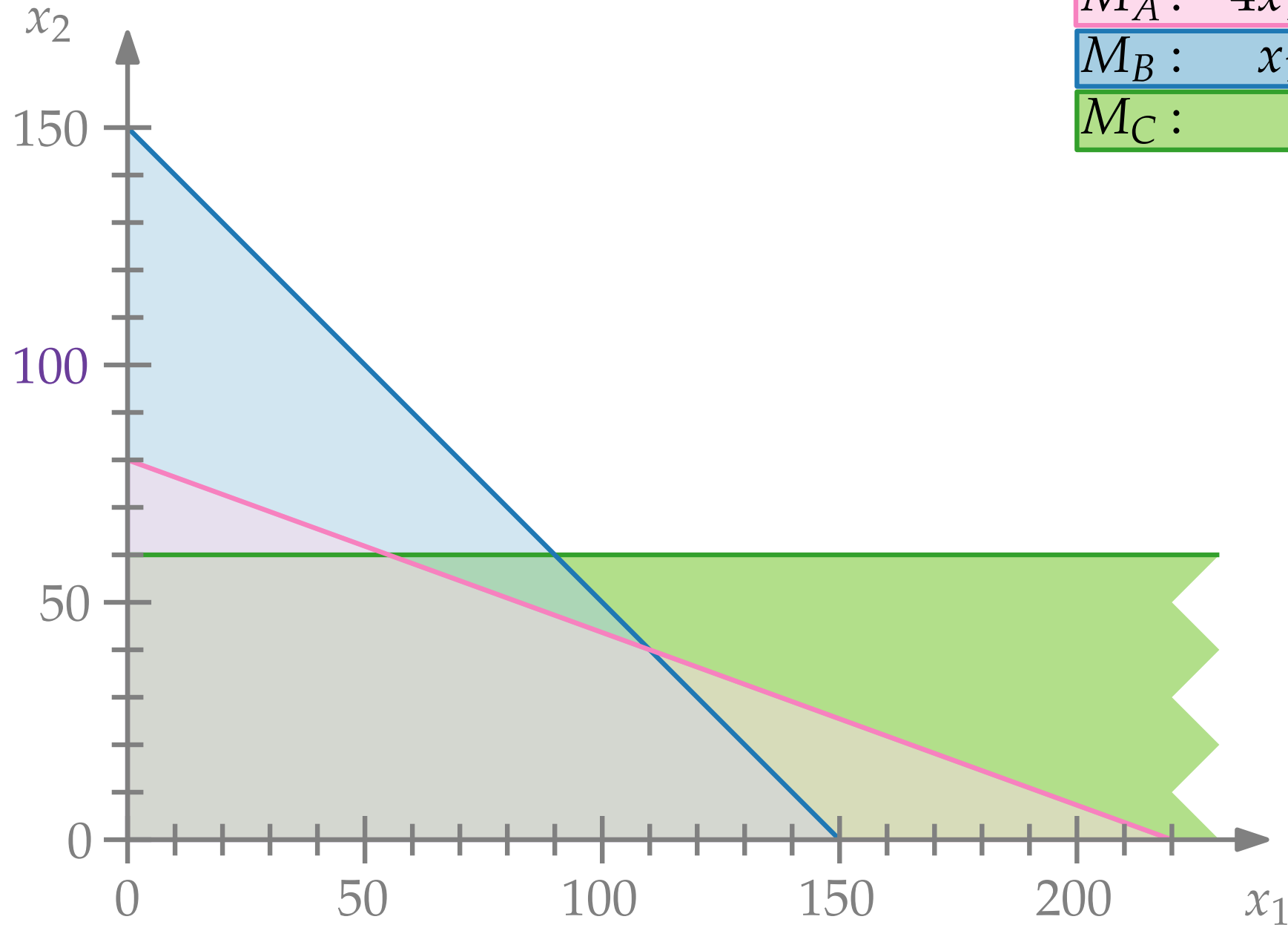
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung

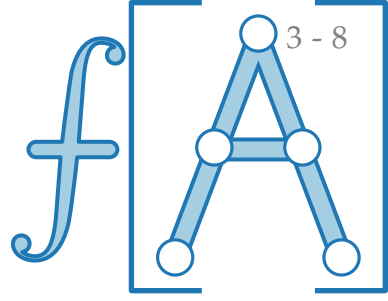


Lineare Beschränkungen:

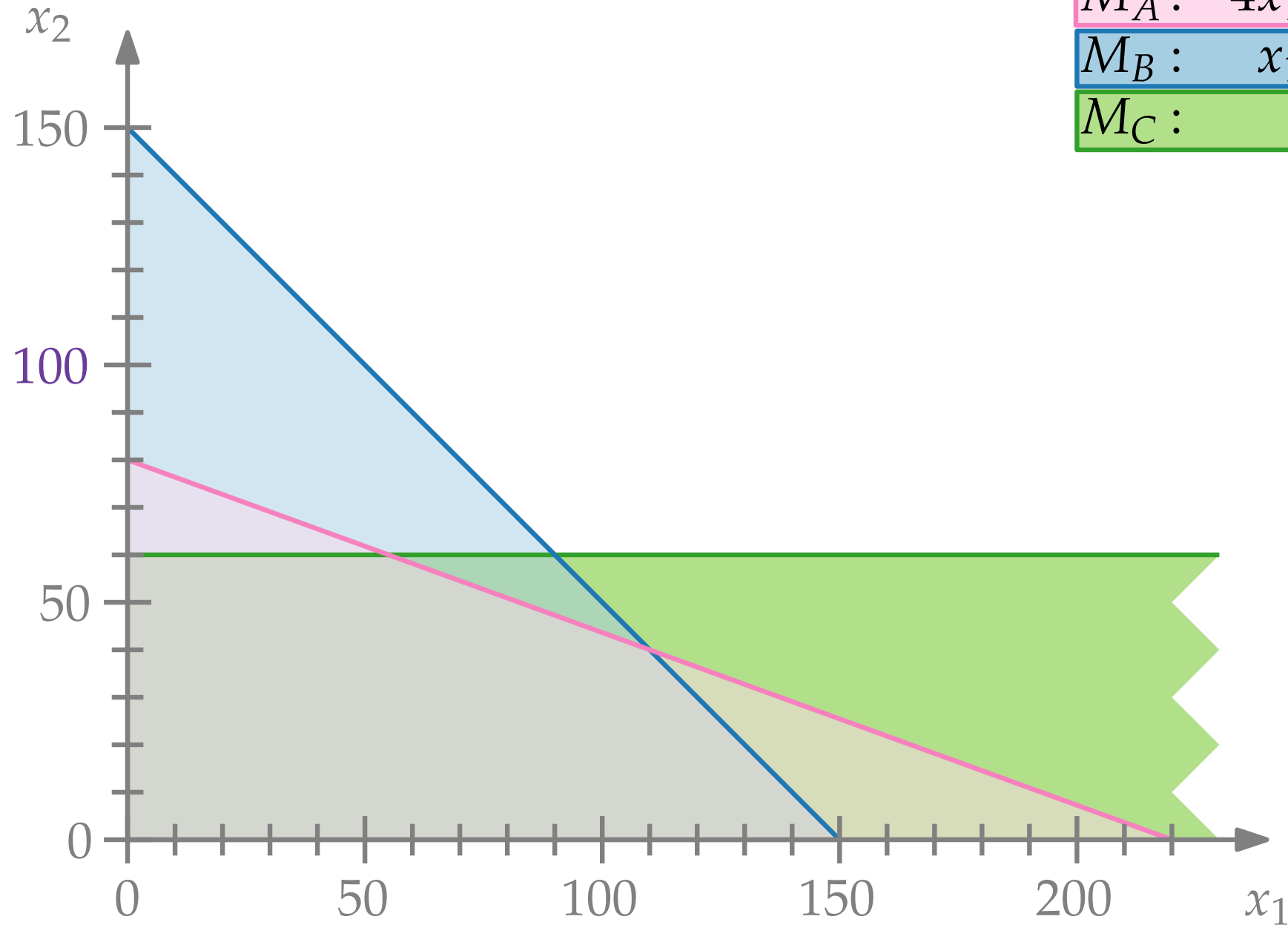
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$



Lösung



Lineare Beschränkungen:

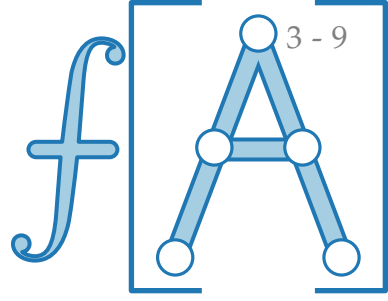
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

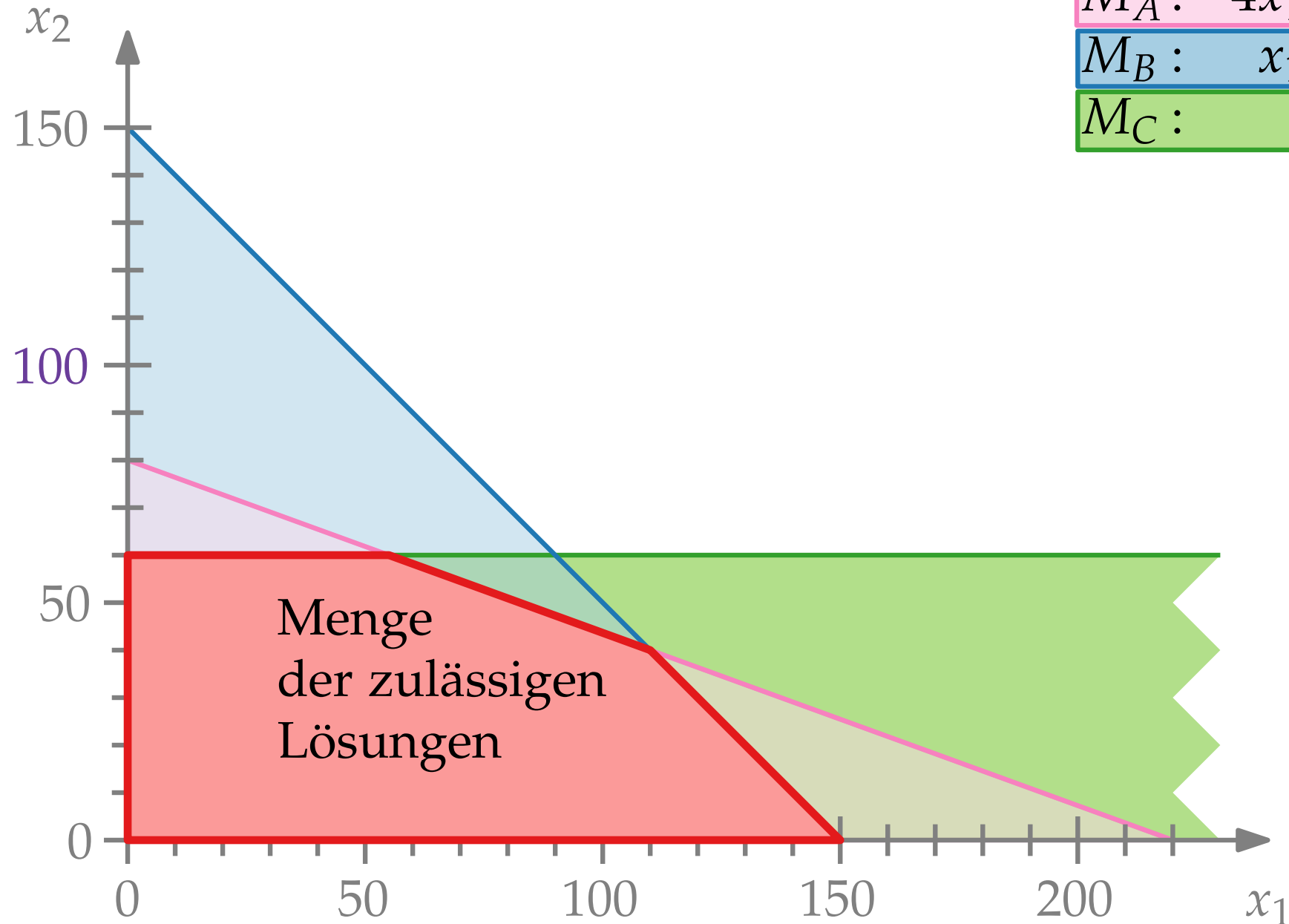
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$



Lösung



Lineare Beschränkungen:

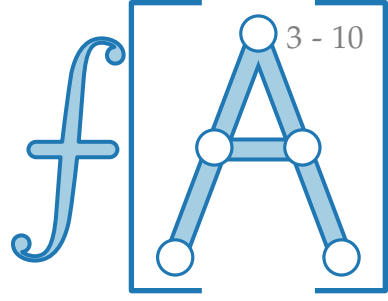
$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

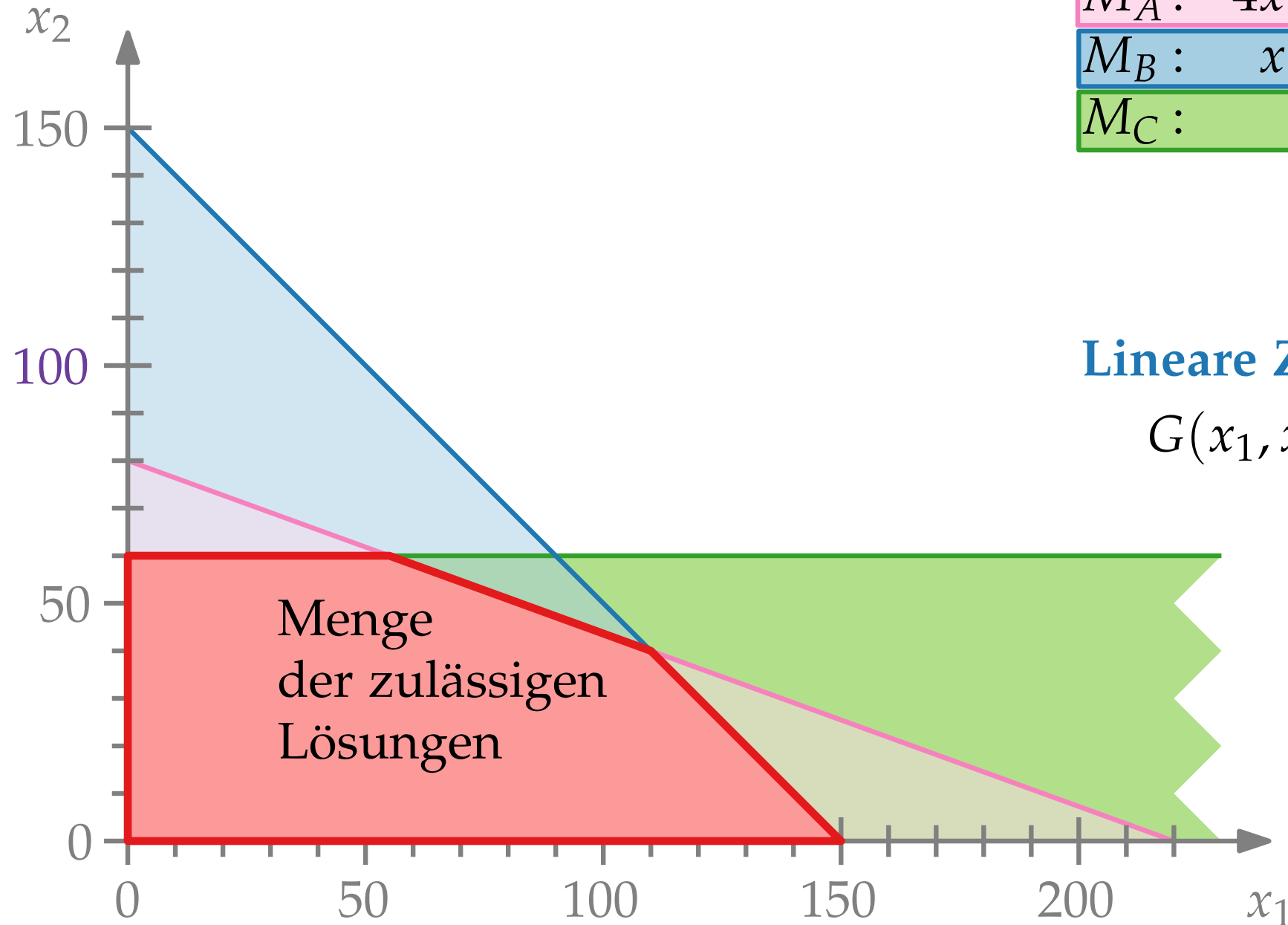
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

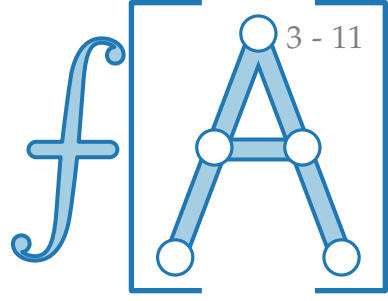
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

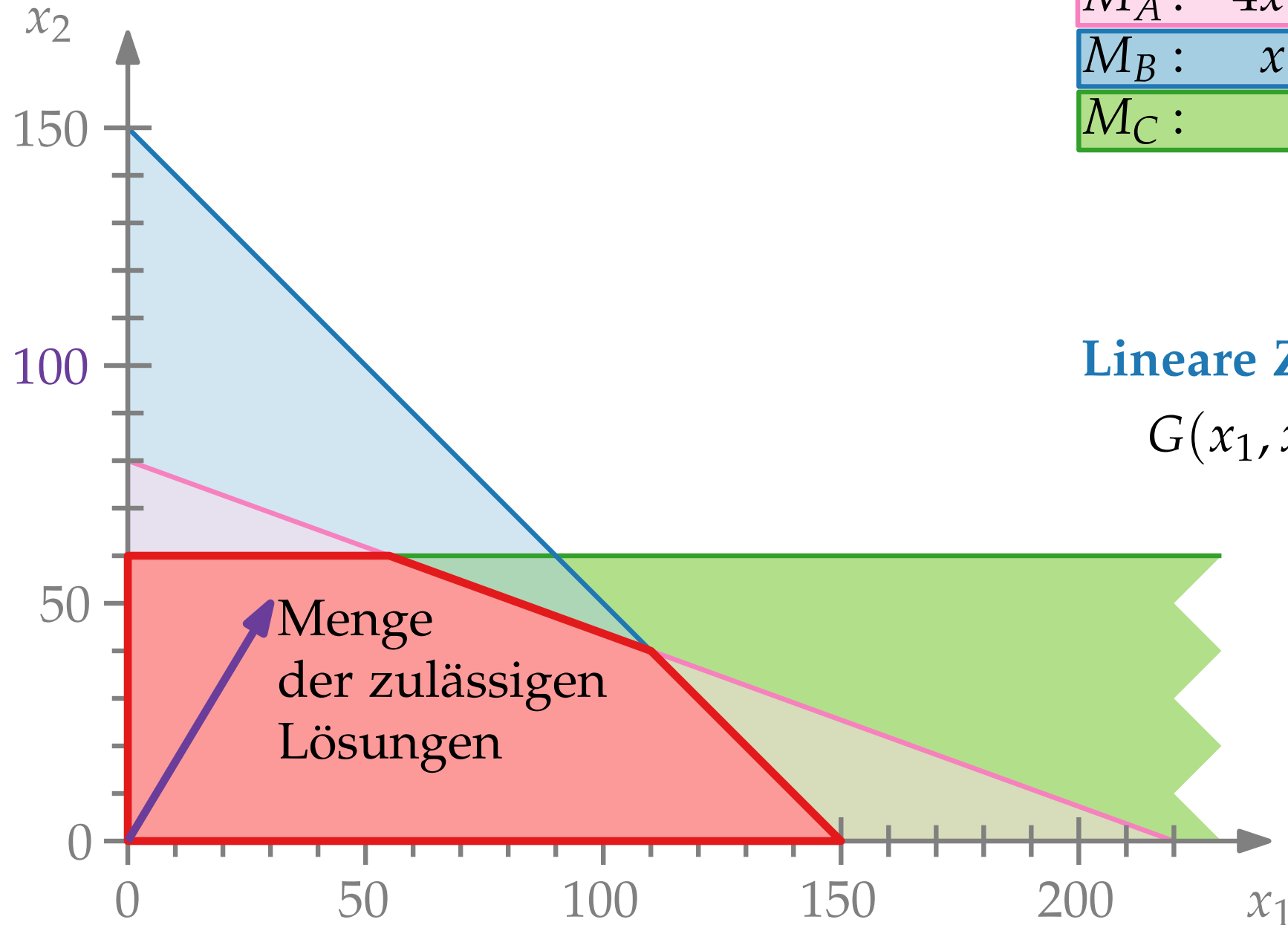
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$G(x_1, x_2) = 30x_1 + 50x_2$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

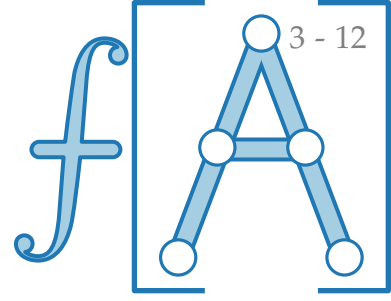
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

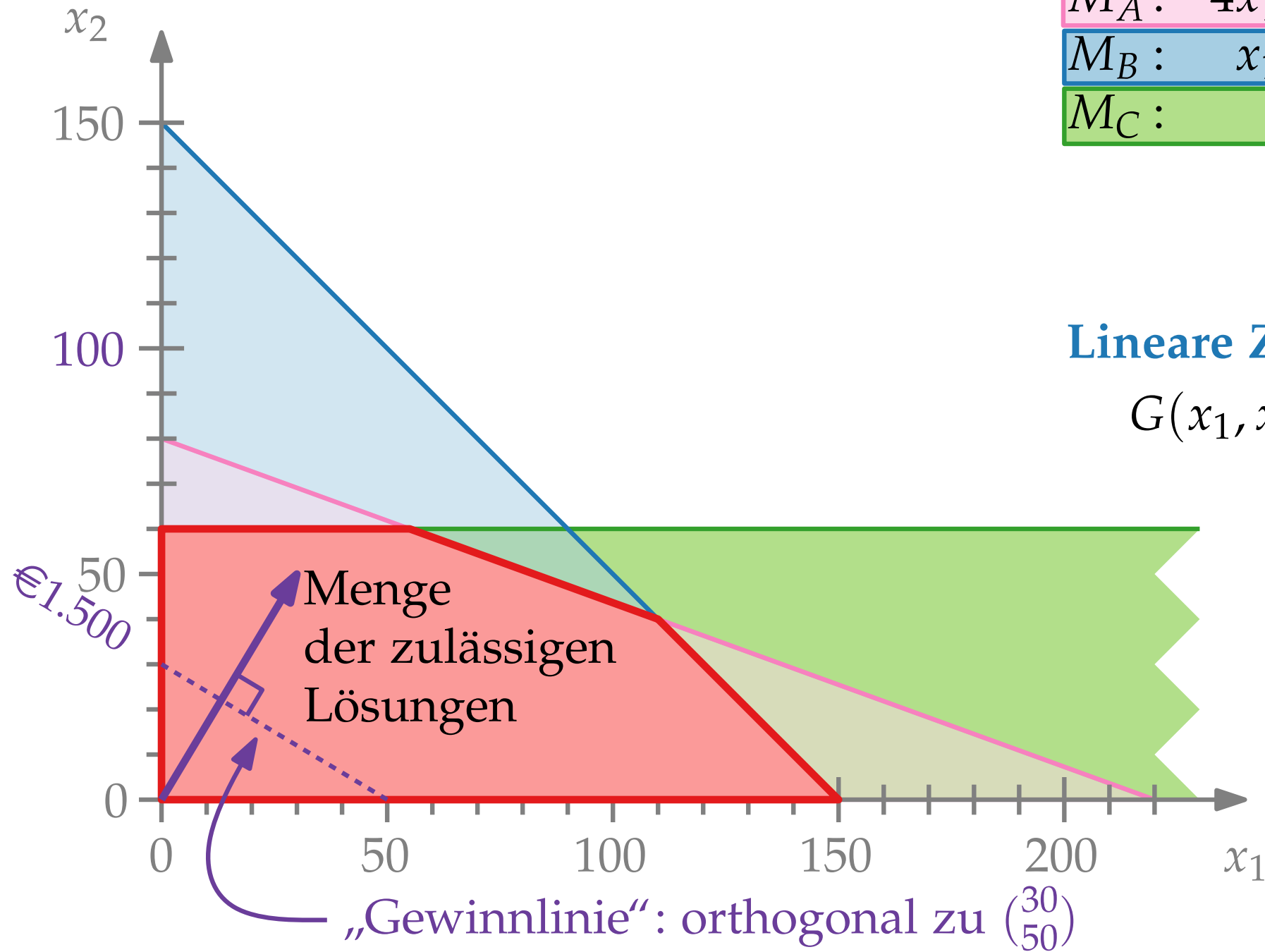
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

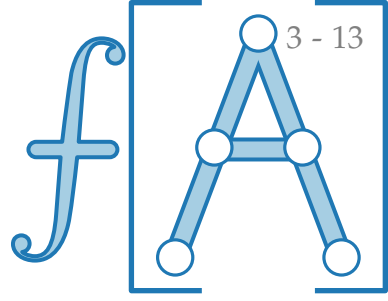
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

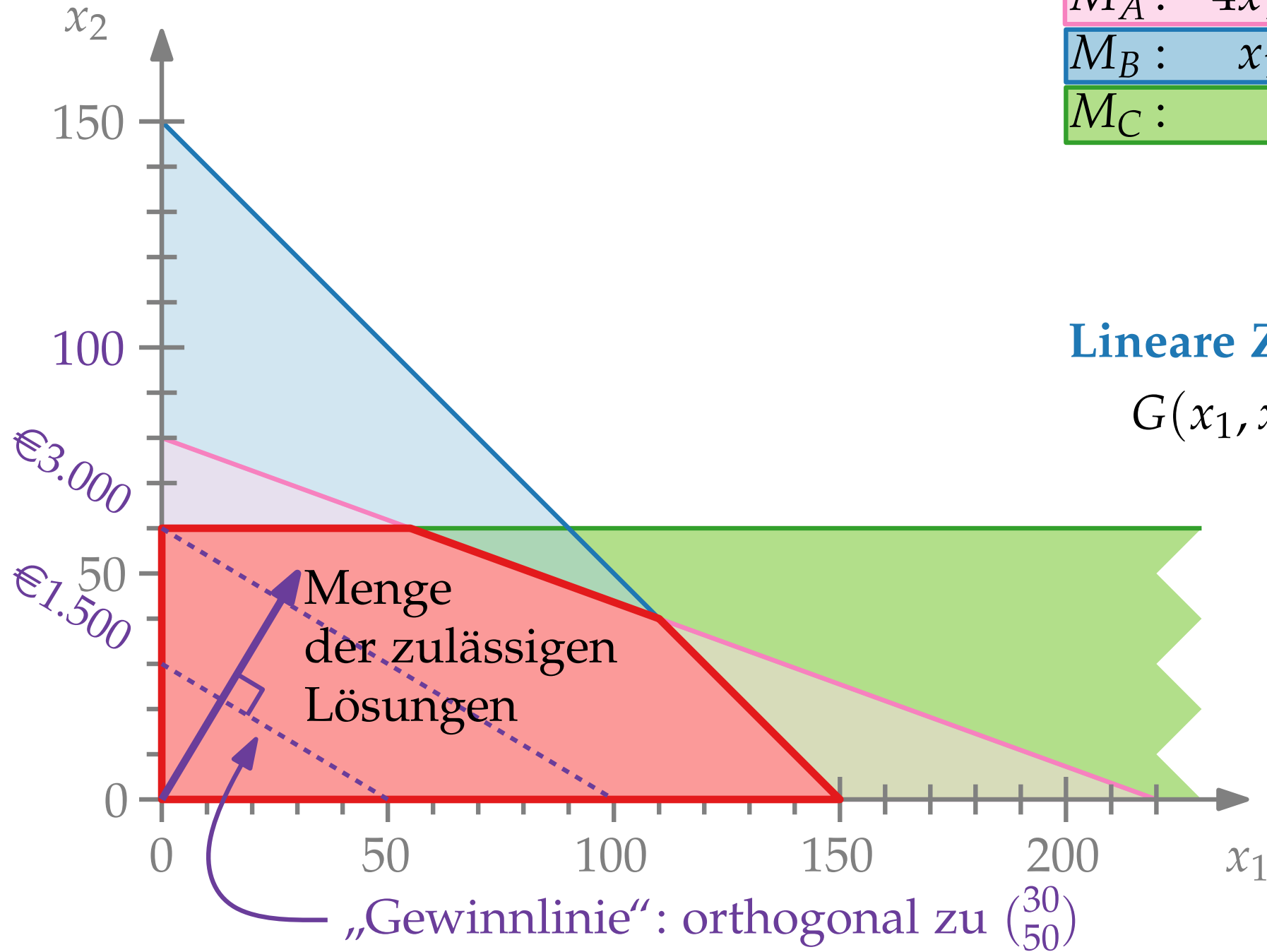
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

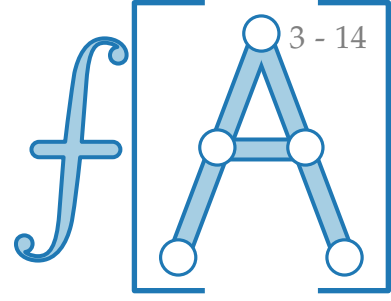
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

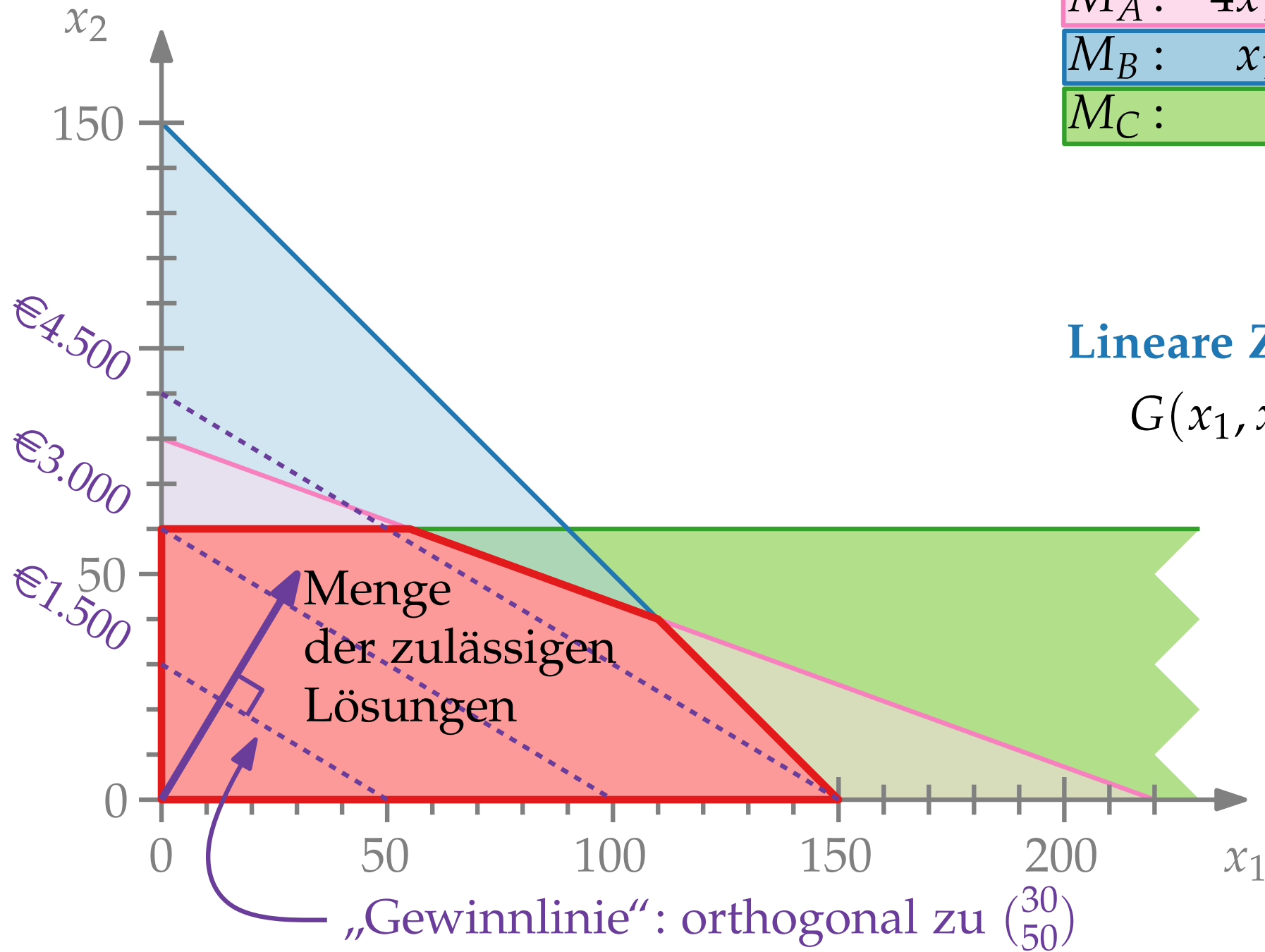
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

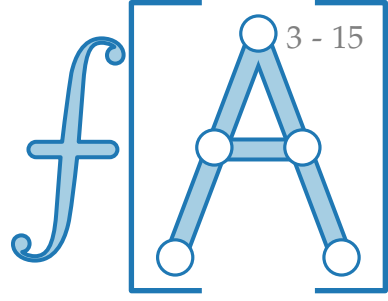
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

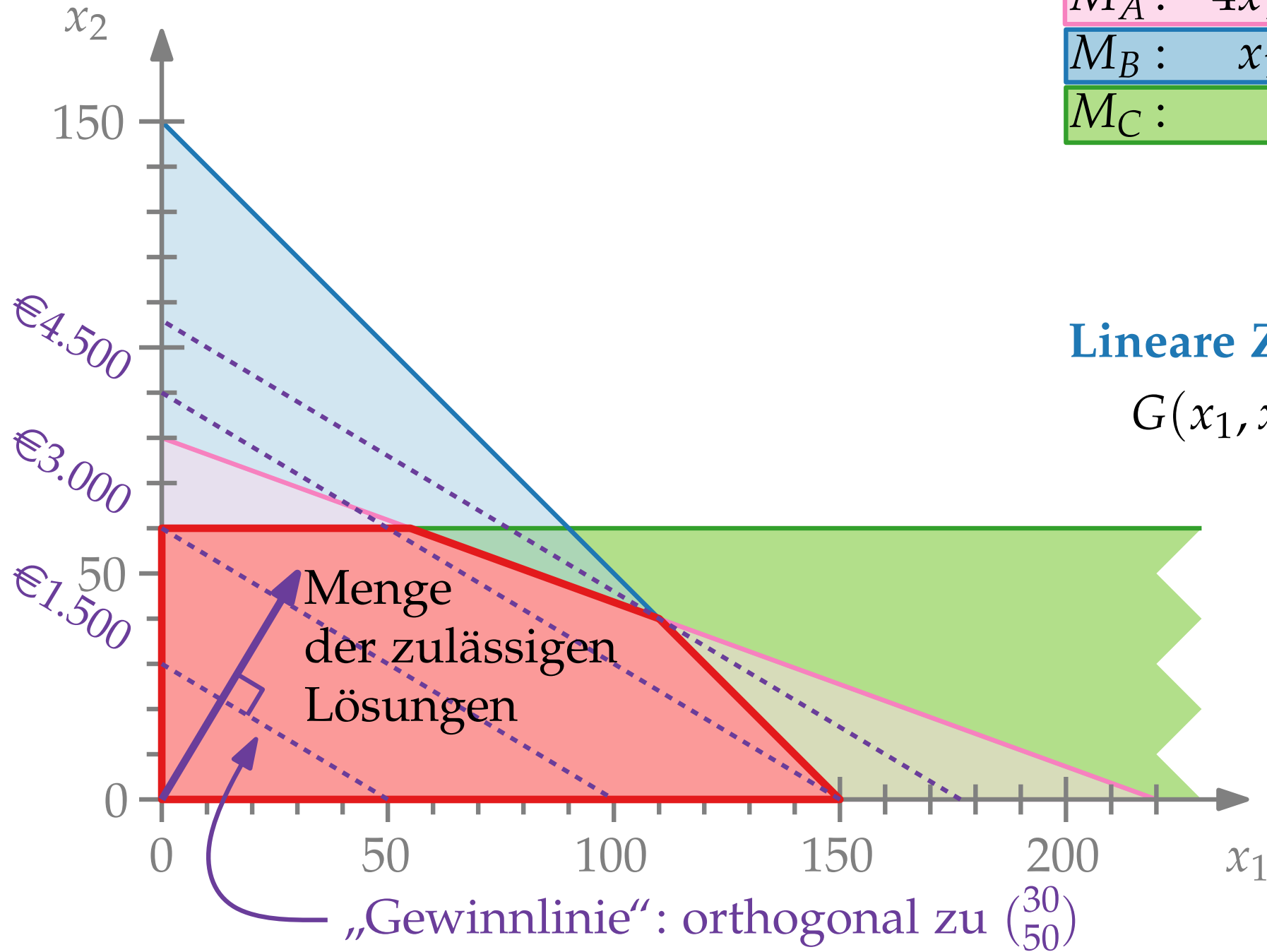
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

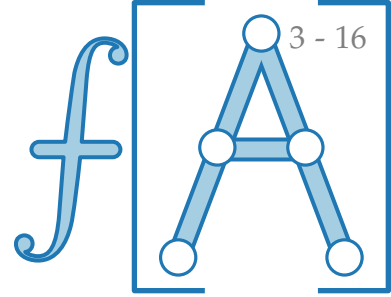
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

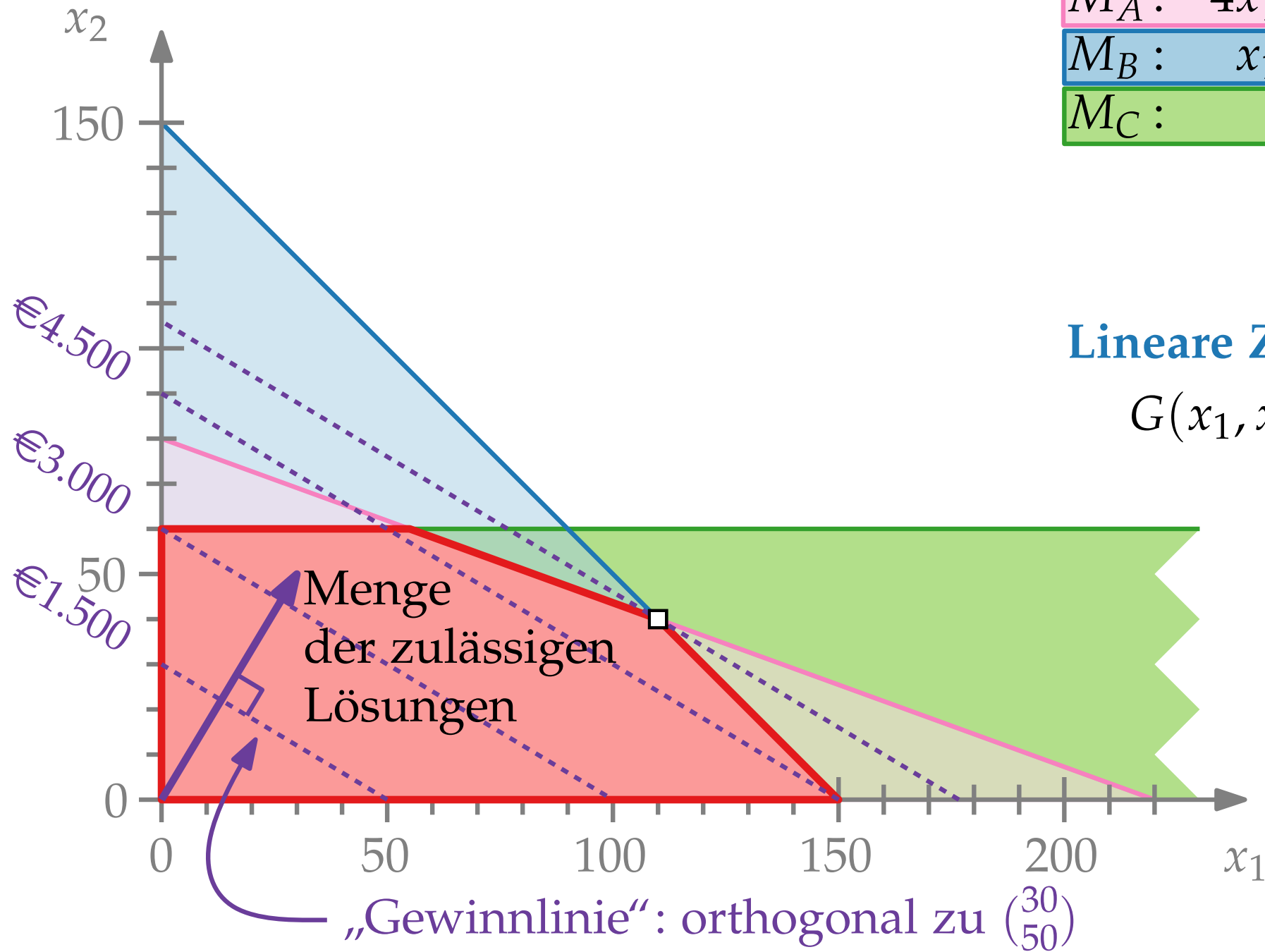
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

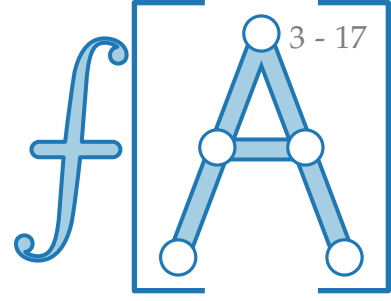
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung

Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

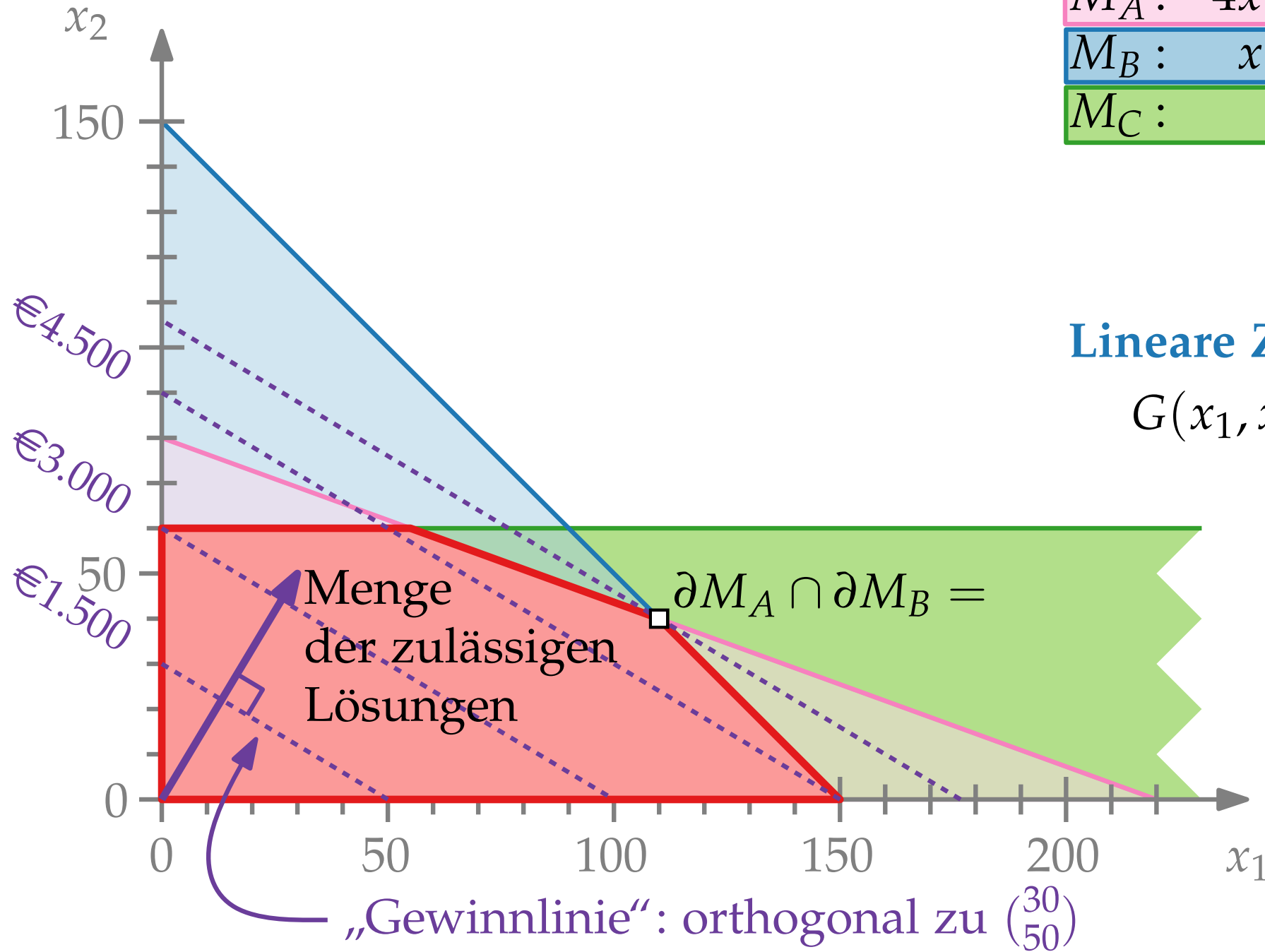
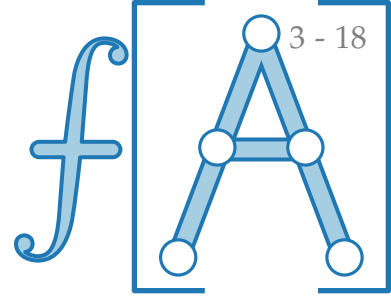
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung

Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

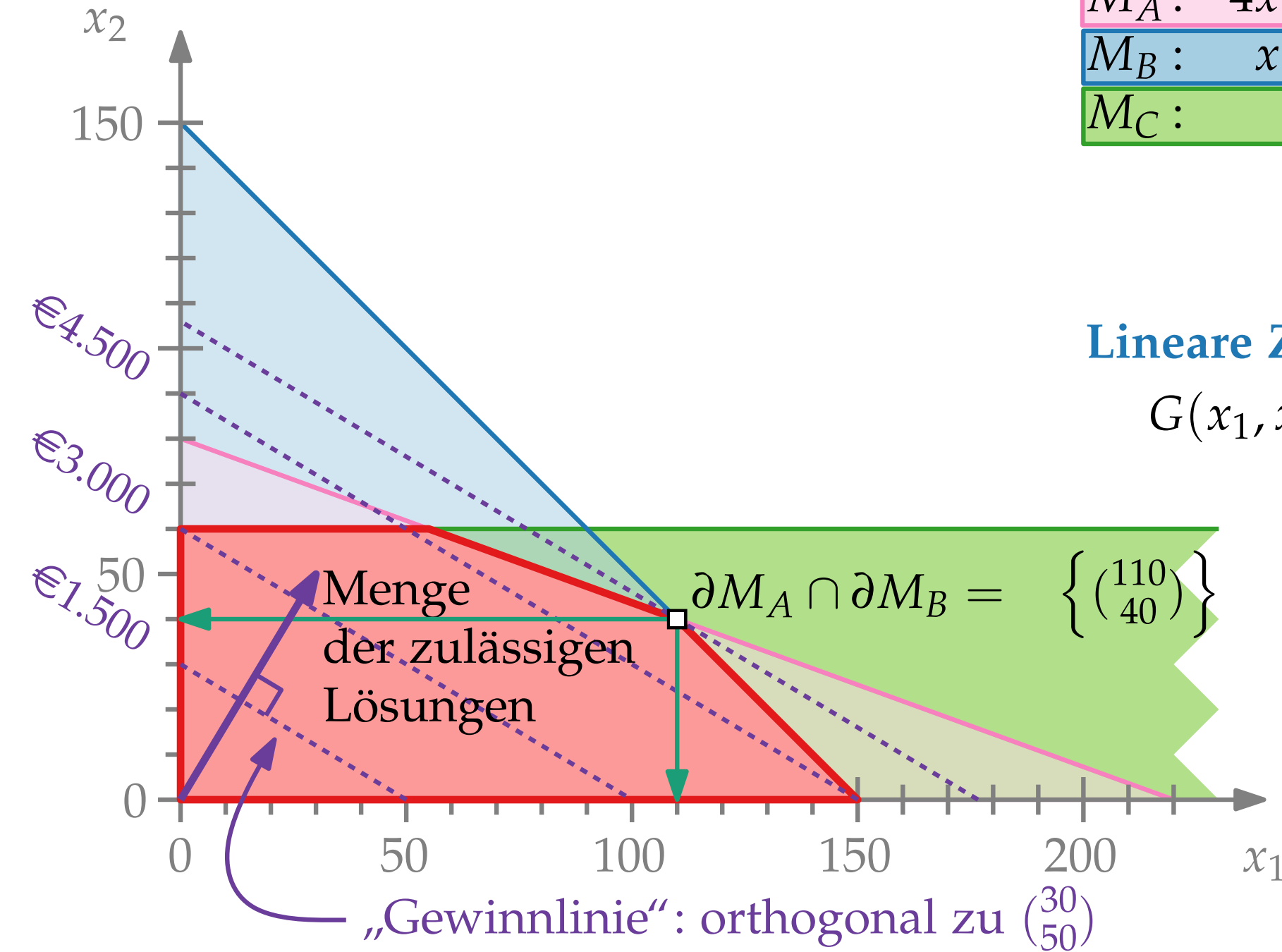
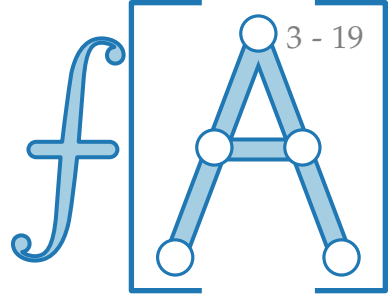
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

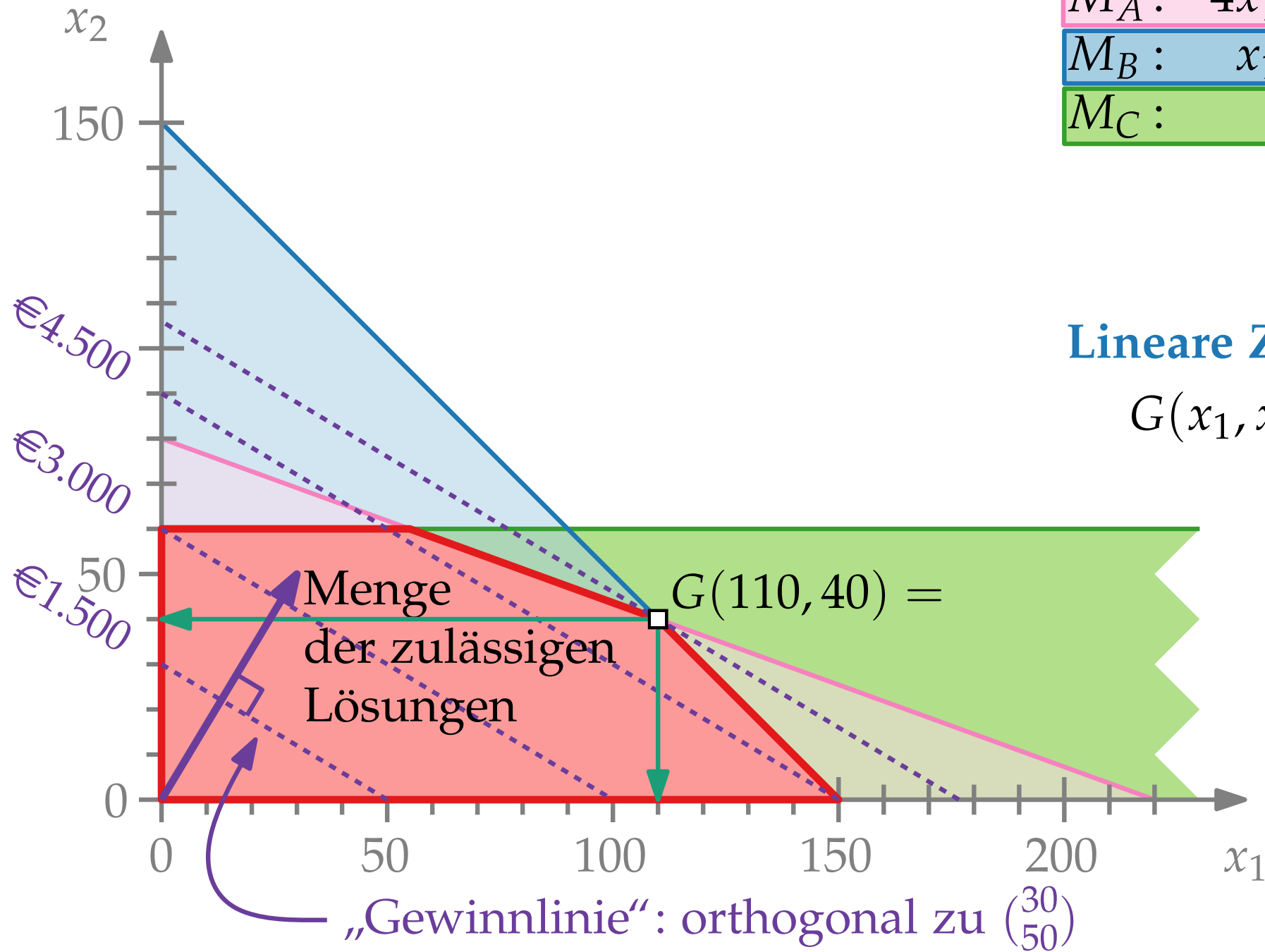
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

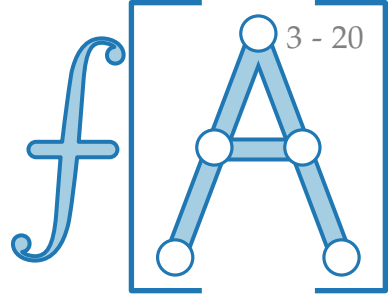
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Lineare Zielfunktion:

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$



Lösung

Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

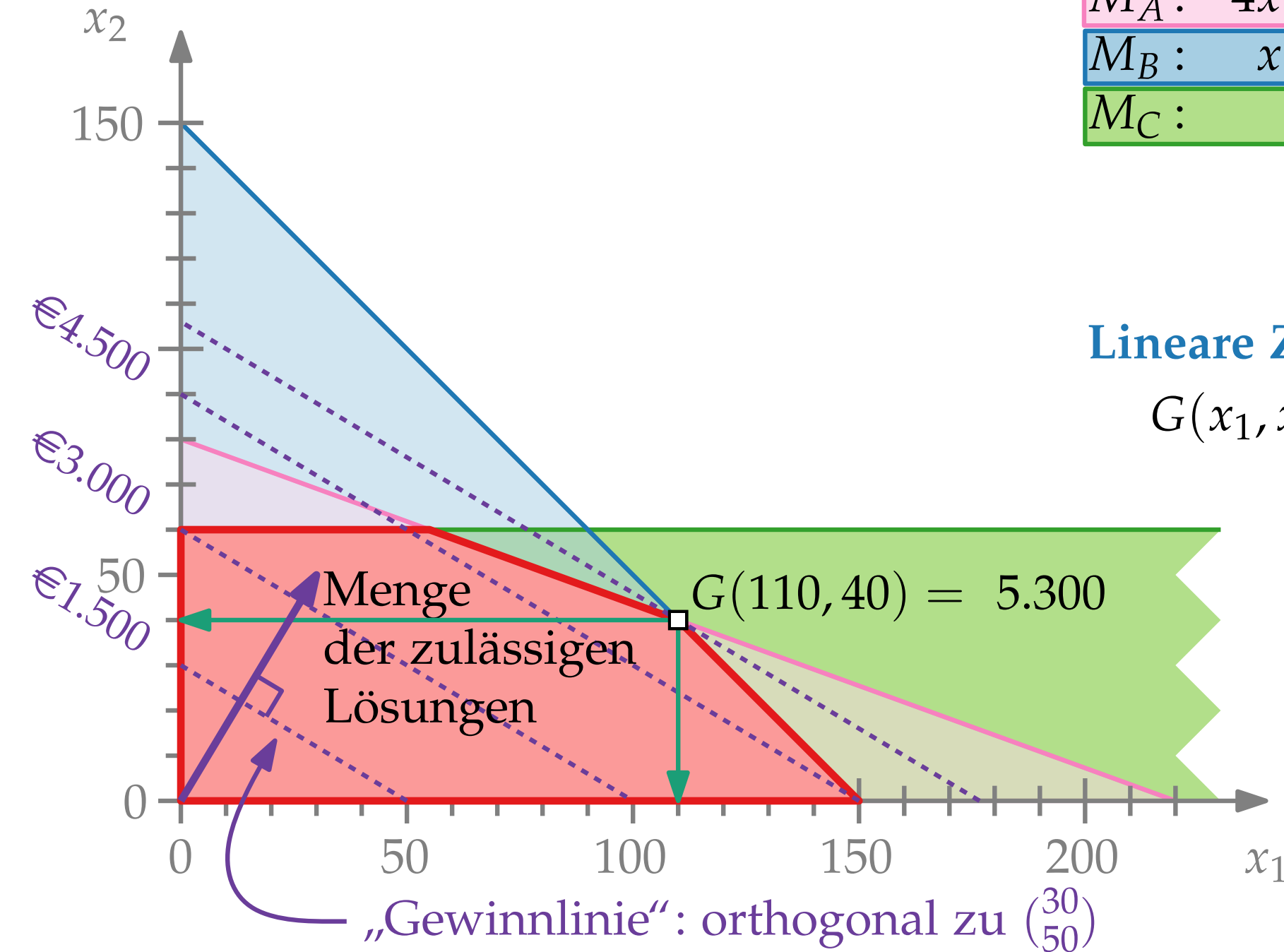
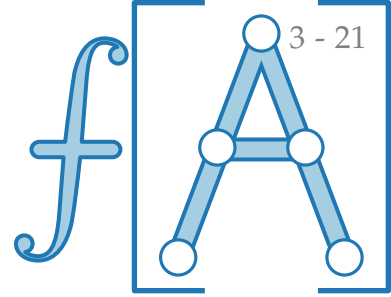
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung

Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

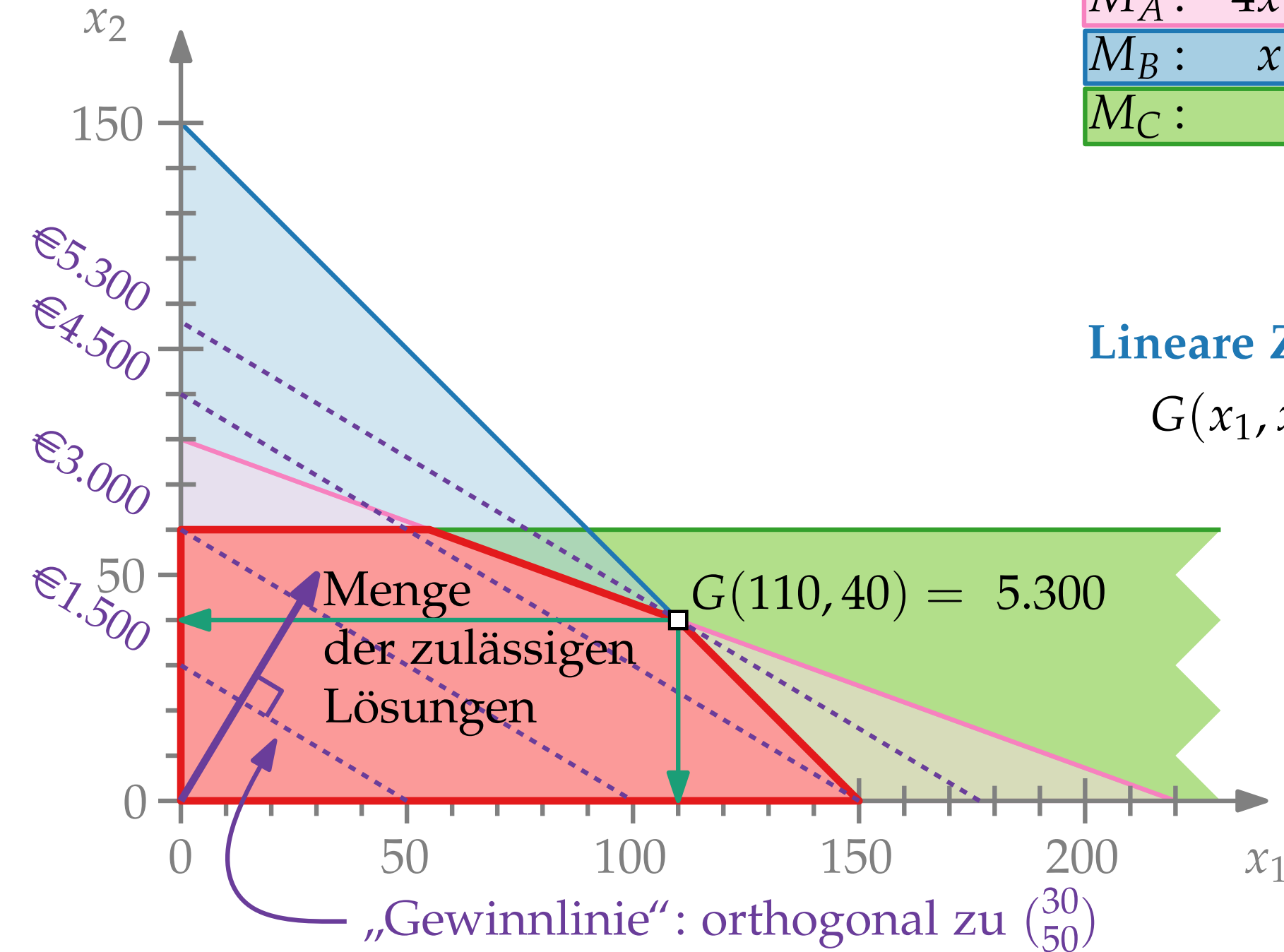
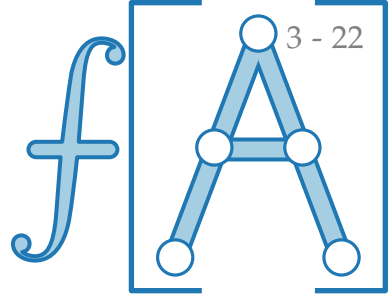
$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

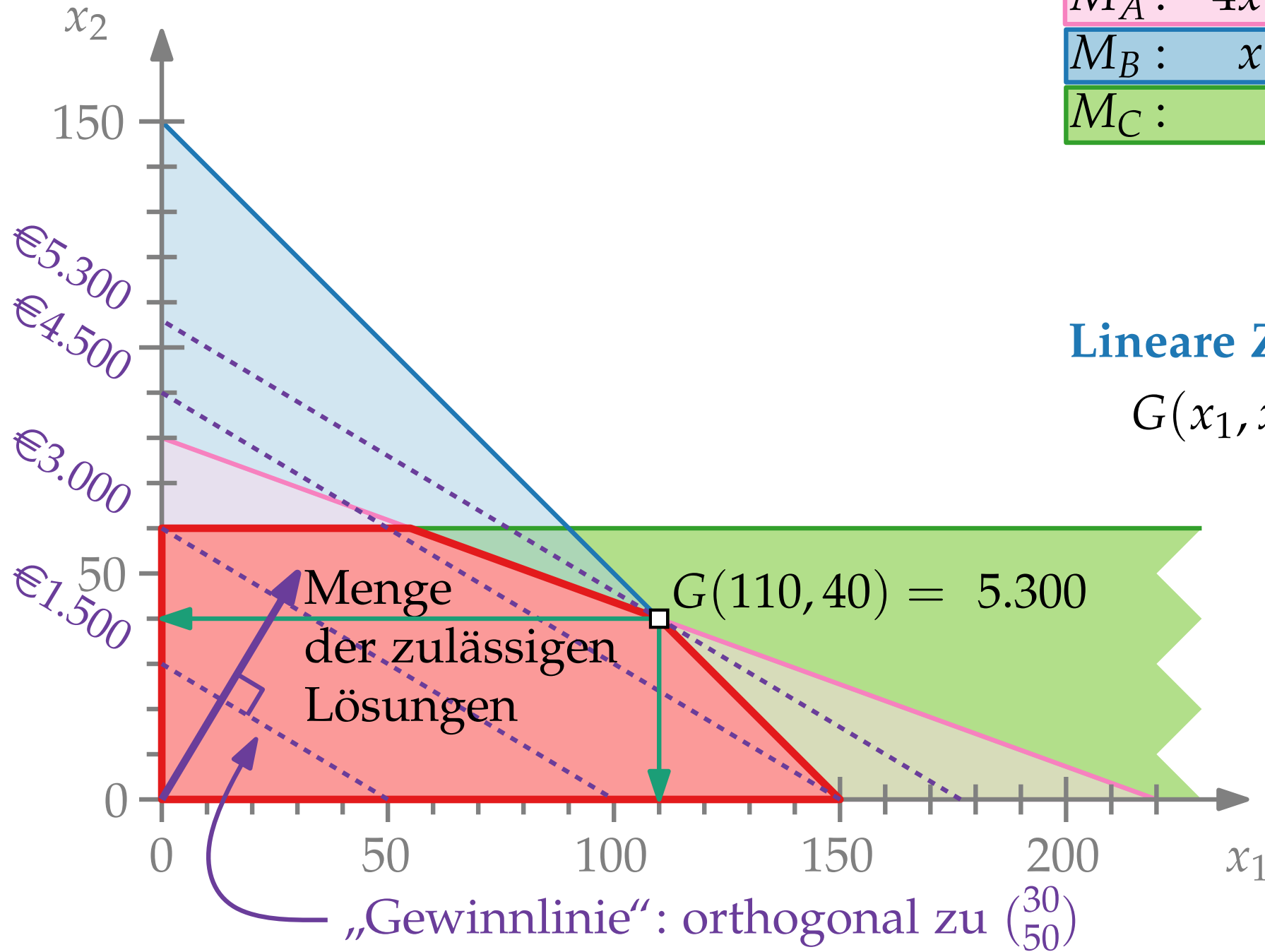
$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$



Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$

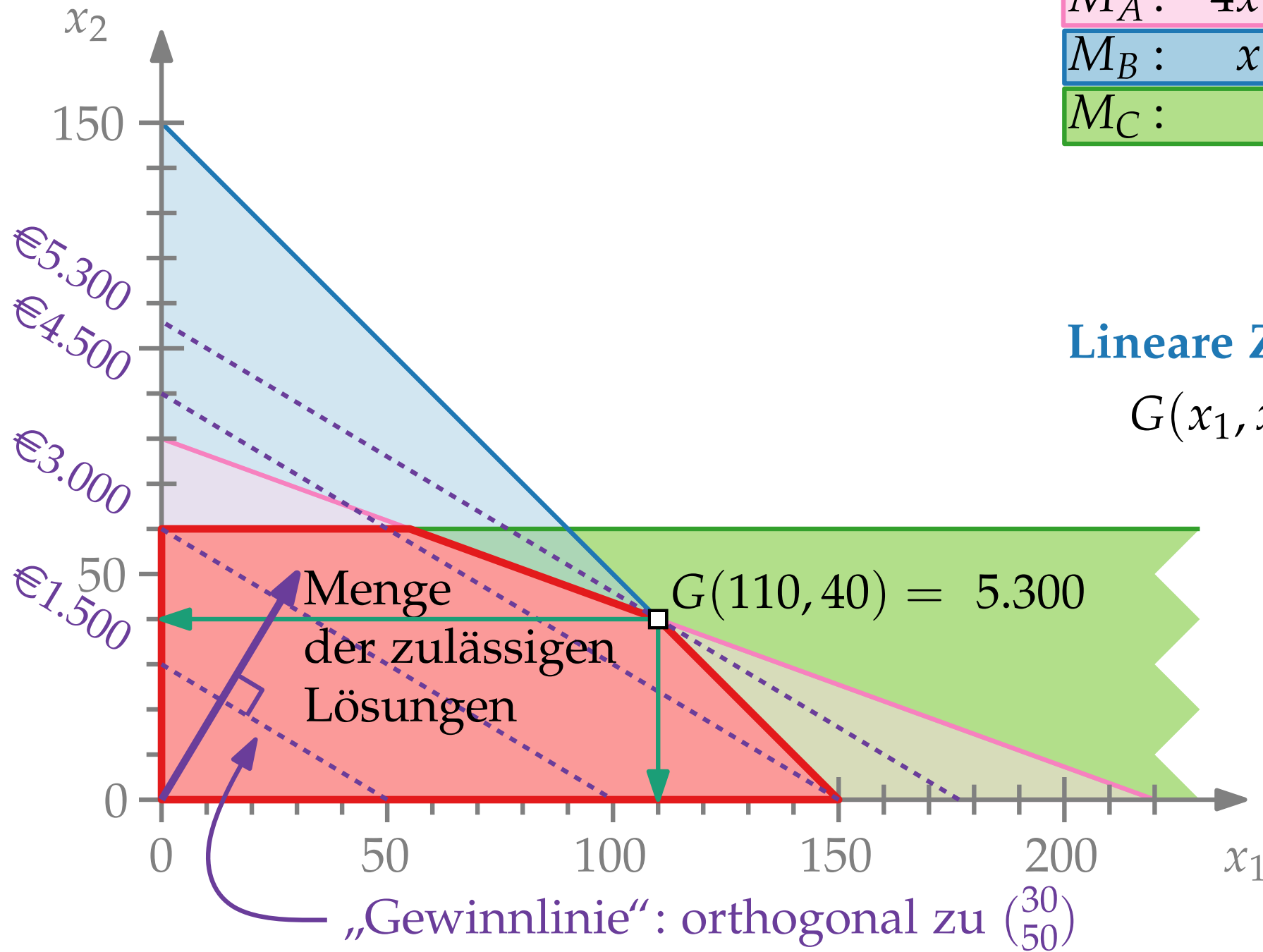
$$x_1 \geq 0$$

$$x_2 \geq 0$$

Lineare Zielfunktion:

$$\begin{aligned} G(x_1, x_2) &= 30x_1 + 50x_2 \\ &= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \end{aligned}$$

Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

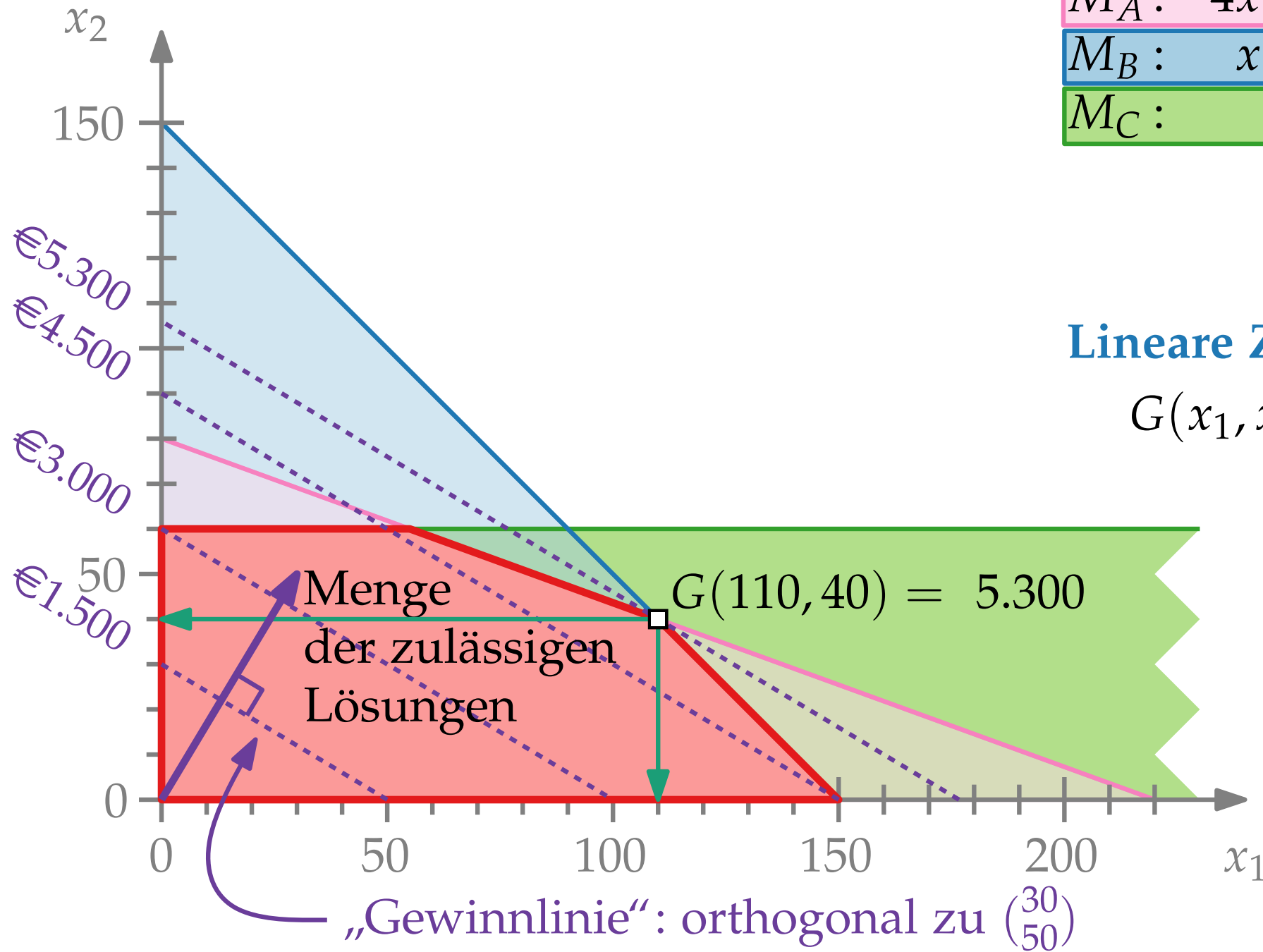
$$x_2 \geq 0$$

$$Ax \leq b$$

Lineare Zielfunktion:

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

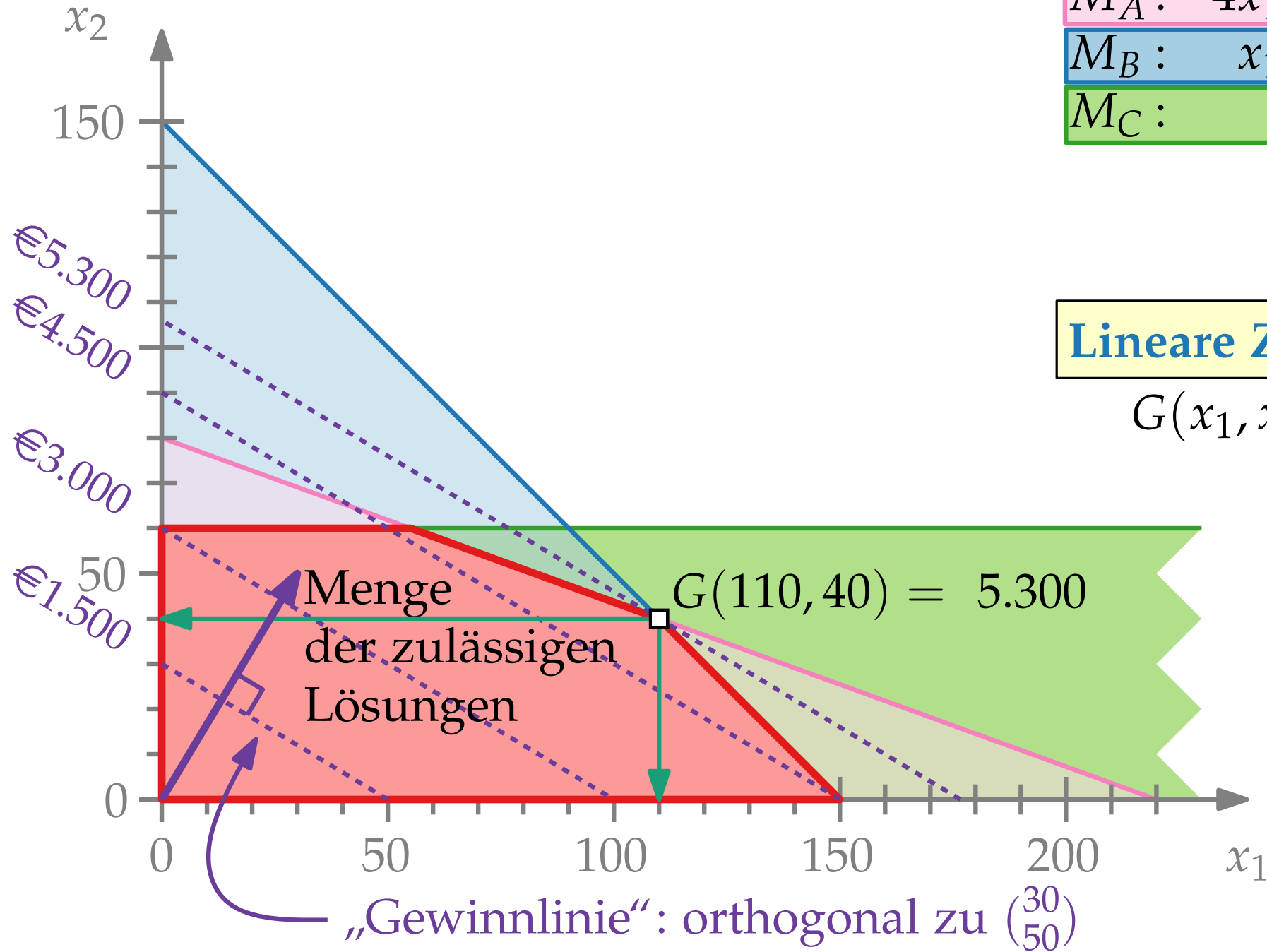
$$Ax \leq b$$

$$x \geq 0$$

Lineare Zielfunktion:

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

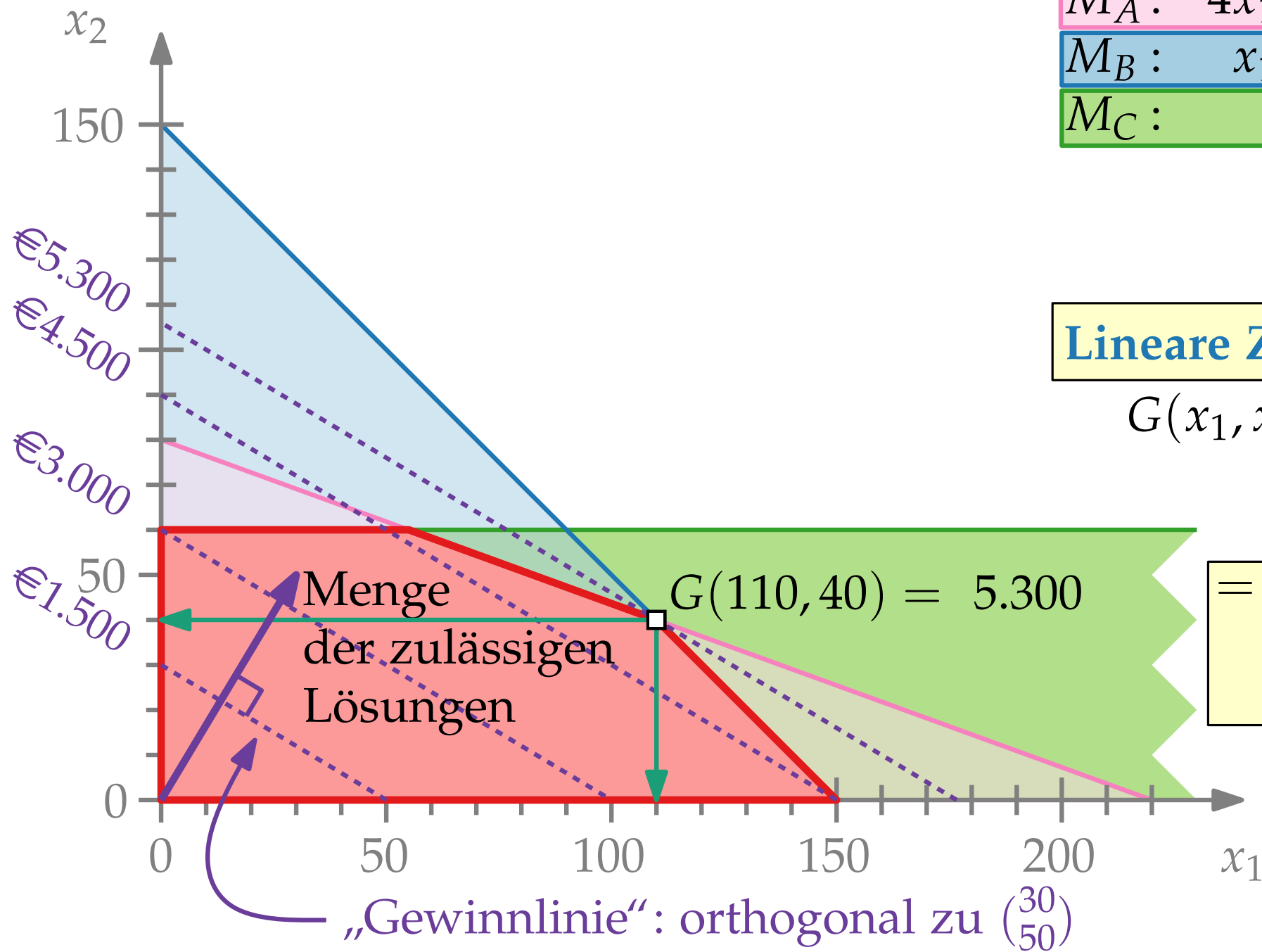
$$Ax \leq b$$

$$x \geq 0$$

Lineare Zielfunktion: maximiere $c^T x$

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$

$$Ax \leq b$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

$$x \geq 0$$

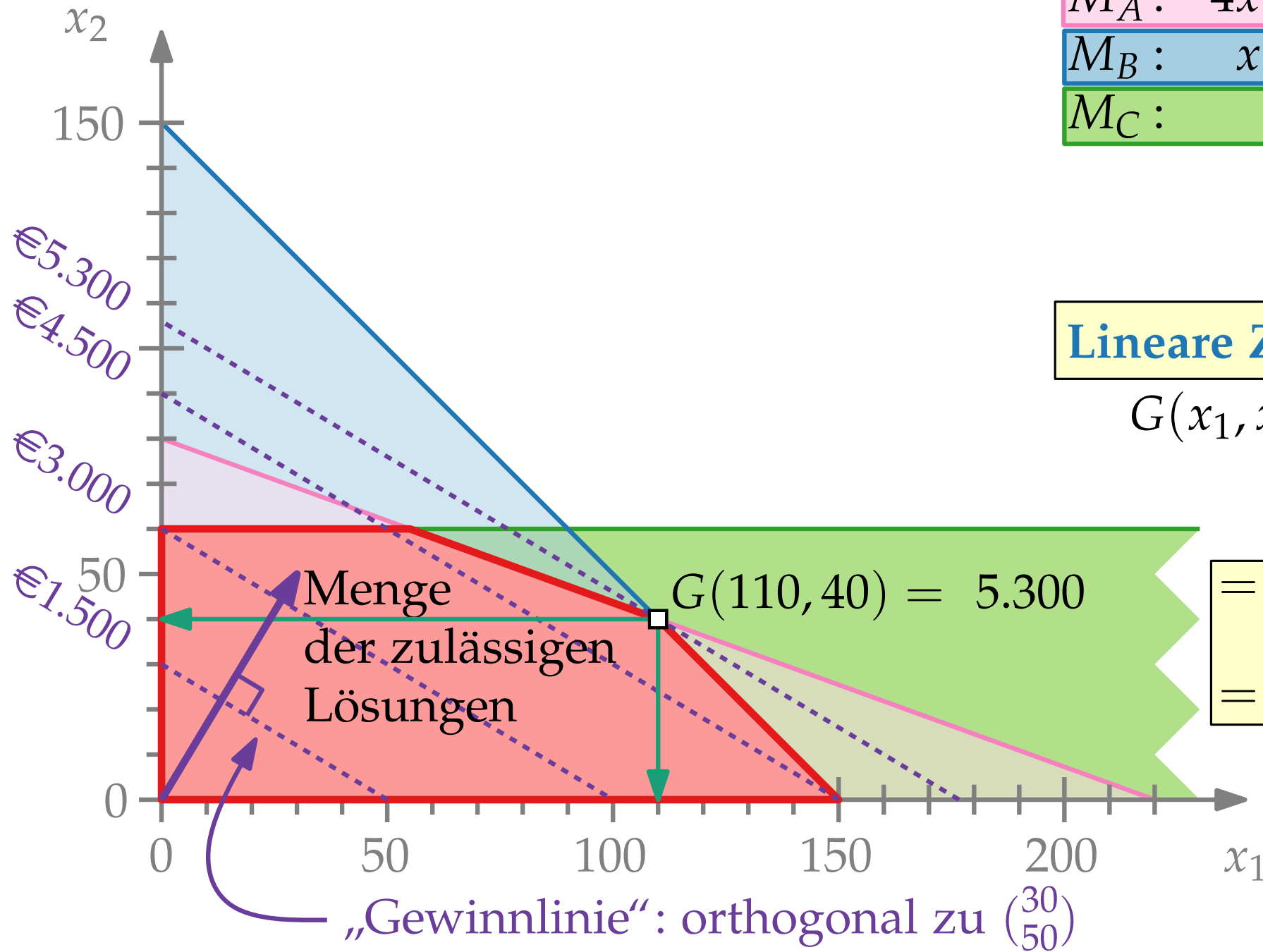
Lineare Zielfunktion: maximiere $c^T x$

$$G(x_1, x_2) = 30x_1 + 50x_2$$

$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

= maximaler Wert der Zielfunktion unter Beschränk.

Lösung



Lineare Beschränkungen:

$$M_A: 4x_1 + 11x_2 \leq 880$$

$$M_B: x_1 + x_2 \leq 150$$

$$M_C: x_2 \leq 60$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

$$Ax \leq b$$

$$x \geq 0$$

Lineare Zielfunktion: maximiere $c^T x$

$$G(x_1, x_2) = 30x_1 + 50x_2$$
$$= (30, 50) \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

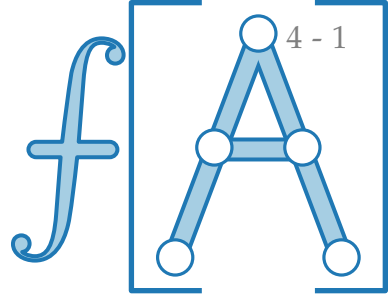
= maximaler Wert der Zielfunktion unter Beschränk.

$$= \max\{c^T x \mid Ax \leq b, x \geq 0\}$$

Lineares Programmieren I

Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

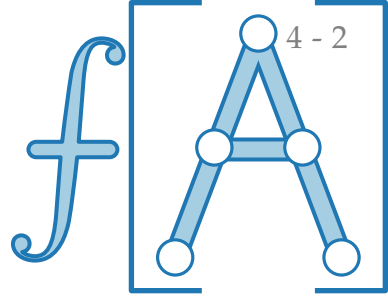
Gesucht:



Lineares Programmieren I

Gegeben: $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, c \in \mathbb{R}^n$

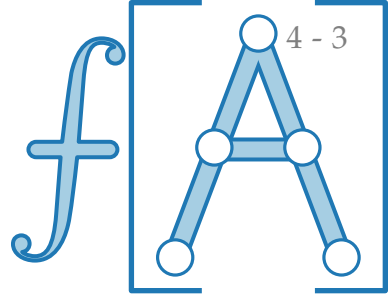
Gesucht: $x^* \in \mathbb{R}^n$



Lineares Programmieren I

Gegeben: $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

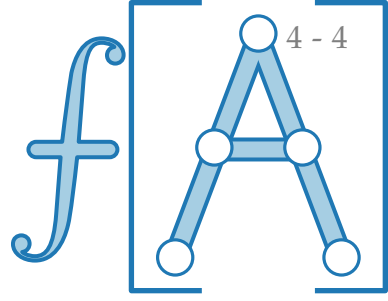


Lineares Programmieren I

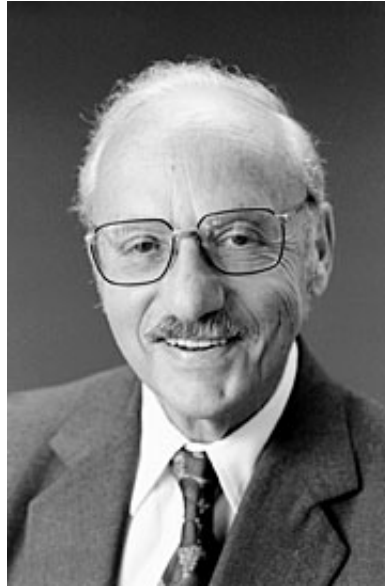
Gegeben: $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

Satz. Der Simplex-Algorithmus löst lineare Programme.
[Dantzig, 1947]



George Dantzig
*1914 Portland, OR
† 2005 Stanford, CA

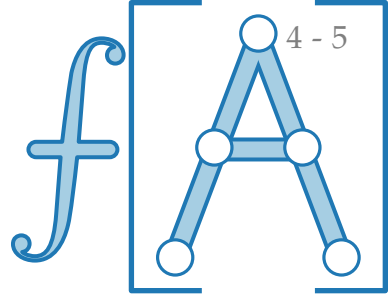
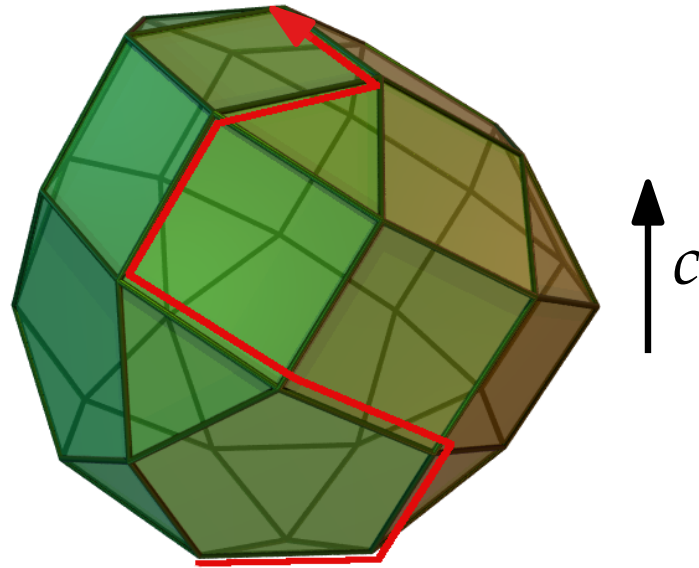


Lineares Programmieren I

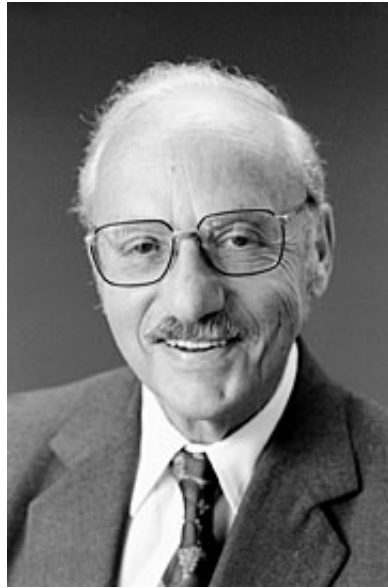
Gegeben: $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

Satz. Der Simplex-Algorithmus löst lineare Programme.
[Dantzig, 1947]



George Dantzig
*1914 Portland, OR
† 2005 Stanford, CA

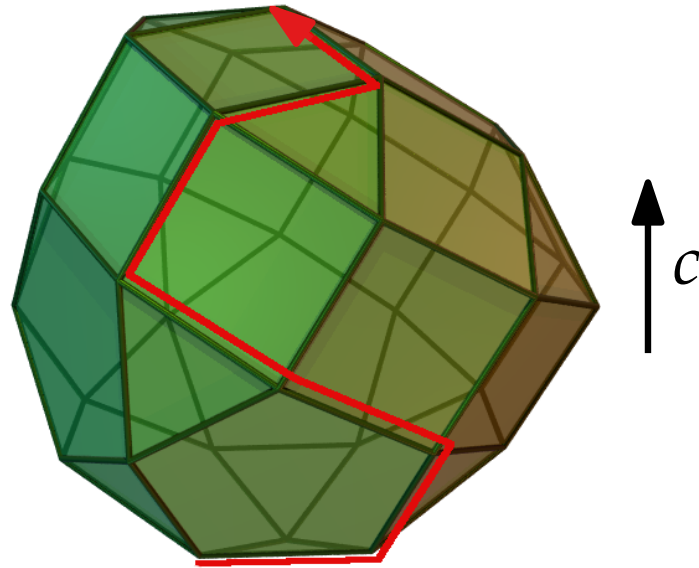


Lineares Programmieren I

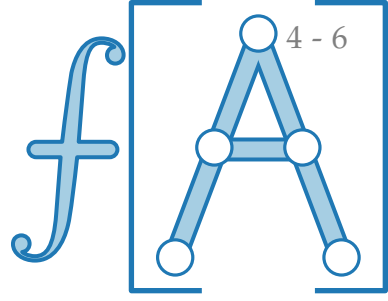
Gegeben: $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

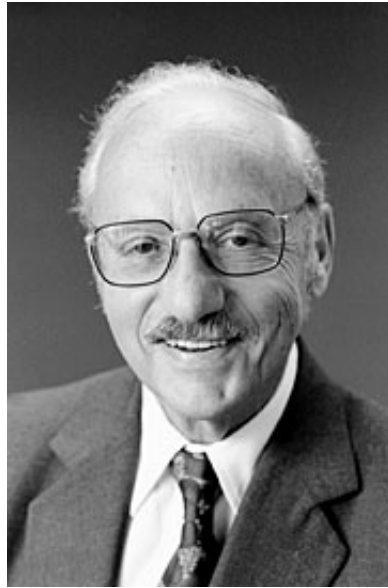
Satz. Der Simplex-Algorithmus löst lineare Programme.
[Dantzig, 1947]



Satz. Es gibt Beispiele, auf denen der Simplex-Algorithmus exponentielle Zeit benötigt.
[Klee & Minty, 1972]



George Dantzig
*1914 Portland, OR
† 2005 Stanford, CA

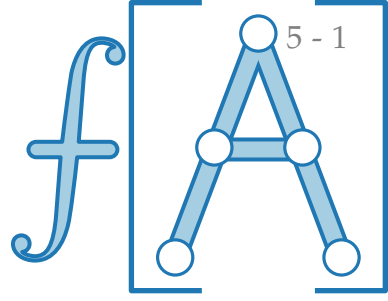


Lineares Programmieren II

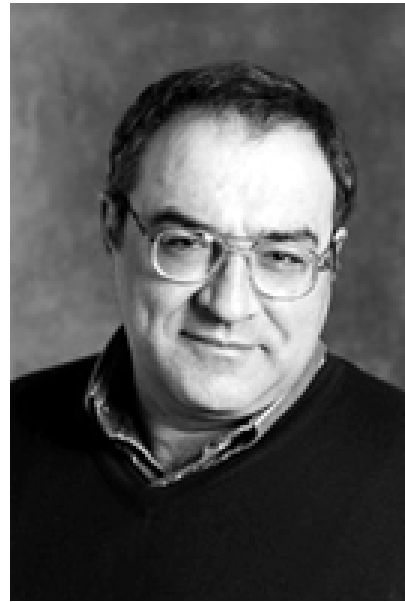
Satz.

Ein Lineares Programm der Dim. n lässt sich in
 $\mathcal{O}(L \cdot n^6)$ Zeit lösen.

[Khachiyan, 1979]



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ



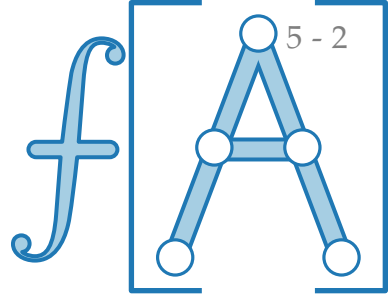
Lineares Programmieren II

Satz.

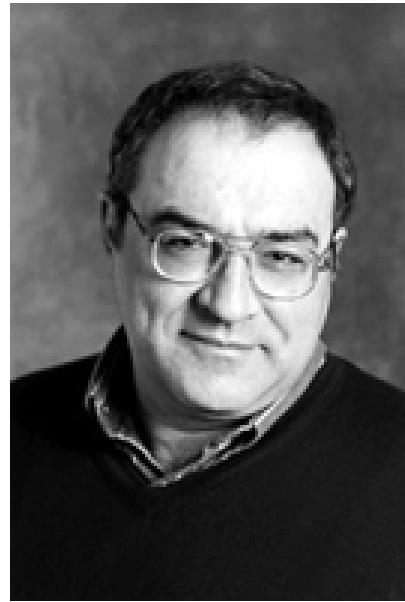
Ein Lineares Programm der Dim. n lässt sich in
 $\mathcal{O}(L \cdot n^6)$ Zeit lösen.

[Khachiyan, 1979]

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

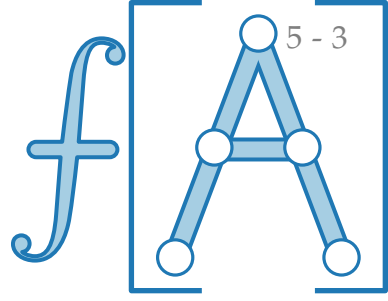


Lineares Programmieren II

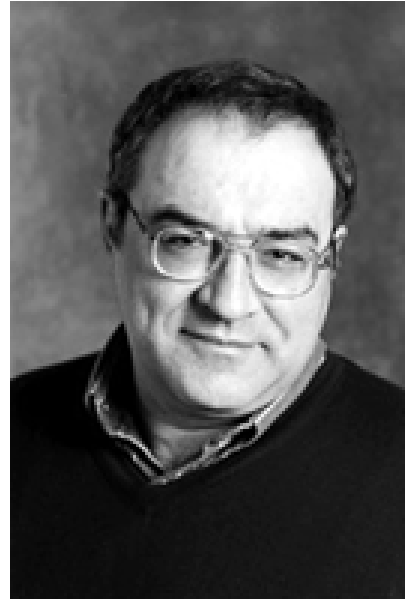
Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

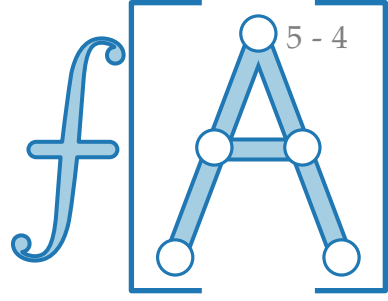
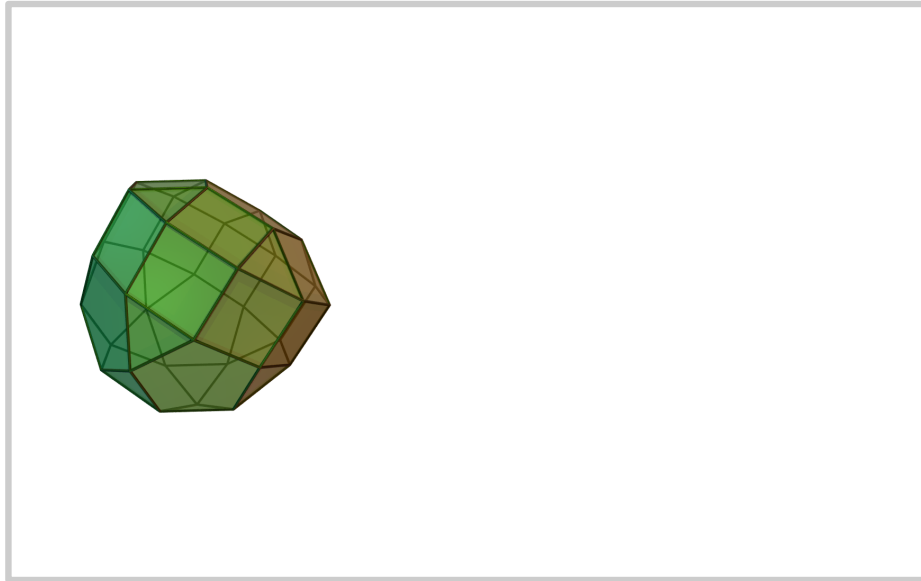


Lineares Programmieren II

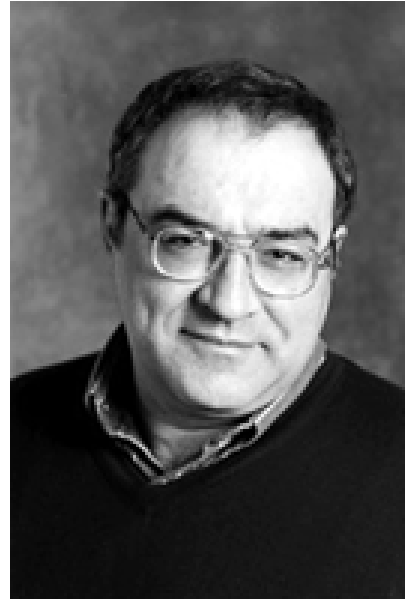
Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

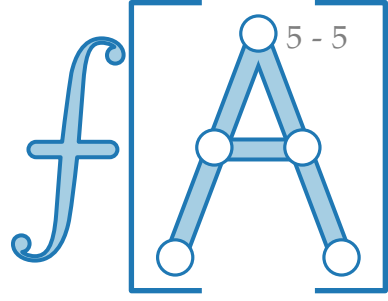
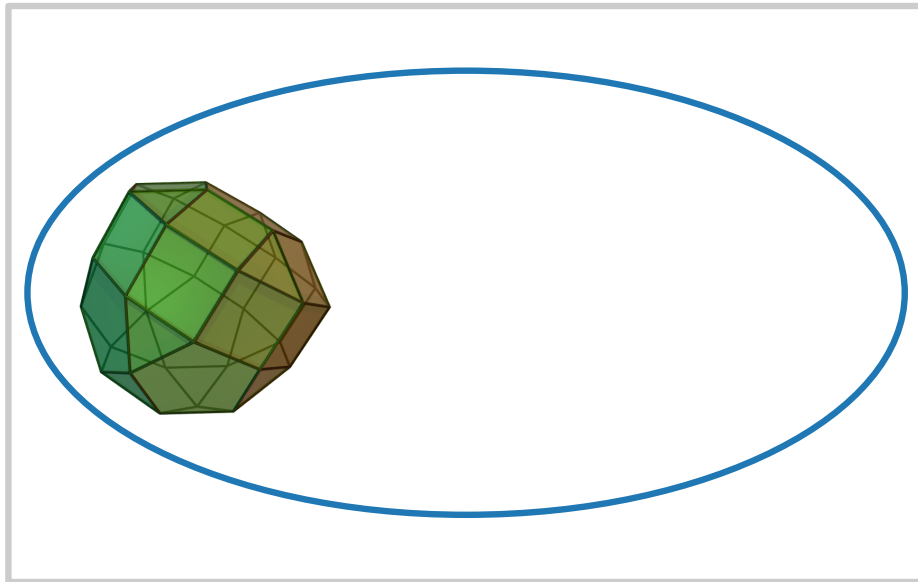


Lineares Programmieren II

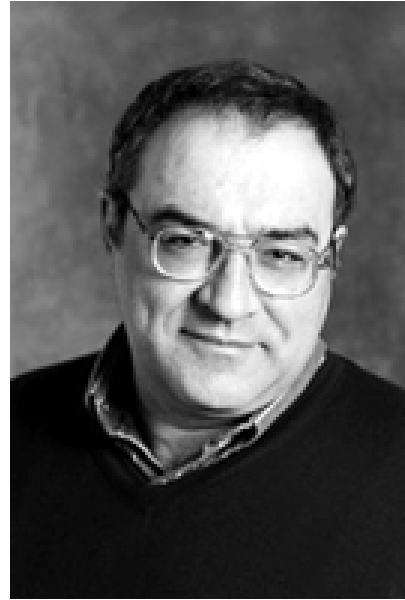
Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
+2005 South Brunswick, NJ

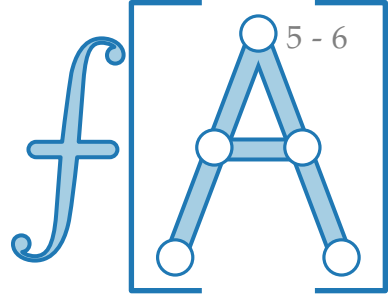
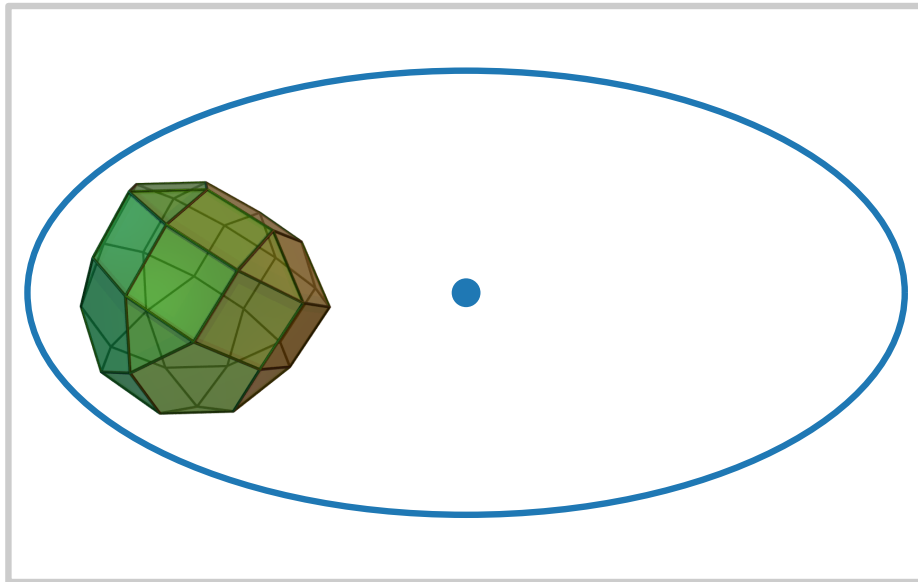


Lineares Programmieren II

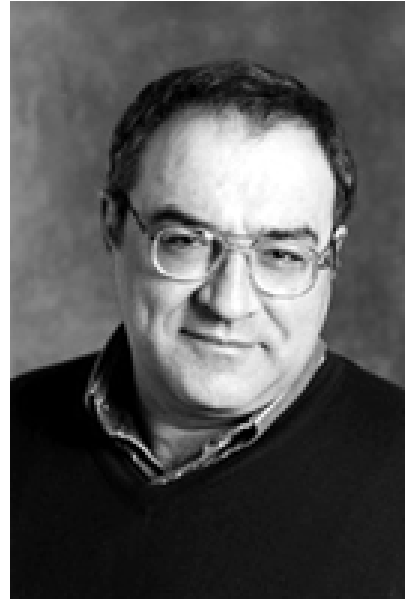
Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

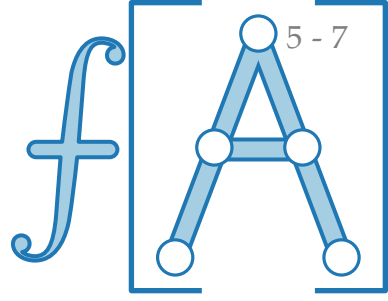
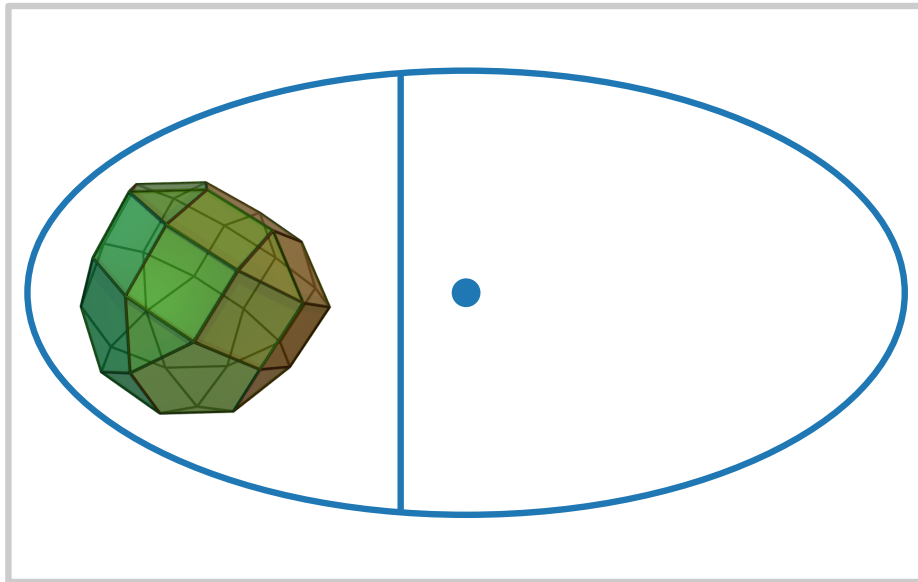


Lineares Programmieren II

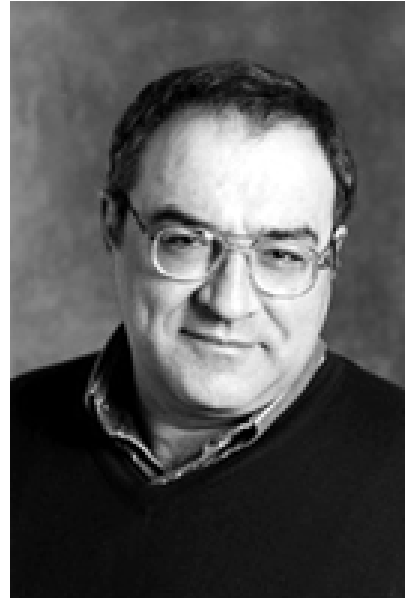
Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

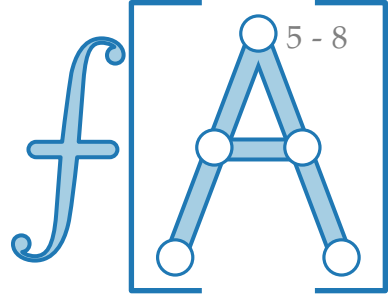
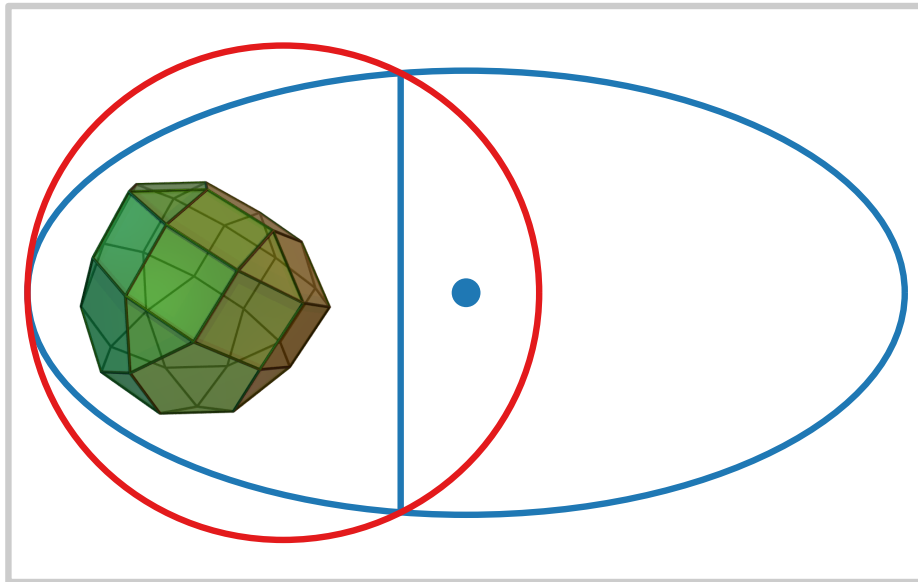


Lineares Programmieren II

Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

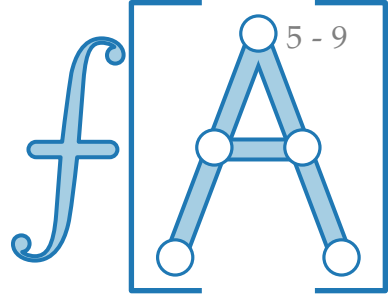
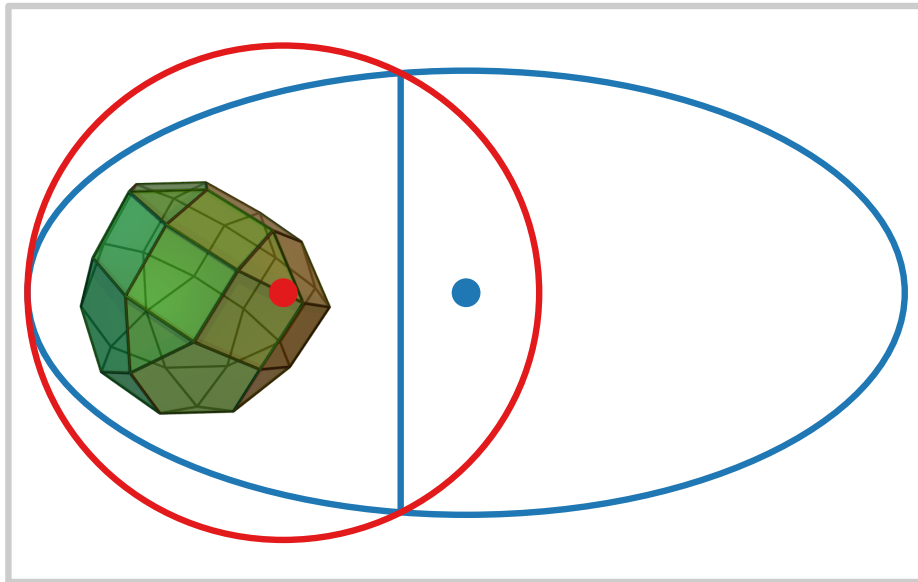


Lineares Programmieren II

Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ

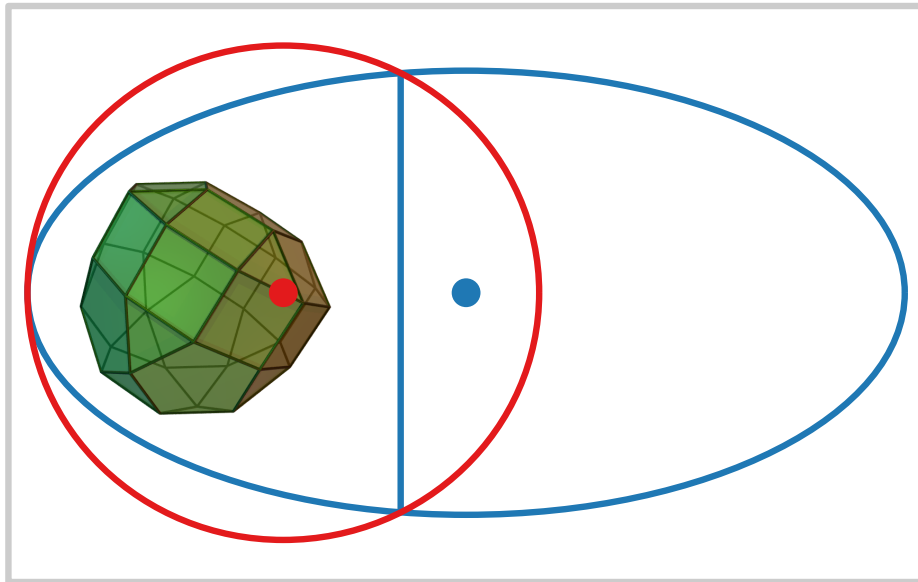


Lineares Programmieren II

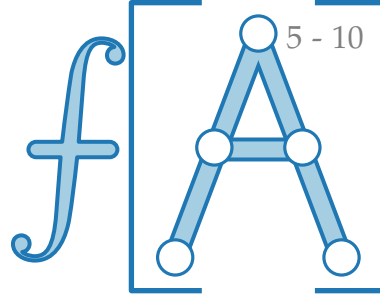
Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei $L = \text{Anz. Bits in der Eingabe.}$



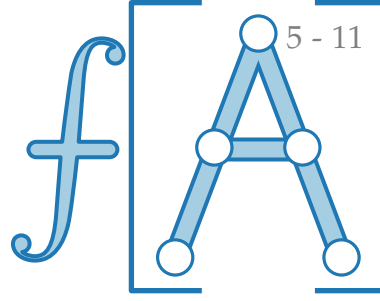
Realität: dramatisch langsamer



Leonid Khachiyan
*1952 Leningrad
†2005 South Brunswick, NJ



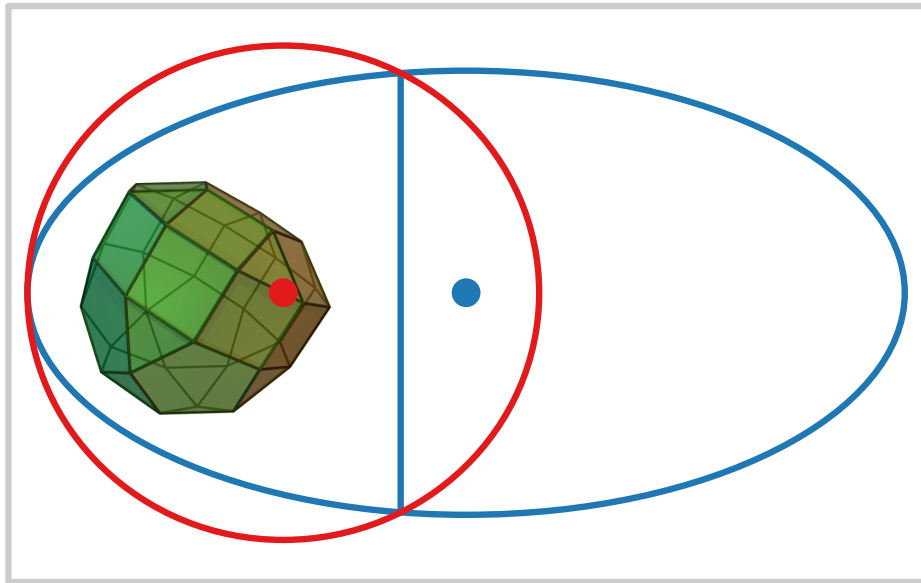
Lineares Programmieren II



Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei L = Anz. Bits in der Eingabe.



Realität: dramatisch langsamer

Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^{3.5})$ Zeit *numerisch stabil* lösen. [Karmakar, 1984]

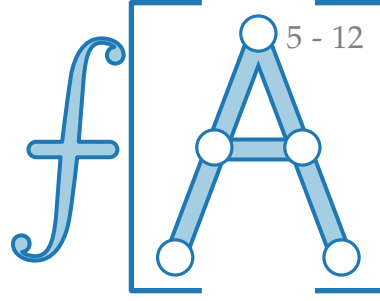
Leonid Khachiyan
*1952 Leningrad
+2005 South Brunswick, NJ



Narendra Karmarkar
*1957 Gwalior, Indien



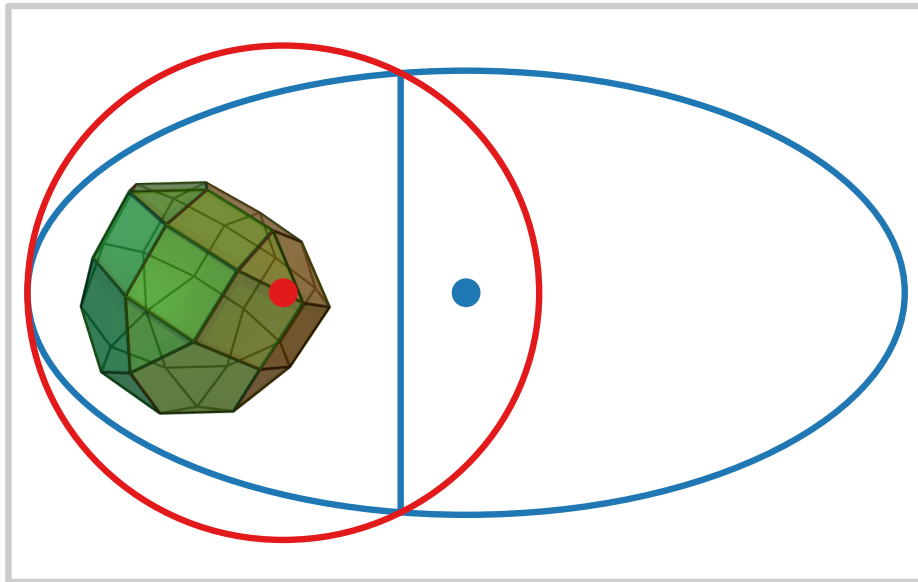
Lineares Programmieren II



Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei L = Anz. Bits in der Eingabe.



Realität: dramatisch langsamer

Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^{3.5})$ Zeit *numerisch stabil* lösen. [Karmakar, 1984]

Innere-Punkt-Methode

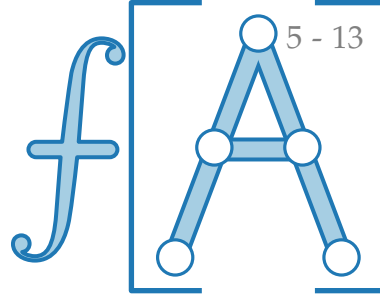
Leonid Khachiyan
*1952 Leningrad
+2005 South Brunswick, NJ



Narendra Karmarkar
*1957 Gwalior, Indien



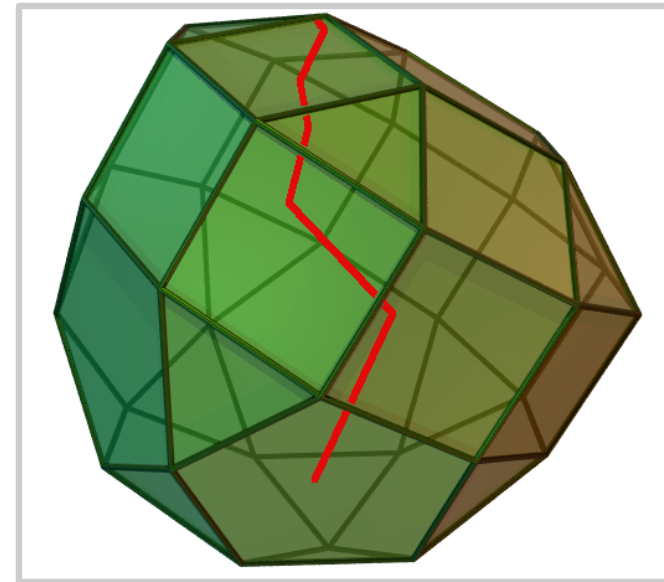
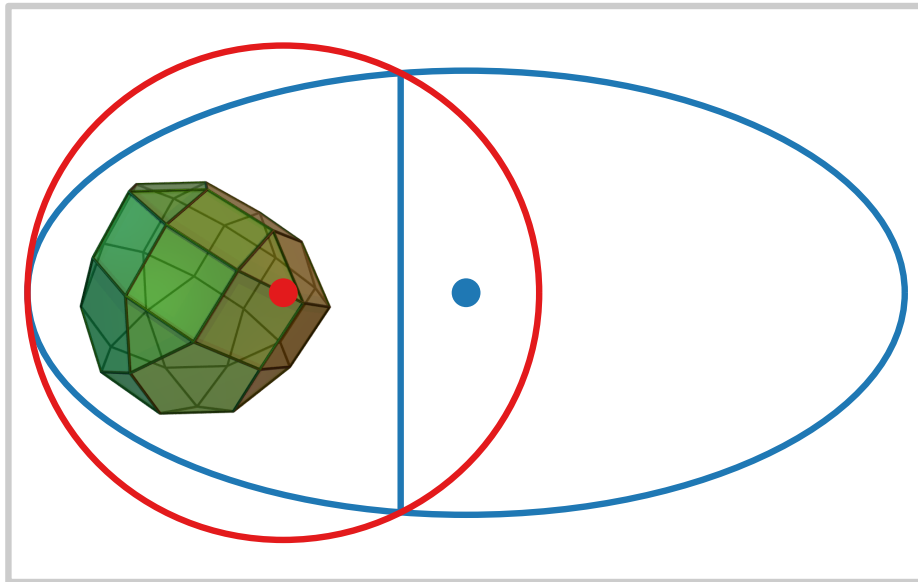
Lineares Programmieren II



Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei L = Anz. Bits in der Eingabe.



Realität: dramatisch langsamer

Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^{3.5})$ Zeit *numerisch stabil* lösen. [Karmakar, 1984]

Innere-Punkt-Methode

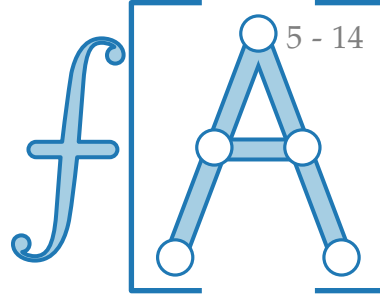
Leonid Khachiyan
*1952 Leningrad
+2005 South Brunswick, NJ



Narendra Karmarkar
*1957 Gwalior, Indien



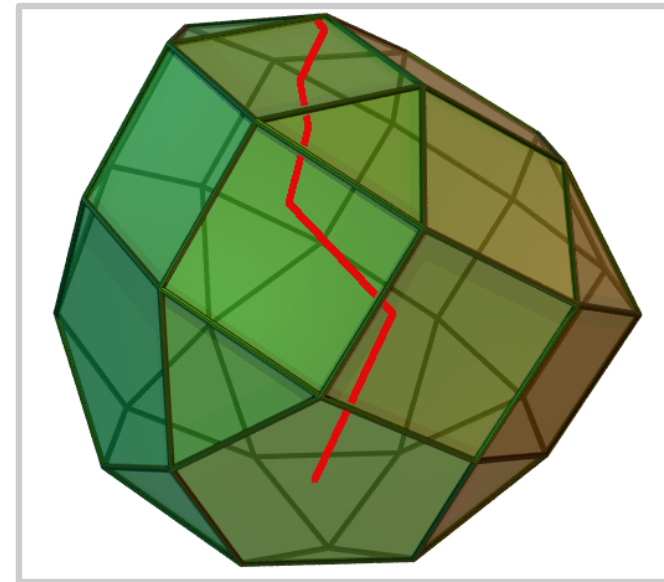
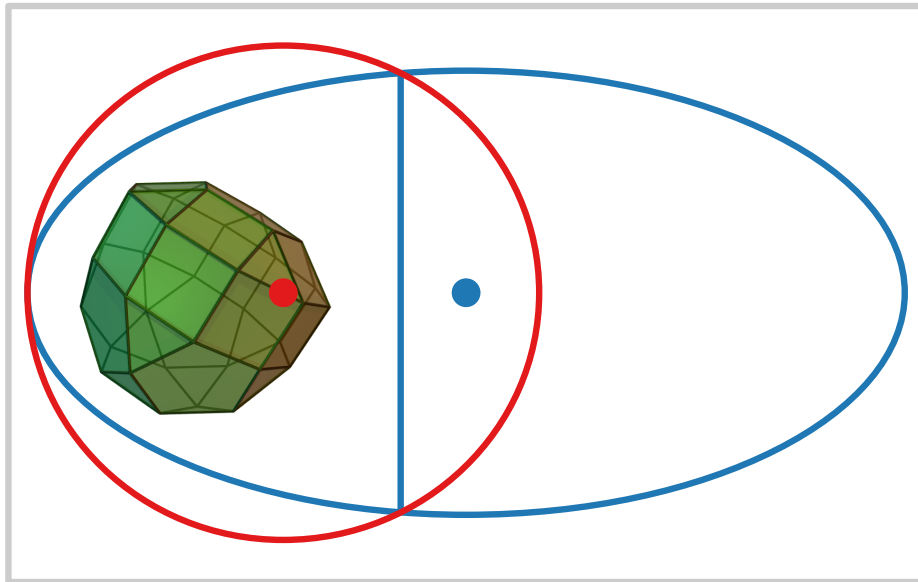
Lineares Programmieren II



Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^6)$ Zeit lösen. [Khachiyan, 1979]

Ellipsoid-Methode

wobei L = Anz. Bits in der Eingabe.



Realität: dramatisch langsamer

Satz. Ein Lineares Programm der Dim. n lässt sich in $\mathcal{O}(L \cdot n^{3.5})$ Zeit *numerisch stabil* lösen. [Karmakar, 1984]

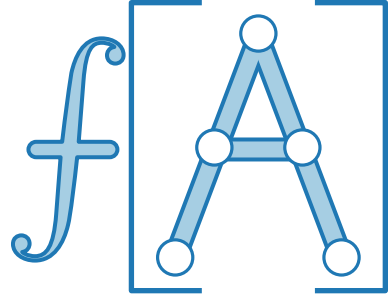
Innere-Punkt-Methode Realität: Konkurrenzfähig mit Simplex

Leonid Khachiyan
*1952 Leningrad
+2005 South Brunswick, NJ



Narendra Karmarkar
*1957 Gwalior, Indien

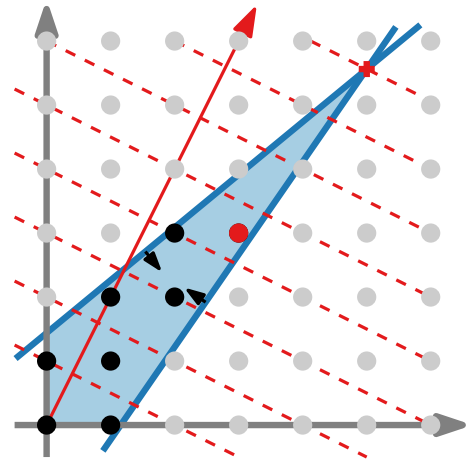




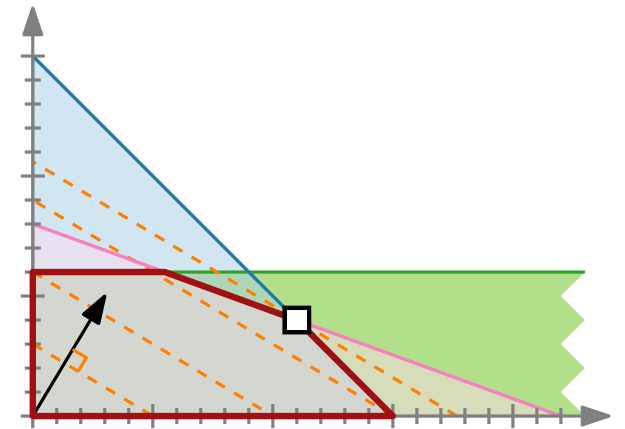
Fortgeschrittene Algorithmen

22. Vorlesung ILP & MaxSAT Solver

$$\begin{pmatrix} \text{blue} & \text{green} & \text{teal} \\ 0 & \dots & 0 & 1 & 0 & \dots & 0 \\ \text{red} & \text{pink} & \text{purple} \end{pmatrix}$$



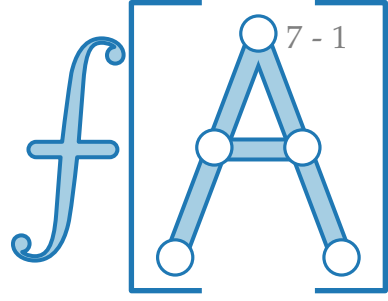
Teil II:
Ganzzahlige lineare Programmierung
& MaxSAT



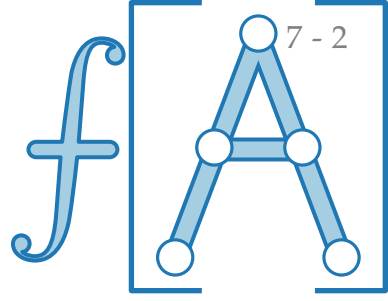
Ganzzahlige lineare Programmierung (ILP)

Gegeben: $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$



Ganzzahlige lineare Programmierung (ILP)



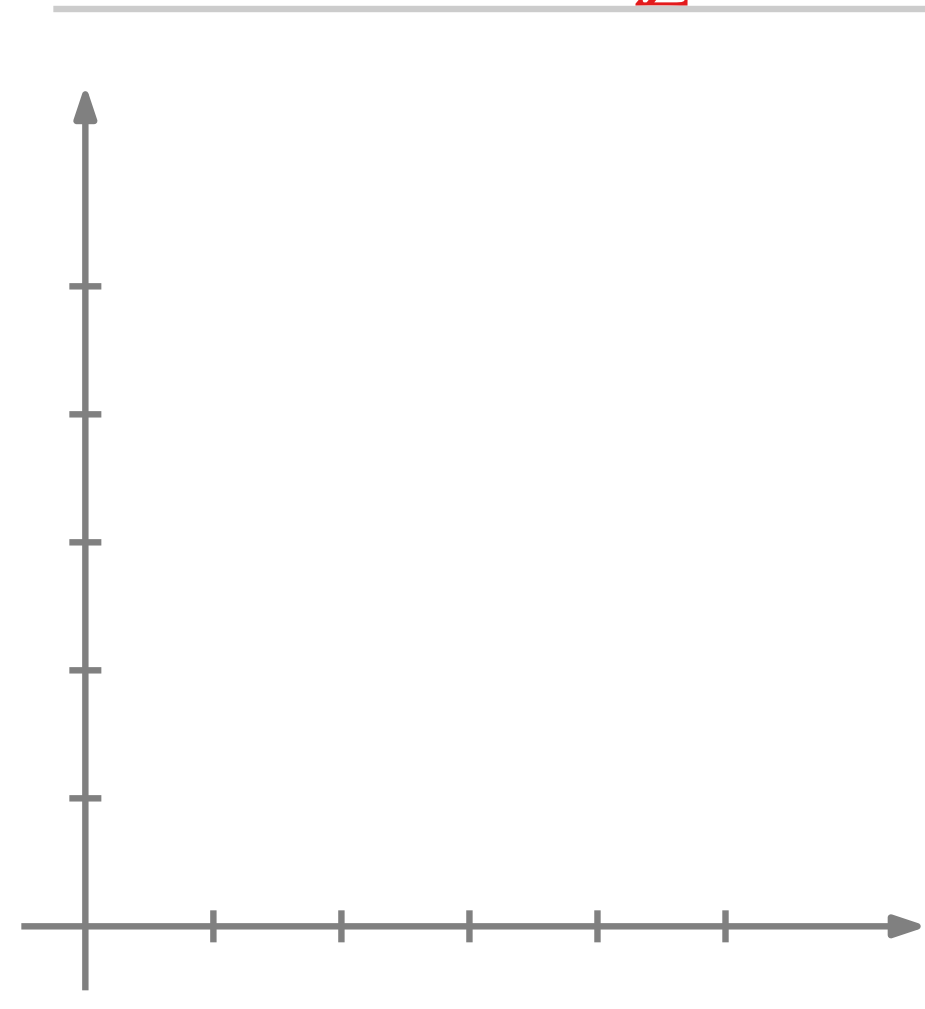
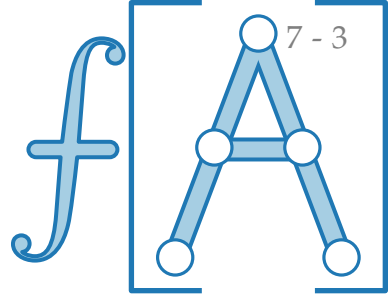
Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$ $x \in \mathbb{Z}^n$

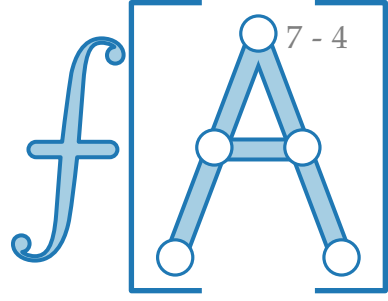
Ganzzahlige lineare Programmierung (ILP)

Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

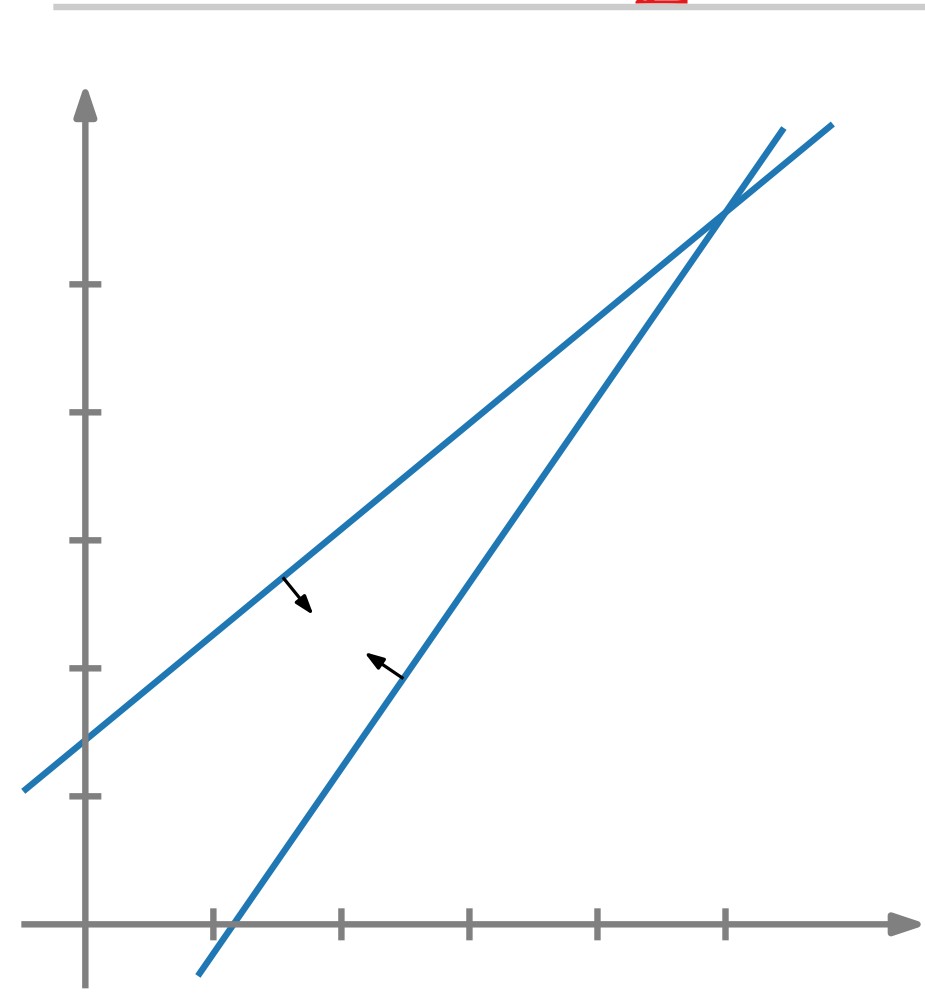


Ganzzahlige lineare Programmierung (ILP)

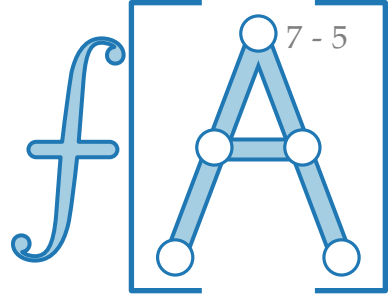


Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

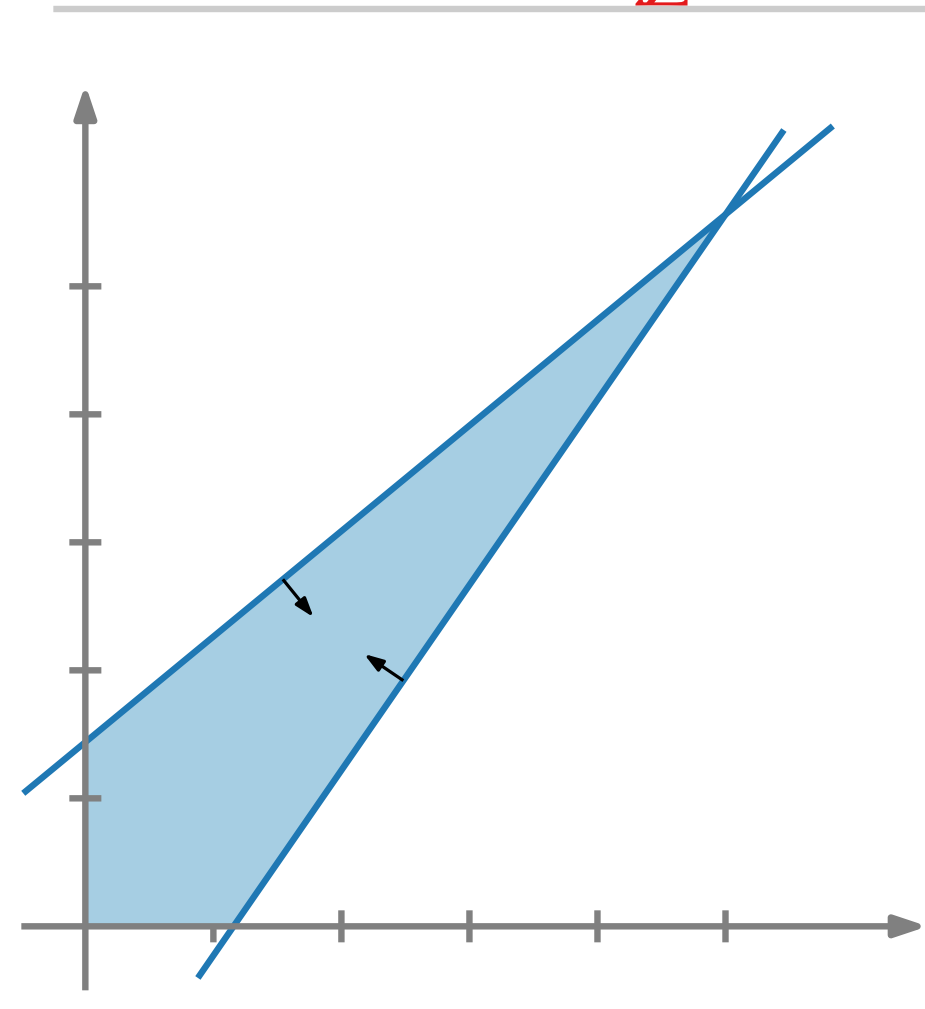


Ganzzahlige lineare Programmierung (ILP)

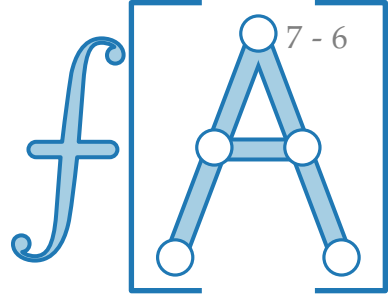


Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

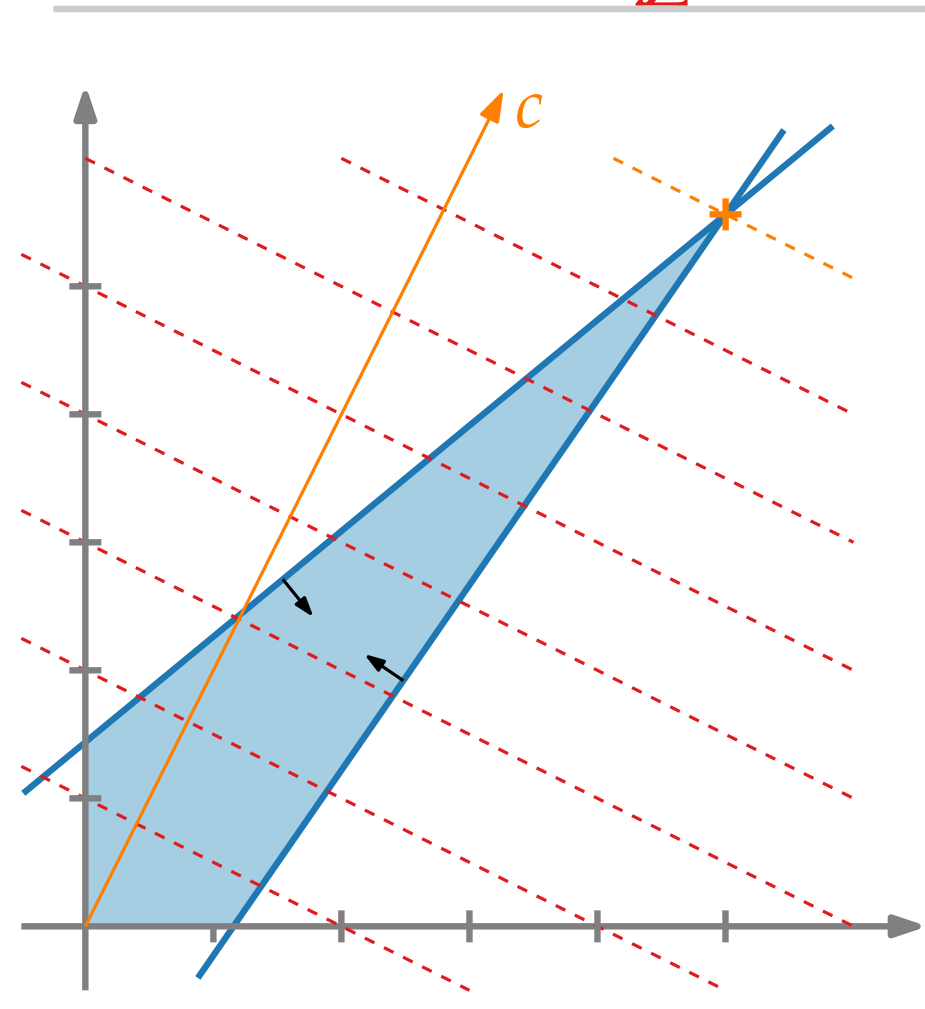


Ganzzahlige lineare Programmierung (ILP)

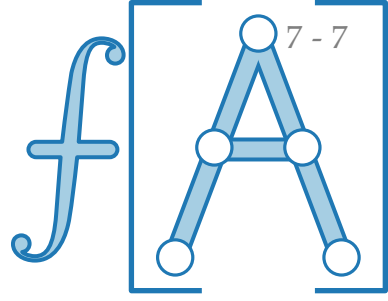


Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

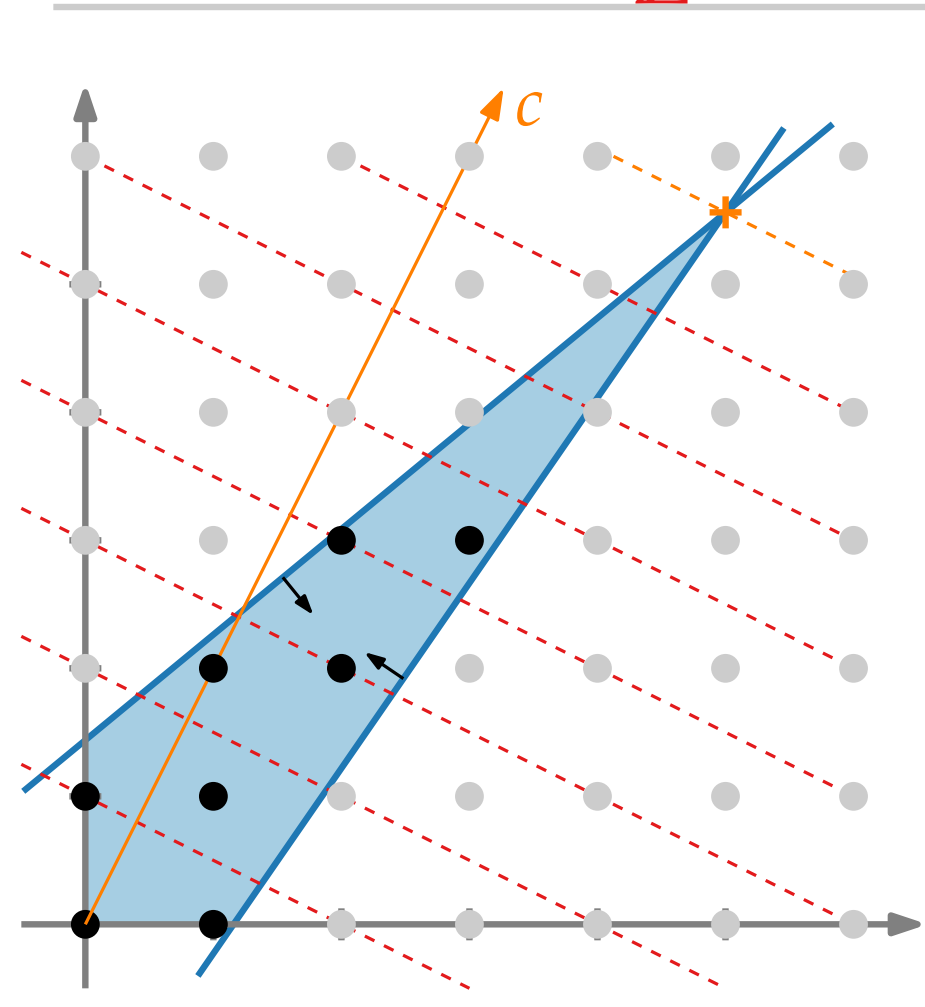


Ganzzahlige lineare Programmierung (ILP)

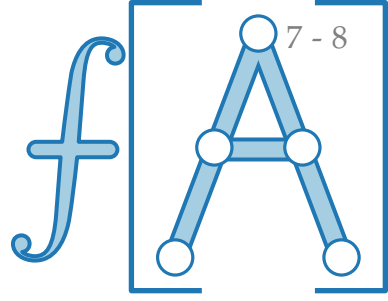


Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$

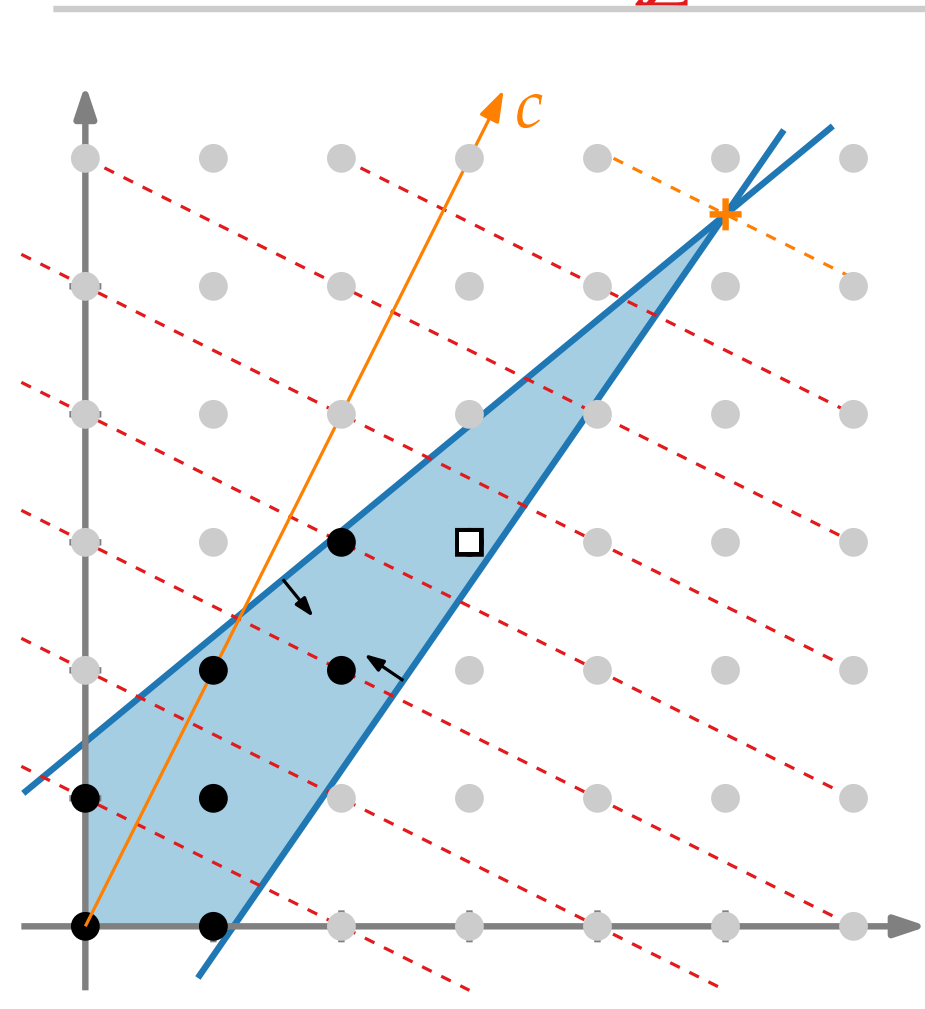


Ganzzahlige lineare Programmierung (ILP)

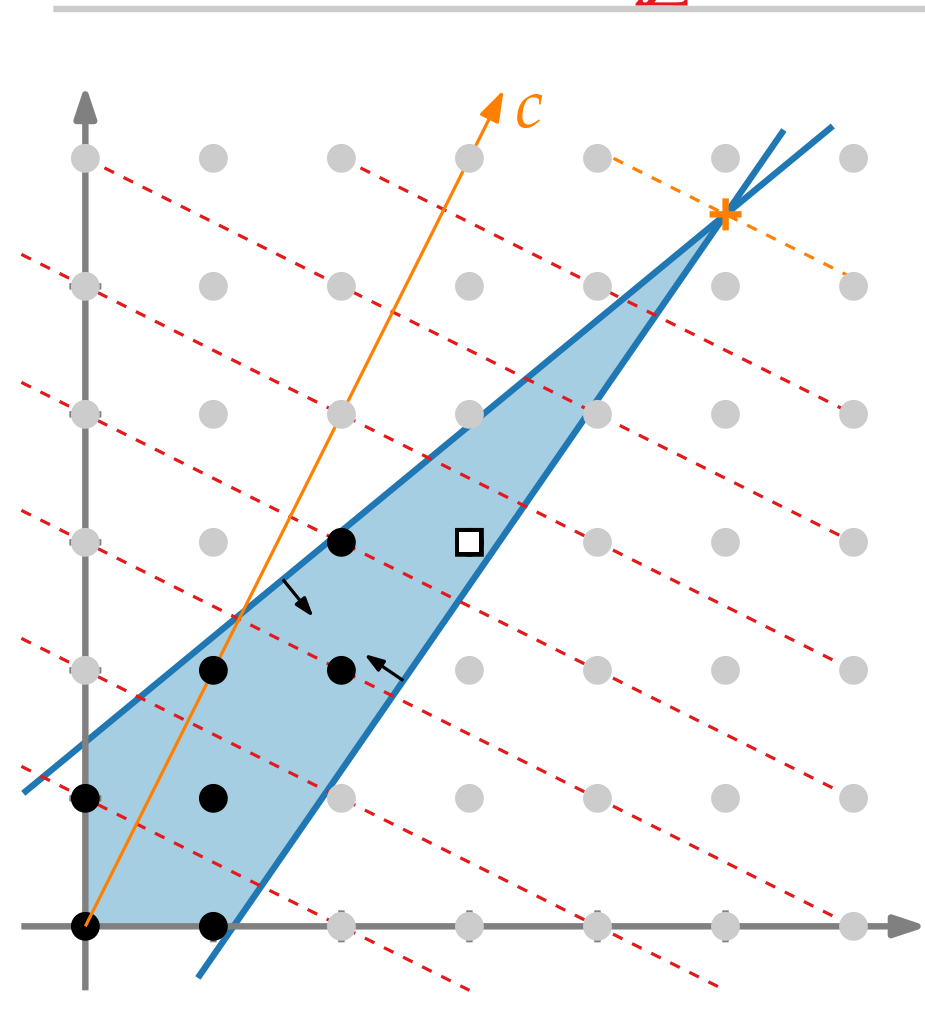


Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

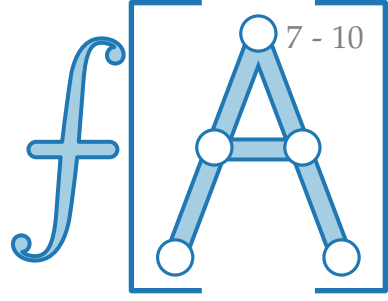
Gesucht: $x^* \in \cancel{\mathbb{R}^n}^{\mathbb{Z}^n}$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$



Gesucht: häufig $\{0, 1\}^n$
 $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
 $\mathbb{Z}^n$ $x \in \mathbb{Z}^n$

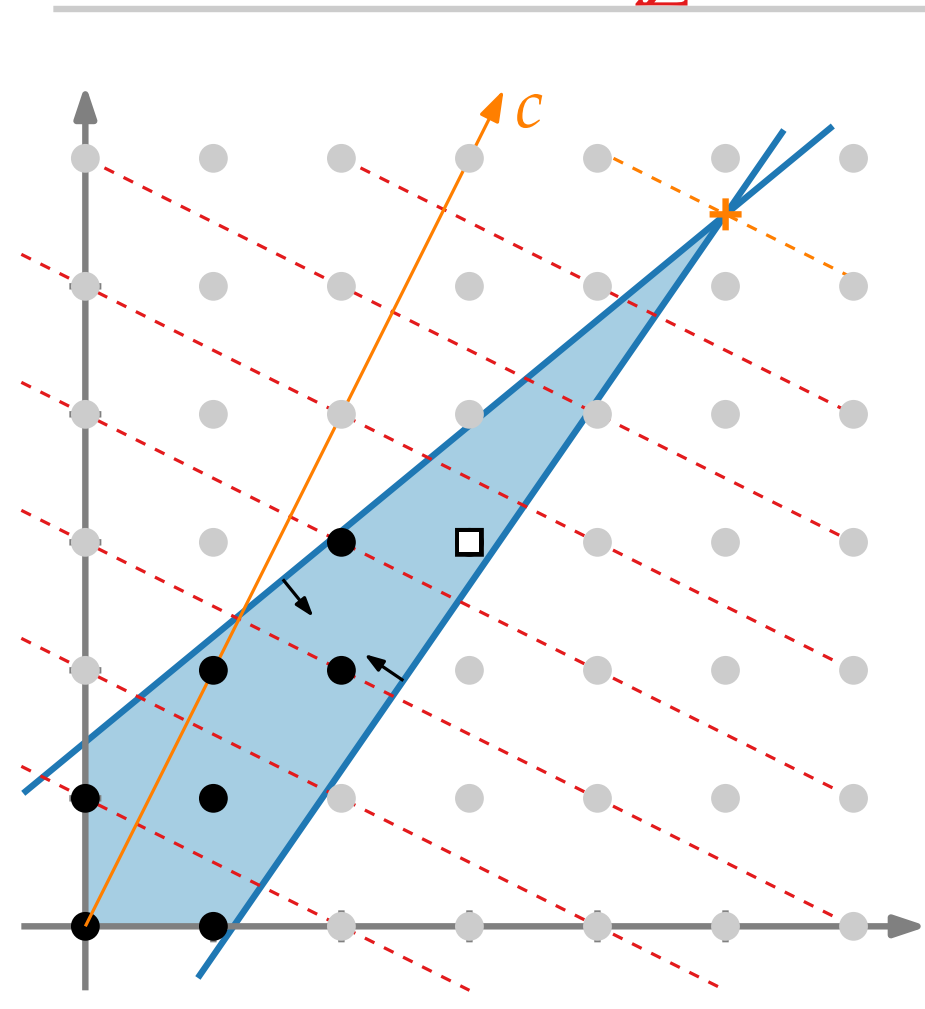


Ganzzahlige lineare Programmierung (ILP)

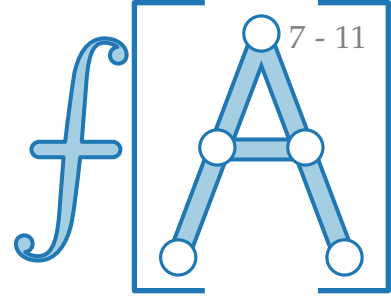


Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ ~~$\mathbb{Z}^n$~~ oder $\arg \min$ $x \in \mathbb{Z}^n$

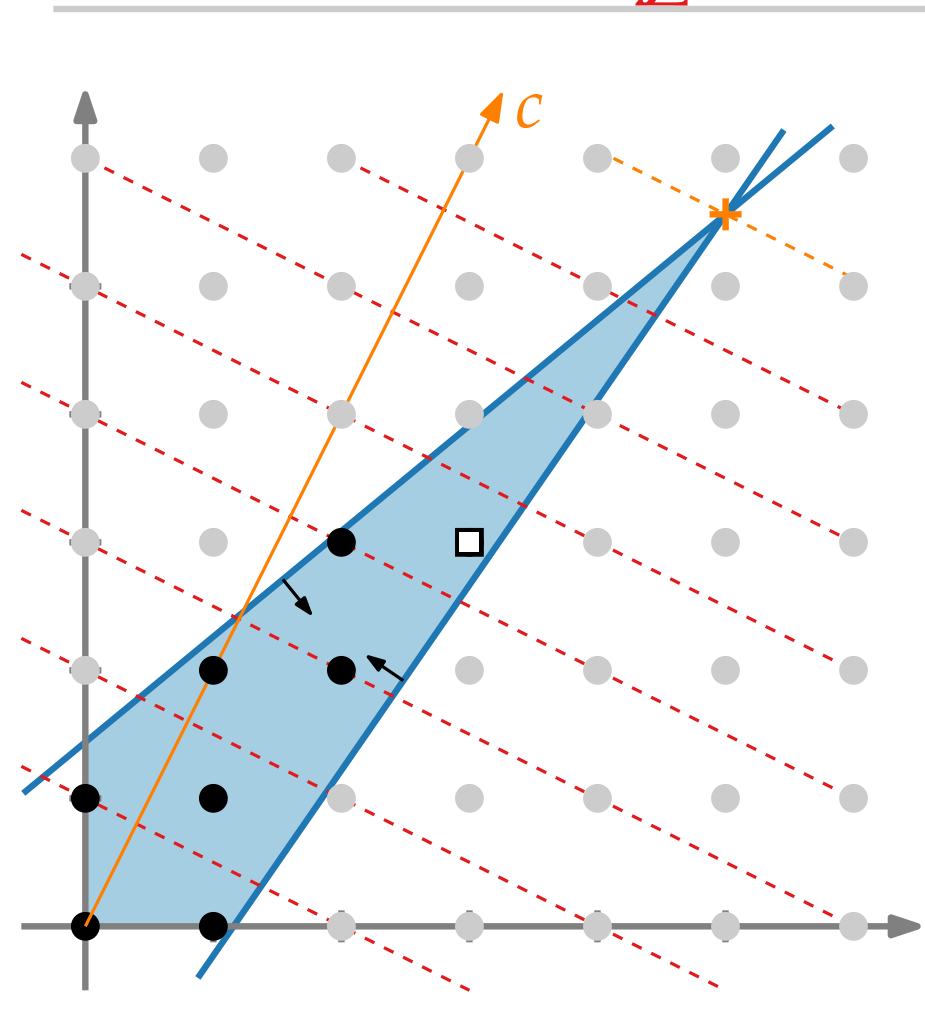


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ ~~$\mathbb{Z}^n$~~ oder $\arg \min$ $x \in \mathbb{Z}^n$

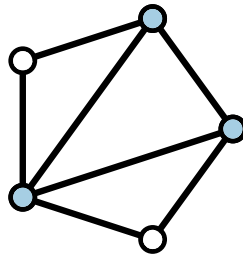


Problem. VERTEXCOVER

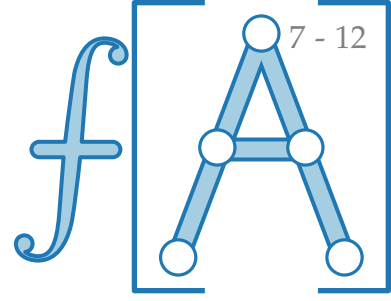
Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

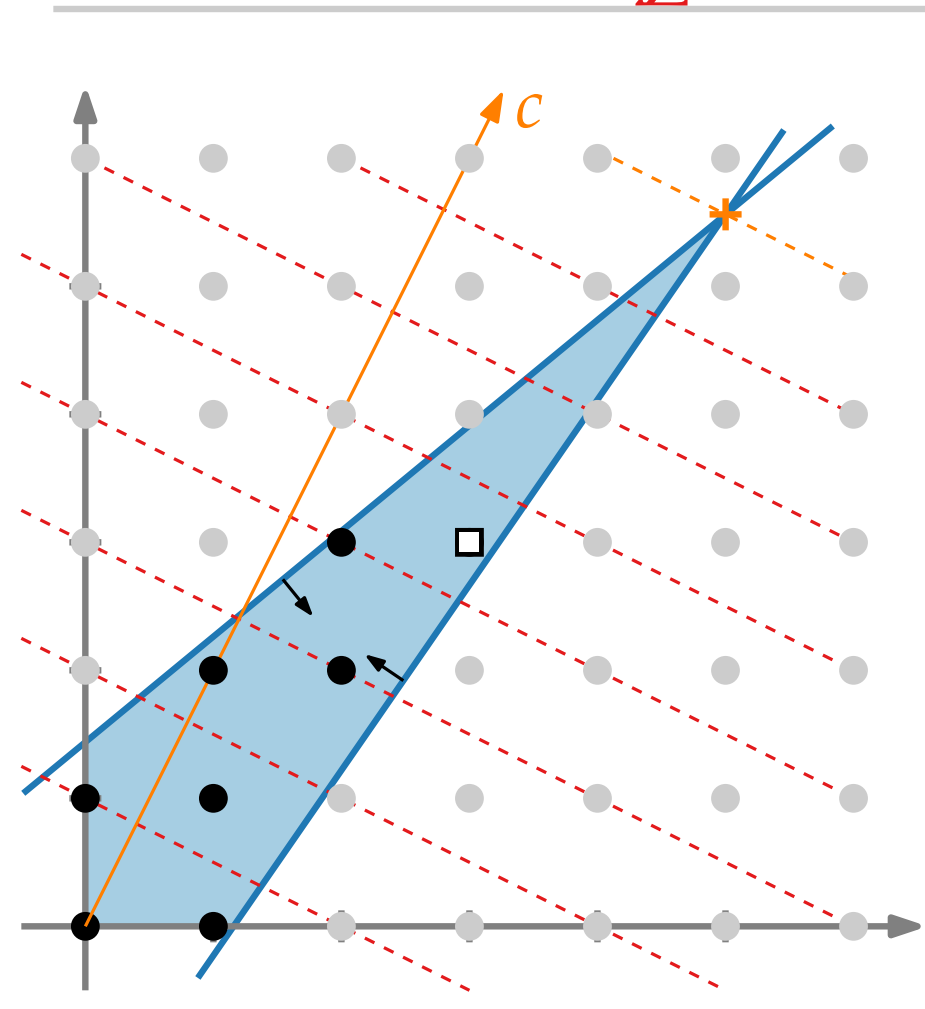


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



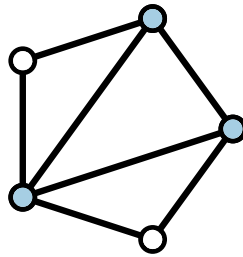
Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

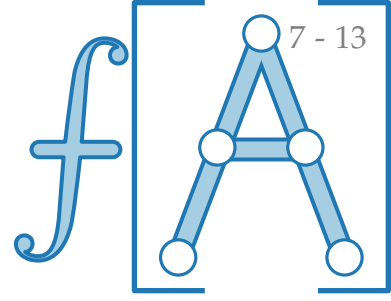
Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

Aufgabe. Modellieren Sie dieses
Problem als ILP!

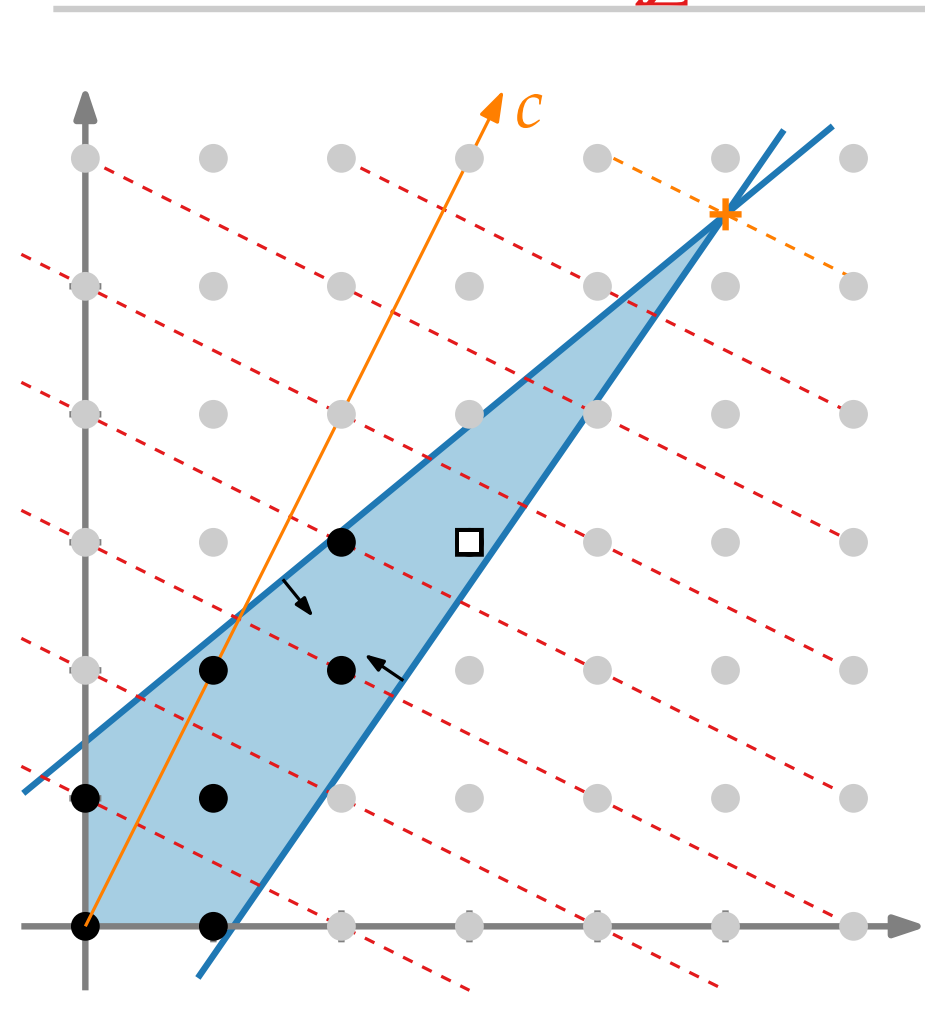


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



Problem. VERTEXCOVER

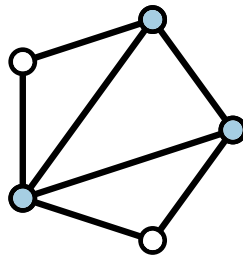
Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

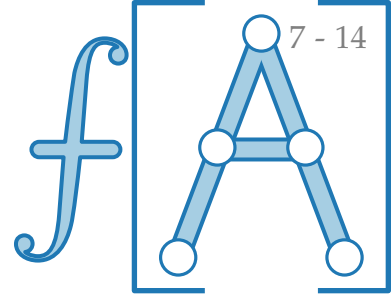
Ziel. $|V'|$ minimal!

Aufgabe. Modellieren Sie dieses
Problem als ILP!

Tipp. Verwenden Sie für jeden
Knoten v eine Variable
 $x_v \in \{0,1\}$, mit
 $x_v = 1 \Leftrightarrow v \in V'$

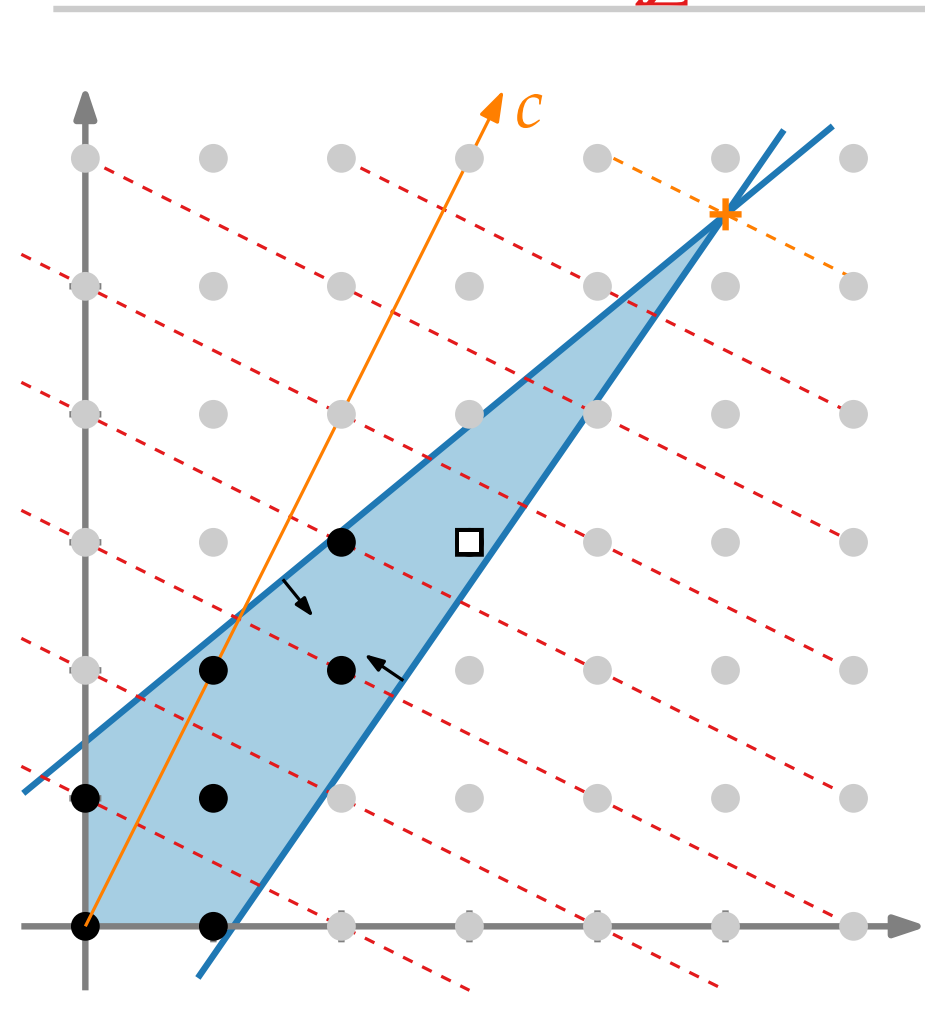


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



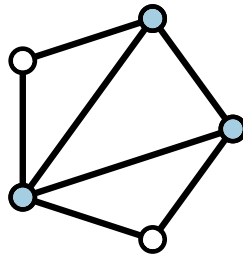
Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

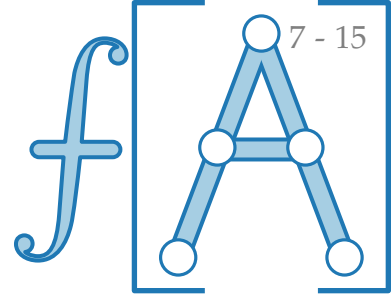
Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

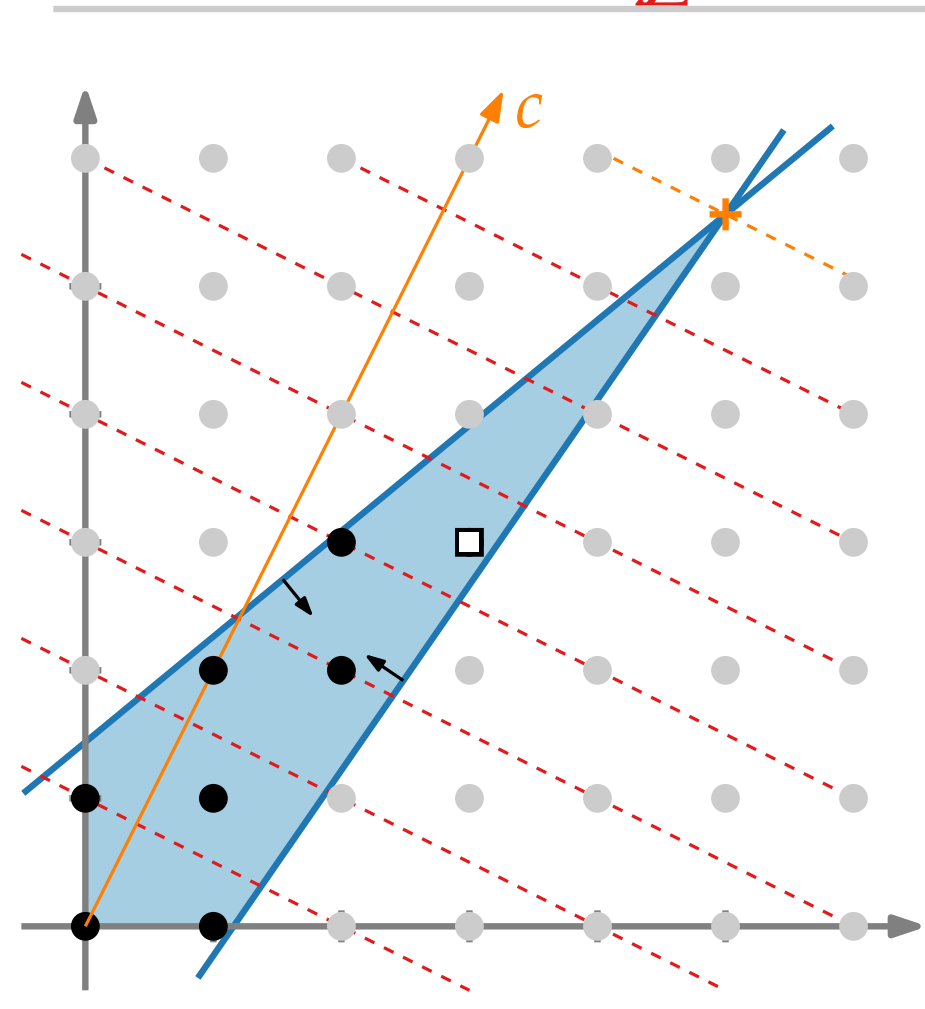


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ ~~$\mathbb{Z}^n$~~ oder $\arg \min$ $x \in \mathbb{Z}^n$



Problem. VERTEXCOVER

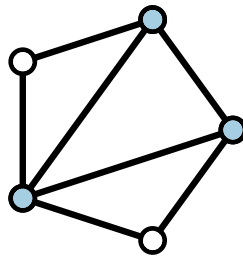
Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

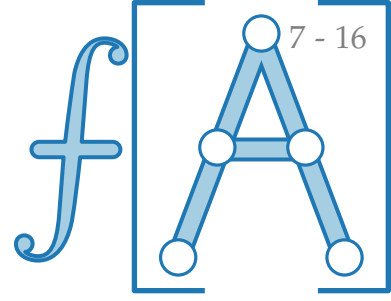
Ziel. $|V'|$ minimal!

Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel:

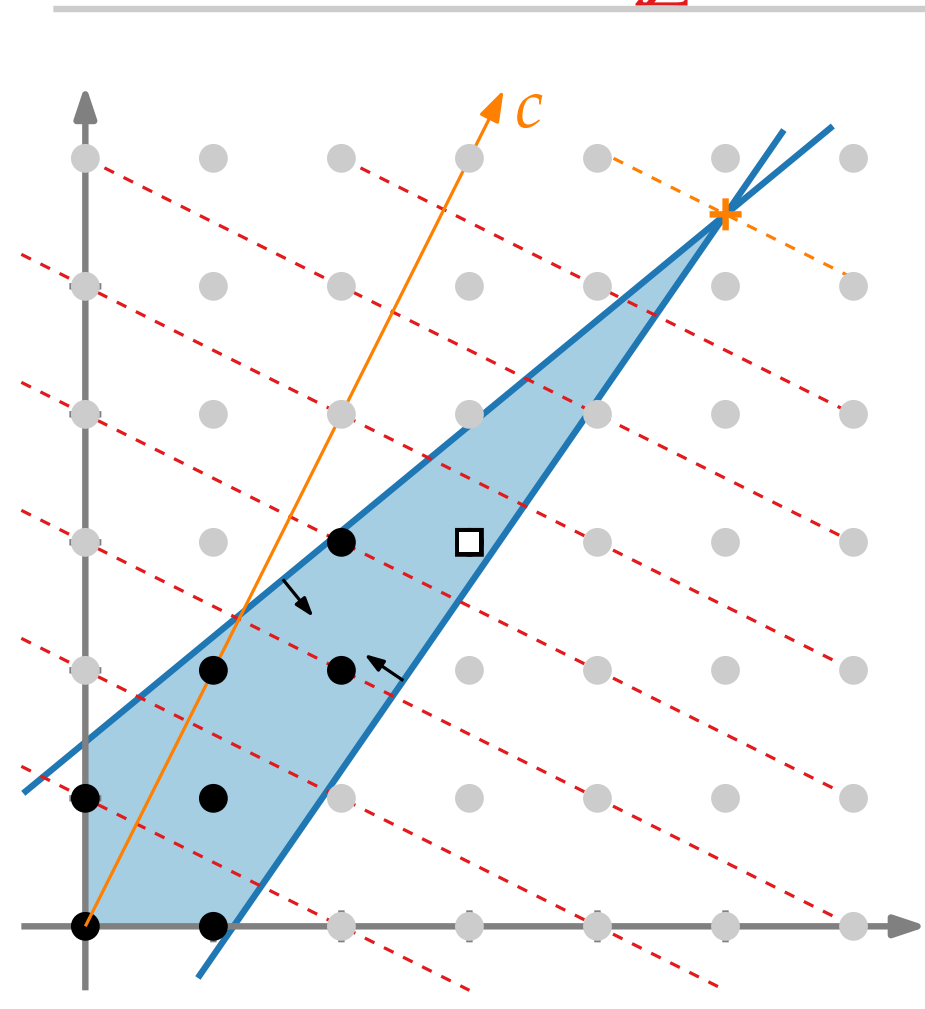


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ ~~$\mathbb{Z}^n$~~ oder $\arg \min$ $x \in \mathbb{Z}^n$



Problem. VERTEXCOVER

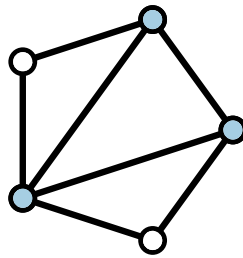
Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

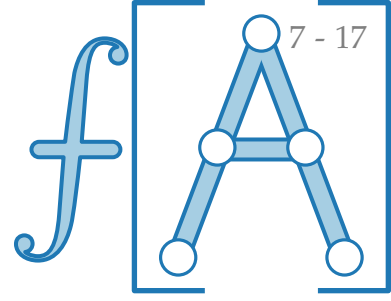
Ziel. $|V'|$ minimal!

Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel: minimiere $\sum_{v \in V} x_v$

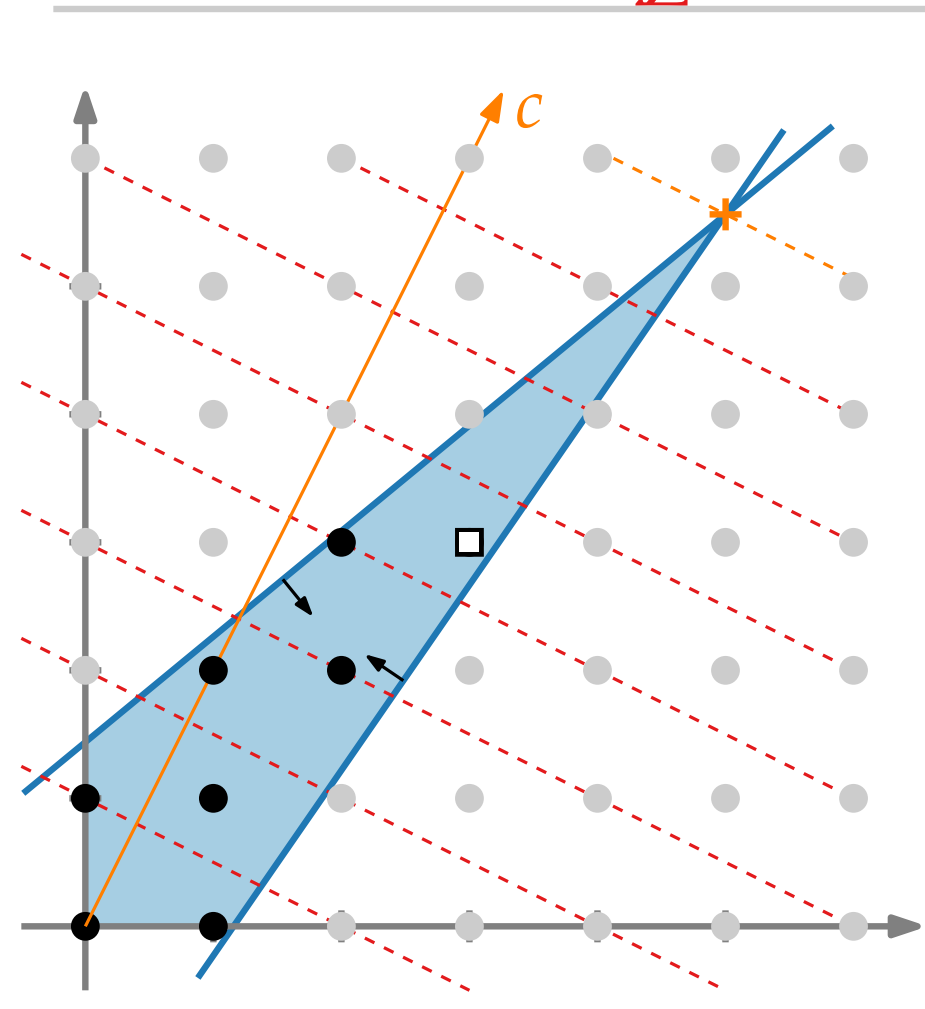


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

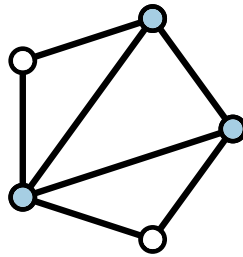
Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

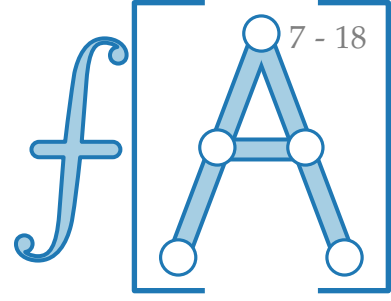
Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel: minimiere $\sum_{v \in V} x_v$

Beschränkungen:

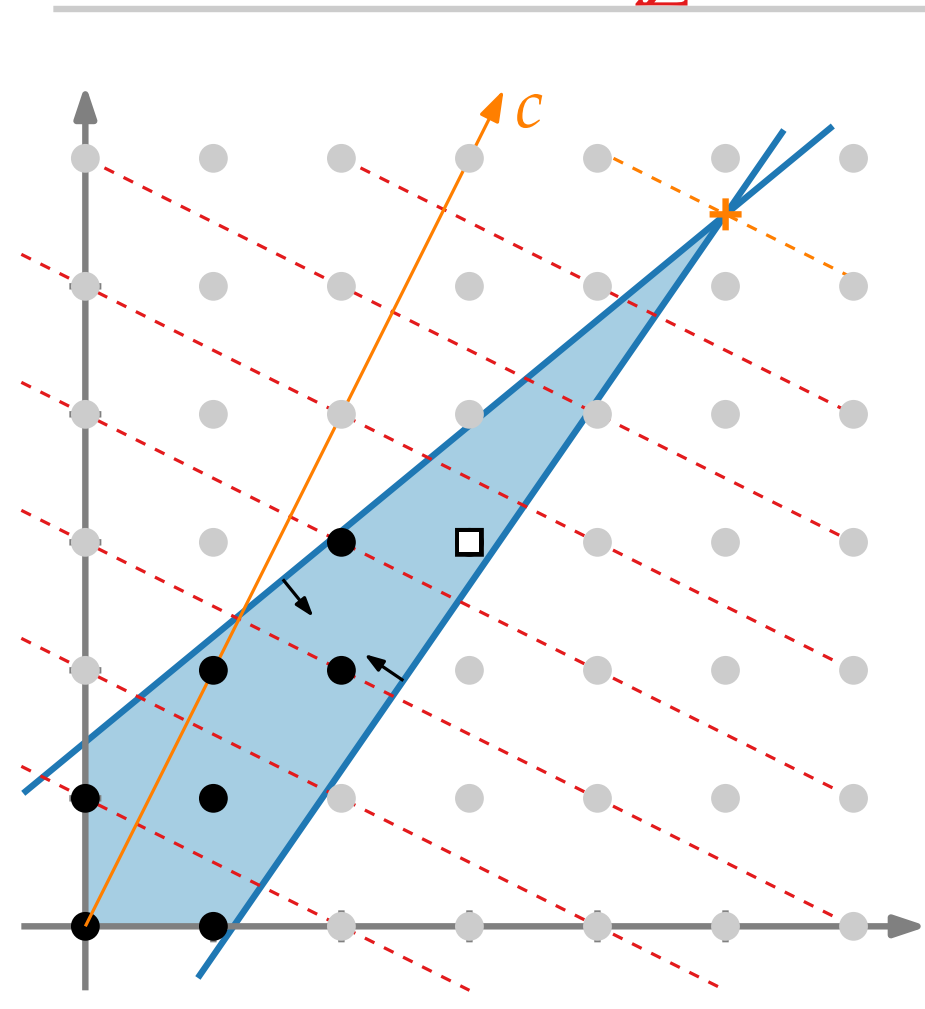


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$

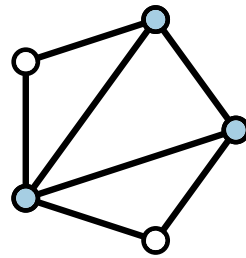


Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!



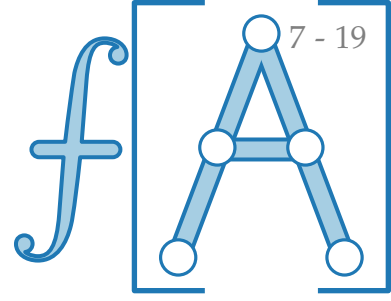
Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel: minimiere $\sum_{v \in V} x_v$

Beschränkungen:

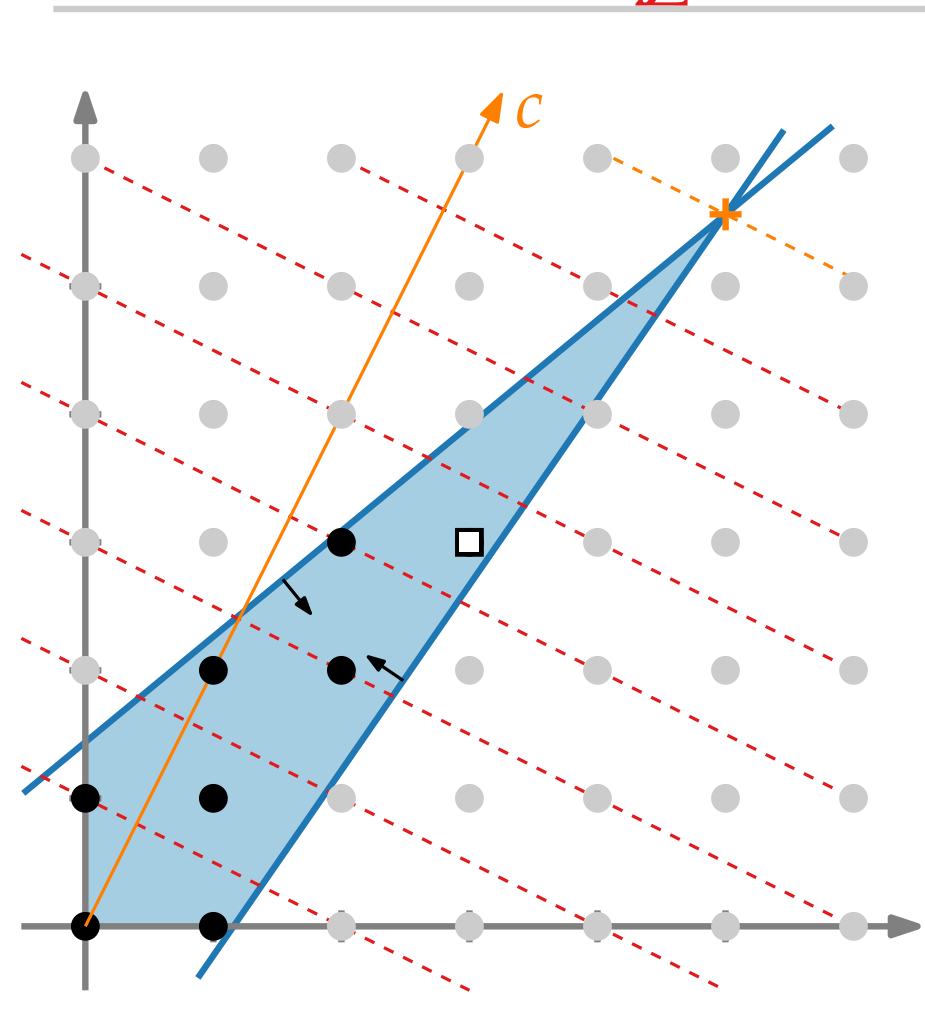
für jede Kante $uv \in E$ fordern wir:
 $x_u + x_v \geq 1$.

Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ ~~$\mathbb{Z}^n$~~ oder $\arg \min$ $x \in \mathbb{Z}^n$



Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

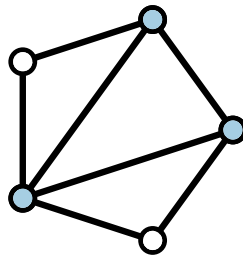
Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel: minimiere $\sum_{v \in V} x_v$

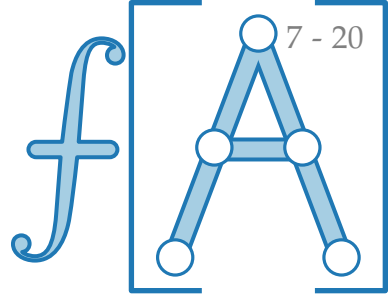
0-1-ILP

Beschränkungen:

für jede Kante $uv \in E$ fordern wir:
 $x_u + x_v \geq 1$.

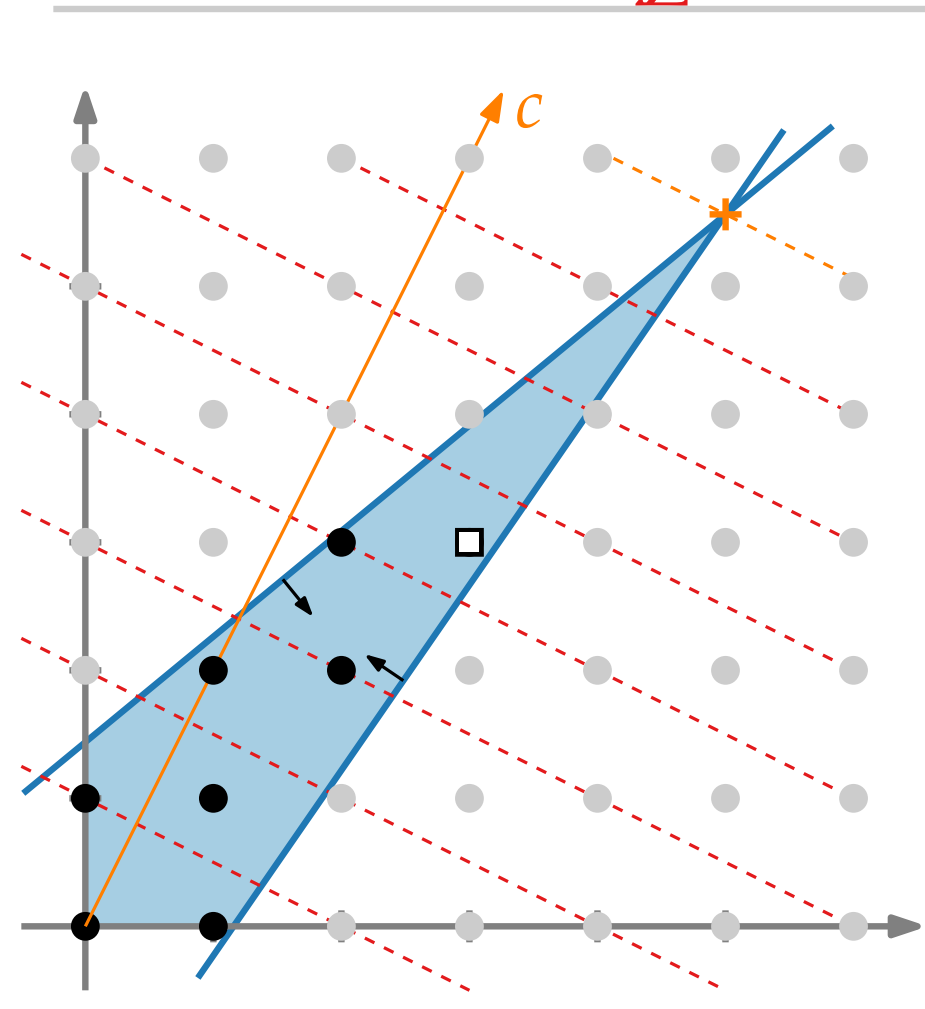


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



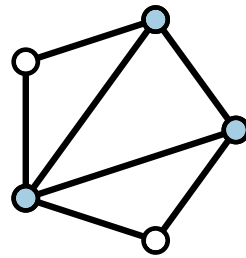
Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

NP-
schwer



Modell. Für $v \in V$ sei $x_v \in \{0,1\}$.

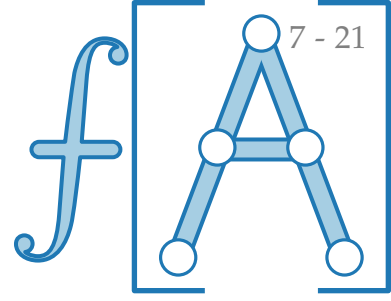
Ziel: minimiere $\sum_{v \in V} x_v$

0-1-ILP

Beschränkungen:

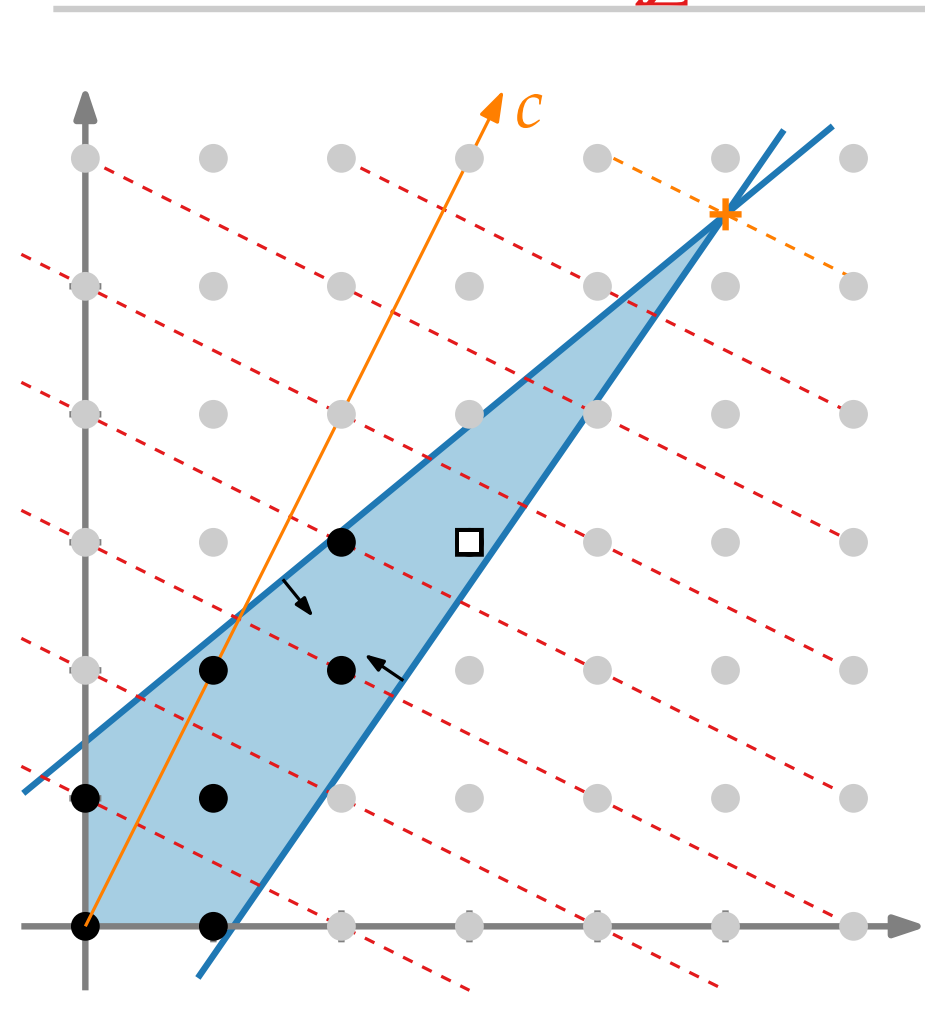
für jede Kante $uv \in E$ fordern wir:
 $x_u + x_v \geq 1$.

Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



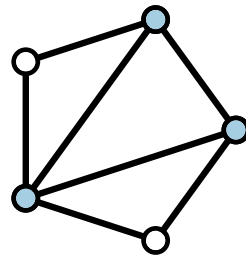
Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

NP-
schwer



Modell. Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel: minimiere $\sum_{v \in V} x_v$

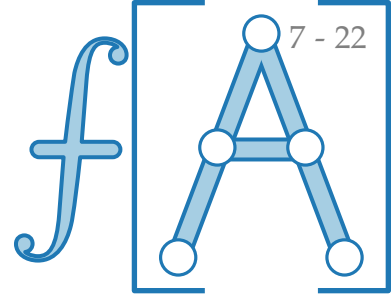
Beschränkungen:

für jede Kante $uv \in E$ fordern wir:
 $x_u + x_v \geq 1$.

0-1-ILP

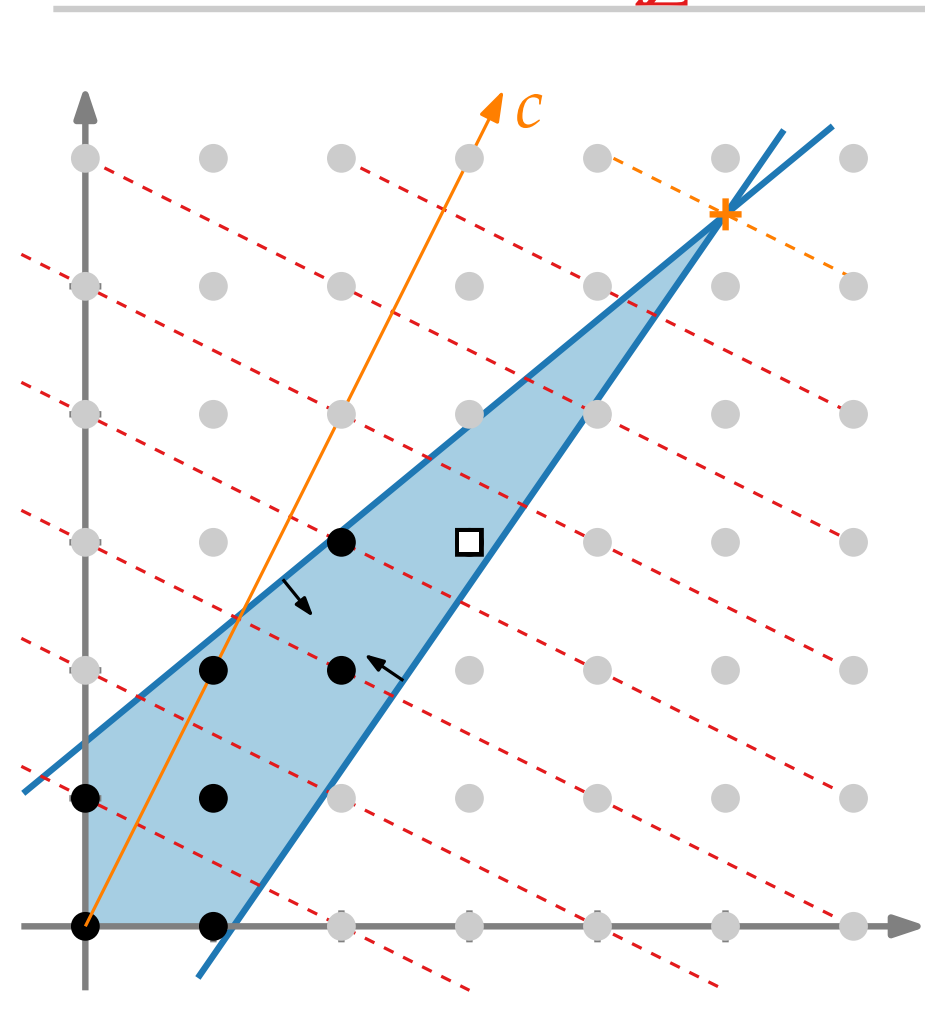


Ganzzahlige lineare Programmierung (ILP)



Gegeben: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$

Gesucht: $x^* \in \mathbb{R}^n$ mit $x^* = \arg \max \{c^T x \mid Ax \leq b, x \geq 0\}$
häufig $\{0,1\}^n$ oder $\arg \min$ $x \in \mathbb{Z}^n$



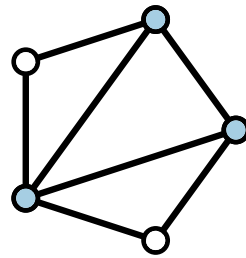
Problem. VERTEXCOVER

Geg. Graph $G = (V, E)$

Ges. Knotenüberdeckung,
d.h. $V' \subseteq V$, so dass jede Kante
mind. einen Endpunkt in V' hat.

Ziel. $|V'|$ minimal!

NP-
schwer



Modell.

Für $v \in V$ sei $x_v \in \{0, 1\}$.

Ziel: minimiere $\sum_{v \in V} x_v$

Beschränkungen:

für jede Kante $uv \in E$ fordern wir:
 $x_u + x_v \geq 1$.

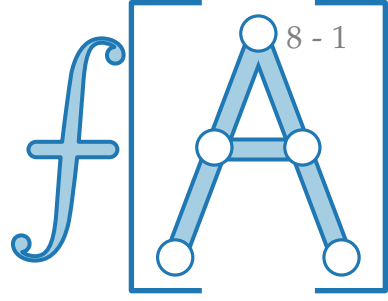
0-1-ILP



NP-
schwer

Satisfiability (SAT)

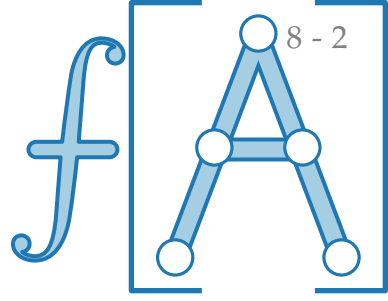
Eine klassische SAT Formel in CNF sieht so aus:



Satisfiability (SAT)

Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

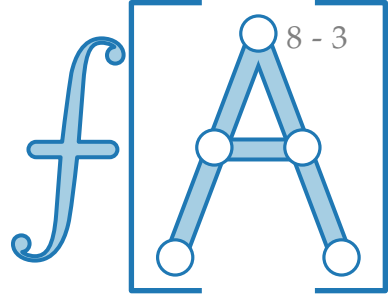


Satisfiability (SAT)

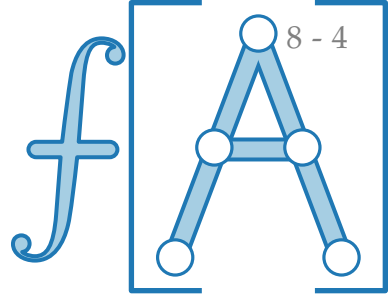
Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$



Satisfiability (SAT)



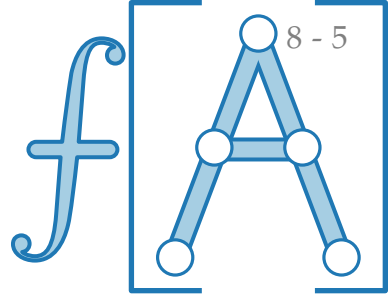
Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$X_1, \dots, X_n$

Klauseln:

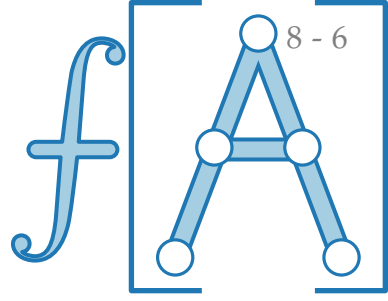
$C_1 \wedge C_2 \wedge \dots \wedge C_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$X_1, \dots, X_n$

Klauseln:

$C_1 \wedge C_2 \wedge \dots \wedge C_m$

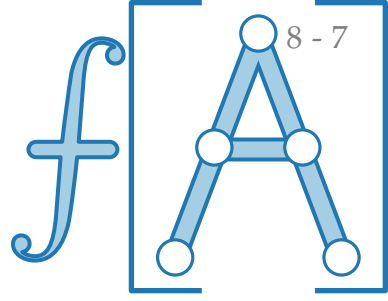
positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$X_1, \dots, X_n$

Klauseln:

$C_1 \wedge C_2 \wedge \dots \wedge C_m$

positive Literale

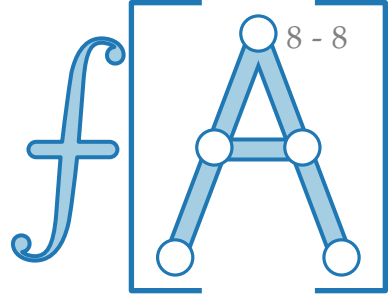
negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$X_1, \dots, X_n$

Klauseln:

$C_1 \wedge C_2 \wedge \dots \wedge C_m$

positive Literale

negative Literale

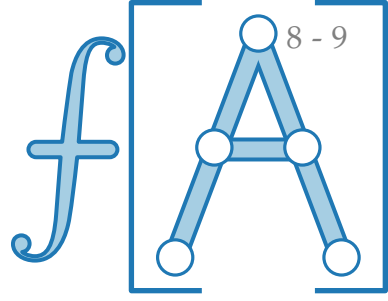
mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

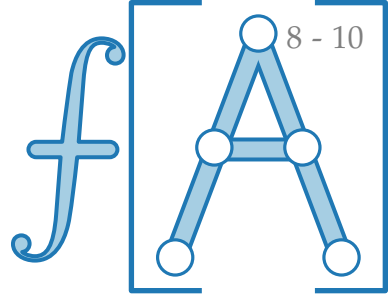
Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$, so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

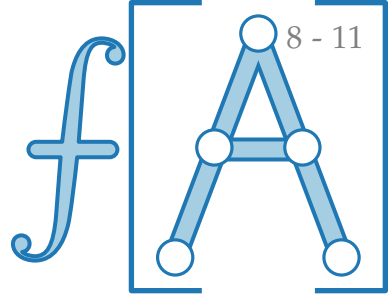
Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

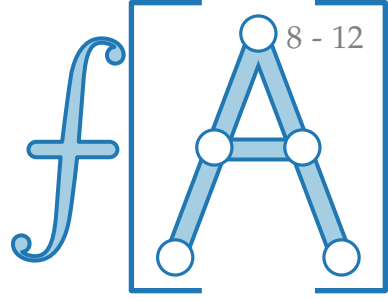
Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Nebenbedingungen:

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$, so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

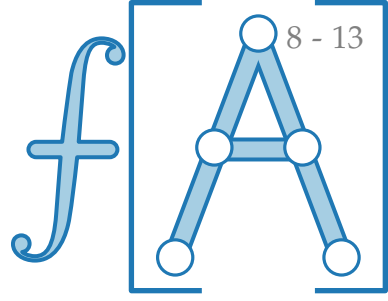
$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Nebenbedingungen:

$$\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$$

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

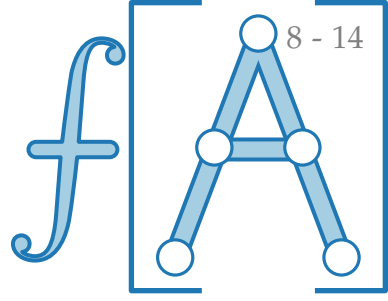
$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Nebenbedingungen:

$$\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$$

mindestens ein positives oder
negatives Literal erfüllt

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

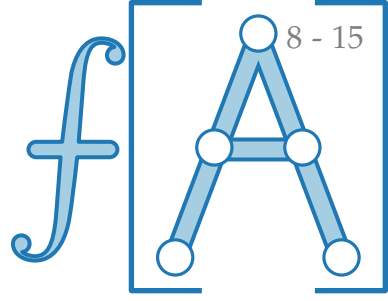
Nebenbedingungen:

$$\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$$

mindestens ein positives oder
negatives Literal erfüllt

Zielfunktion:

Satisfiability (SAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Nebenbedingungen:

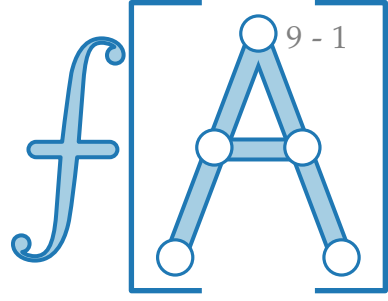
$$\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$$

mindestens ein positives oder
negatives Literal erfüllt

Zielfunktion:

—

Maximum Satisfiability (MaxSAT)



Eine klassische SAT Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$, so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Nebenbedingungen:

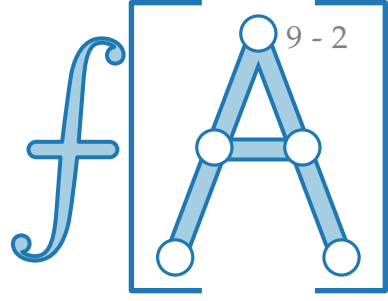
$$\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$$

mindestens ein positives oder negatives Literal erfüllt

Zielfunktion:

—

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen:

$$X_1, \dots, X_n$$

Klauseln:

$$C_1 \wedge C_2 \wedge \dots \wedge C_m$$

positive Literale

negative Literale

$$\text{mit } P_i, N_i \subseteq \{X_1, \dots, X_n\}$$

$$\text{und } C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$, so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen:

$$x_1, \dots, x_n \in \{0, 1\}$$

$$1 \hat{=} \text{TRUE}, 0 \hat{=} \text{FALSE}$$

Nebenbedingungen:

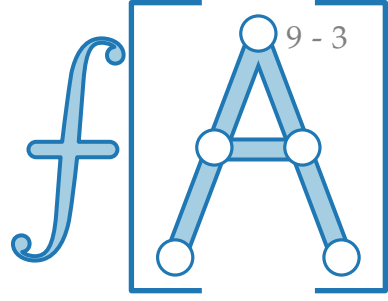
$$\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$$

mindestens ein positives oder negatives Literal erfüllt

Zielfunktion:

—

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$

Klauselgewichte: $w_1, \dots, w_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass alle Klauseln erfüllt sind.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen: $x_1, \dots, x_n \in \{0, 1\}$

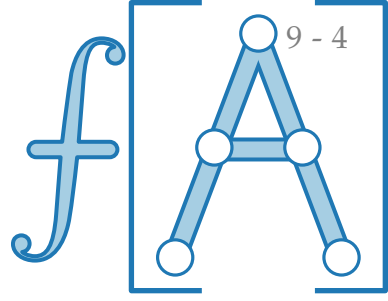
$1 \hat{=}$ TRUE, $0 \hat{=}$ FALSE

Nebenbedingungen: $\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$

mindestens ein positives oder
negatives Literal erfüllt

Zielfunktion: —

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$

Klauselgewichte: $w_1, \dots, w_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass **das Gewicht der erfüllten Klauseln maximiert wird**.

Eine SAT Formel lässt sich als ein ILP formulieren:

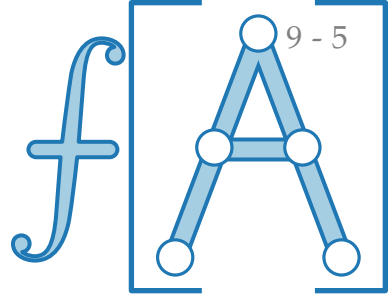
Variablen: $x_1, \dots, x_n \in \{0, 1\}$

$1 \hat{=}$ TRUE, $0 \hat{=}$ FALSE

Nebenbedingungen: $\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$ mindestens ein positives oder negatives Literal erfüllt

Zielfunktion: —

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$

Klauselgewichte: $w_1, \dots, w_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass **das Gewicht der erfüllten Klauseln maximiert wird**.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen: $x_1, \dots, x_n \in \{0, 1\}$

$1 \hat{=}$ TRUE, $0 \hat{=}$ FALSE

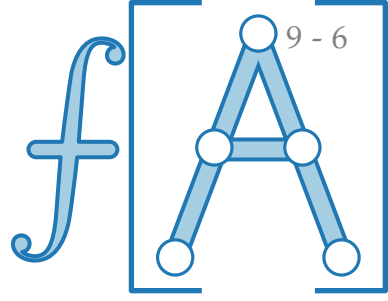
$c_1, \dots, c_m \in \{0, 1\}$

Nebenbedingungen: $\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$

mindestens ein positives oder
negatives Literal erfüllt

Zielfunktion: —

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$
Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$
Klauselgewichte: $w_1, \dots, w_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass **das Gewicht der erfüllten Klauseln maximiert wird**.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen: $x_1, \dots, x_n \in \{0, 1\}$
 $c_1, \dots, c_m \in \{0, 1\}$

$1 \hat{=}$ TRUE, $0 \hat{=}$ FALSE

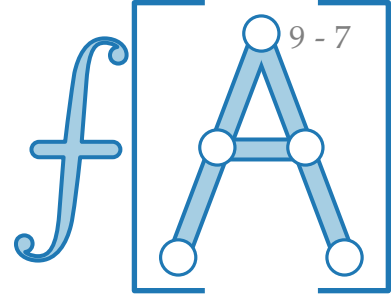
$1 \hat{=}$ Klausel erfüllt, $0 \hat{=}$ Klausel nicht erfüllt

Nebenbedingungen: $\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq 1$

mindestens ein positives oder
negatives Literal erfüllt

Zielfunktion: —

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$
Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$
Klauselgewichte: $w_1, \dots, w_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass **das Gewicht der erfüllten Klauseln maximiert wird**.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen: $x_1, \dots, x_n \in \{0, 1\}$
 $c_1, \dots, c_m \in \{0, 1\}$

$1 \hat{=}$ TRUE, $0 \hat{=}$ FALSE

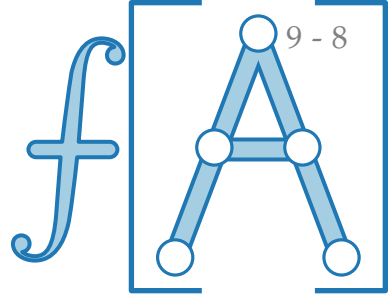
$1 \hat{=}$ Klausel erfüllt, $0 \hat{=}$ Klausel nicht erfüllt

Nebenbedingungen: $\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq c_j$

Klausel nur erfüllt, wenn mindestens ein
positives oder negatives Literal erfüllt

Zielfunktion: —

Maximum Satisfiability (MaxSAT)



Eine **MaxSAT** Formel in CNF sieht so aus:

Binäre Variablen: $X_1, \dots, X_n$

Klauseln: $C_1 \wedge C_2 \wedge \dots \wedge C_m$

Klauselgewichte: $w_1, \dots, w_m$

positive Literale

negative Literale

mit $P_i, N_i \subseteq \{X_1, \dots, X_n\}$

und $C_j = \bigvee_{X_i \in P_j} X_i \vee \bigvee_{X_i \in N_j} \neg X_i$

Eine Lösung ist eine Zuweisung der Variablen $X_i \in \{\text{TRUE}, \text{FALSE}\}$,
so dass **das Gewicht der erfüllten Klauseln maximiert wird**.

Eine SAT Formel lässt sich als ein ILP formulieren:

Variablen: $x_1, \dots, x_n \in \{0, 1\}$

$c_1, \dots, c_m \in \{0, 1\}$

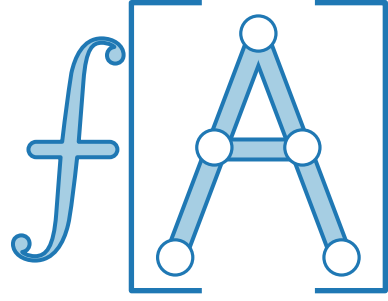
$1 \hat{=}$ TRUE, $0 \hat{=}$ FALSE

$1 \hat{=}$ Klausel erfüllt, $0 \hat{=}$ Klausel nicht erfüllt

Nebenbedingungen: $\sum_{X_i \in P_j} x_i + \sum_{X_i \in N_j} (1 - x_i) \geq c_j$

Klausel nur erfüllt, wenn mindestens ein
positives oder negatives Literal erfüllt

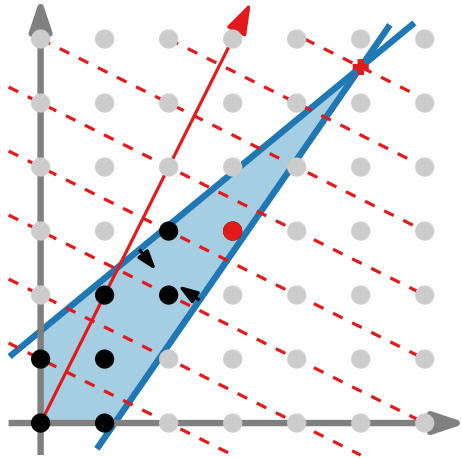
Zielfunktion: maximiere $\sum_{j=1}^m w_j \cdot c_j$



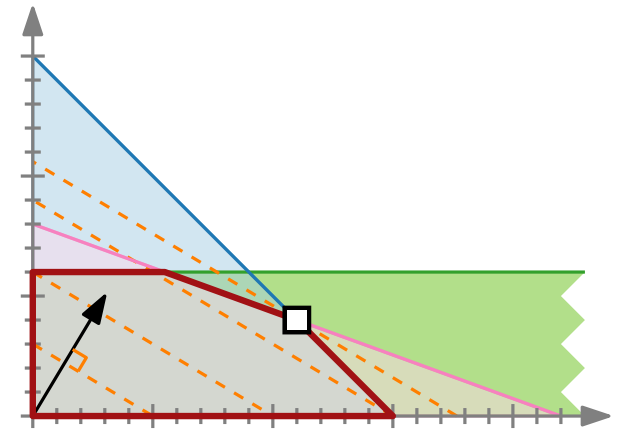
Fortgeschrittene Algorithmen

22. Vorlesung ILP & MaxSAT Solver

Teil III: Beispiele



$$\begin{pmatrix} \text{blue} & \text{green} & \text{teal} \\ 0 & \dots & 0 & 1 & 0 & \dots & 0 \\ \text{red} & \text{pink} & \text{purple} \end{pmatrix}$$

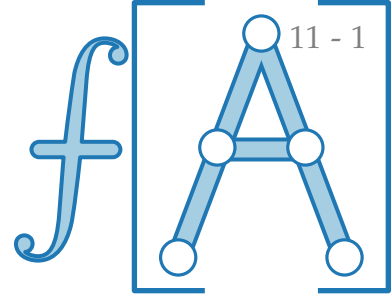
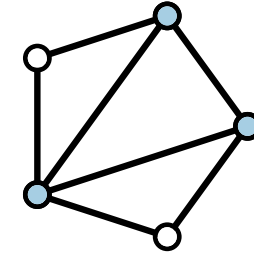


Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G



Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

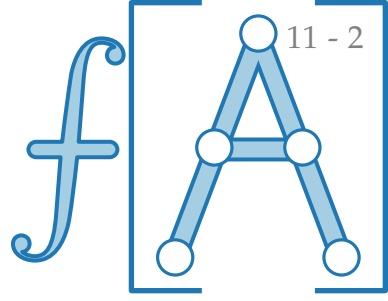
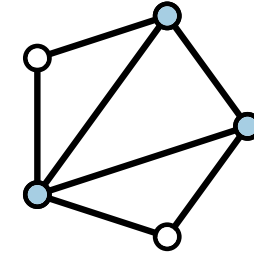
Ges. eine kleinste Knotenüberdeckung von G

ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$



Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

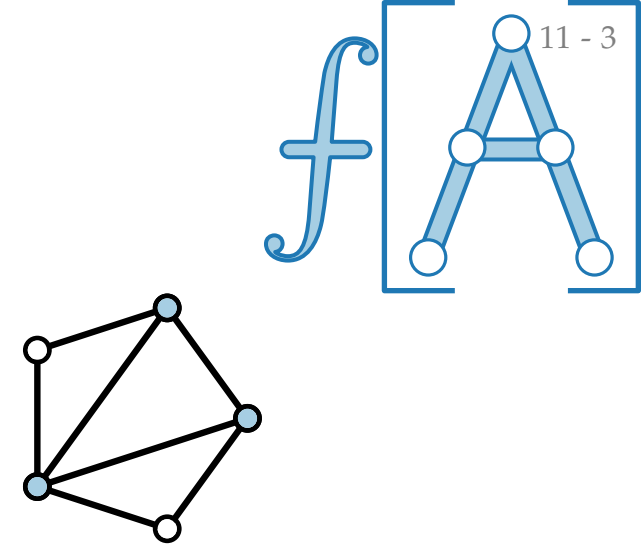
ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:



Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

ILP Formulierung:

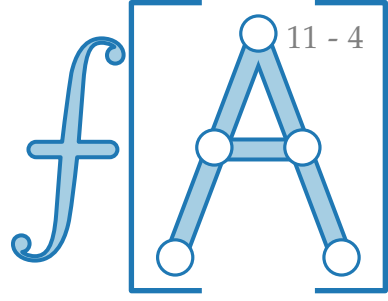
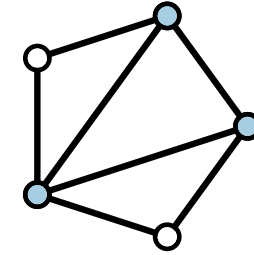
Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

Binäre Variablen: X_v für jedes $v \in V$



Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

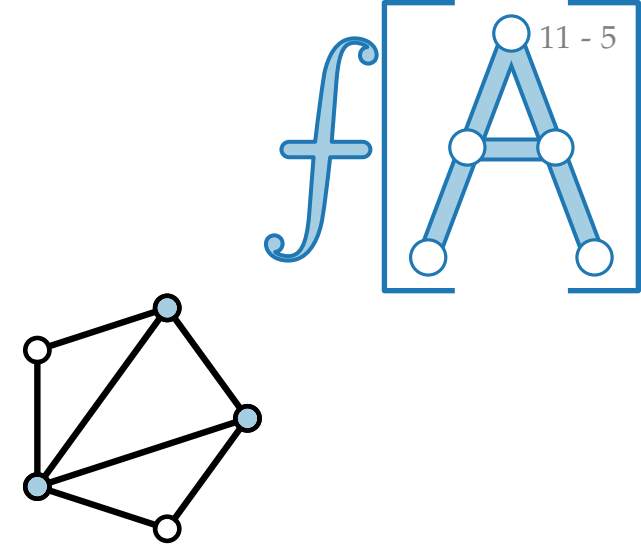
Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

Binäre Variablen: X_v für jedes $v \in V$

Klauseln:



Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G

ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

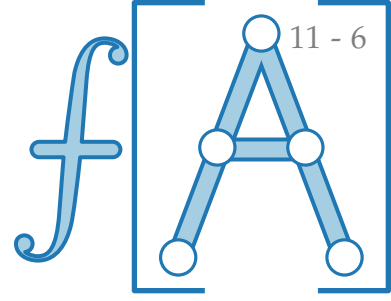
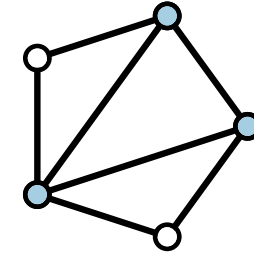
Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

Binäre Variablen: X_v für jedes $v \in V$

Klauseln: $(X_u \vee X_v)$ für jede Kante $uv \in E$

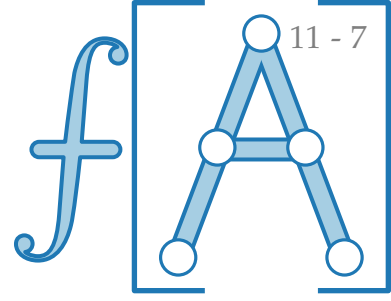
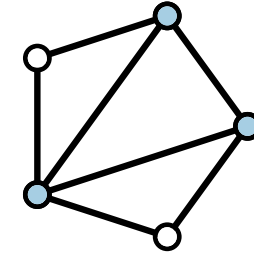


Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G



ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

Binäre Variablen: X_v für jedes $v \in V$

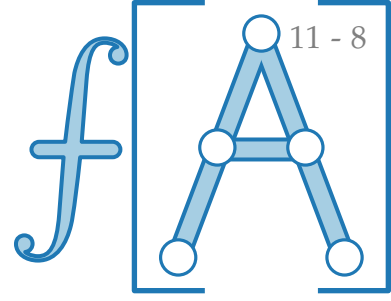
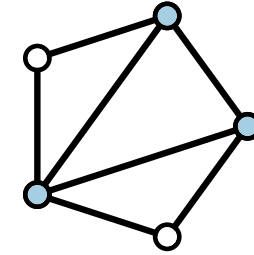
Klauseln: $(X_u \vee X_v)$ für jede Kante $uv \in E$ $(\neg X_v)$ für jedes $v \in V$

Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G



ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

Binäre Variablen: X_v für jedes $v \in V$

Klauseln: $(X_u \vee X_v)$ für jede Kante $uv \in E$ $(\neg X_v)$ für jedes $v \in V$

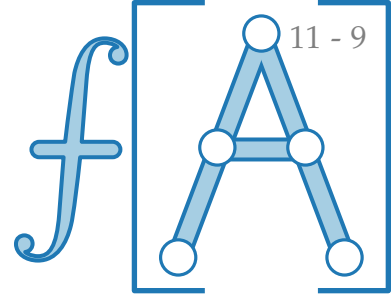
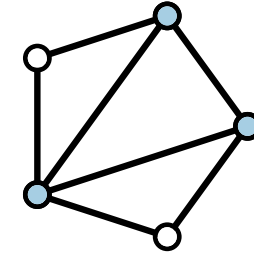
Klauselgewichte:

Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G



ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

Binäre Variablen: X_v für jedes $v \in V$

Klauseln: $(X_u \vee X_v)$ für jede Kante $uv \in E$ $(\neg X_v)$ für jedes $v \in V$

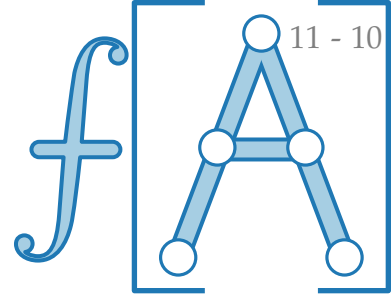
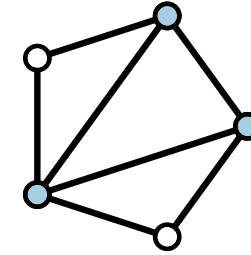
Klauselgewichte: 1

Bsp. I: VERTEXCOVER

Problem. KLEINSTE KNOTENÜBERDECKUNG

Geg. Graph G

Ges. eine kleinste Knotenüberdeckung von G



ILP Formulierung:

Variablen: Für $v \in V$ sei $x_v \in \{0, 1\}$.

Nebenbedingungen: $x_u + x_v \geq 1$ für jede Kante $uv \in E$

Zielfunktion: minimiere $\sum_{v \in V} x_v$

MaxSAT Formulierung:

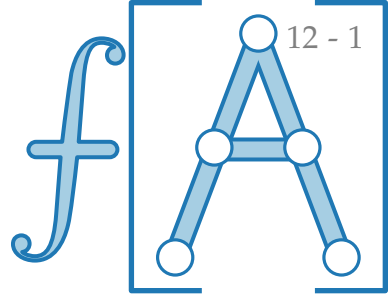
Binäre Variablen: X_v für jedes $v \in V$

Klauseln: $(X_u \vee X_v)$ für jede Kante $uv \in E$ $(\neg X_v)$ für jedes $v \in V$

Klauselgewichte: $> n$, z.B. $2n$ 1

Bsp. II: ILP-Formulierung für MaxClique

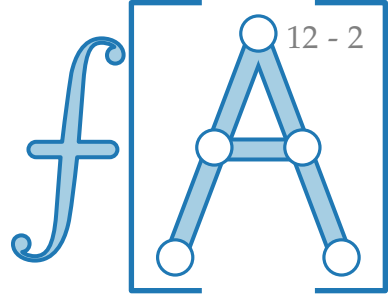
Problem. MaxCLIQUE



Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

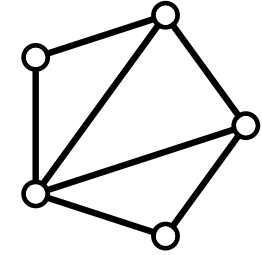
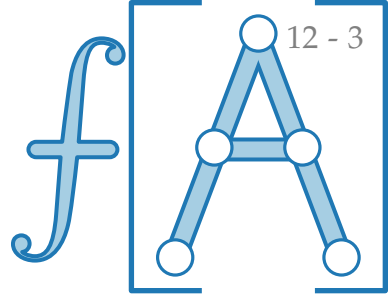
Geg. ungerichteter, ungewichteter Graph $G = (V, E)$



Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

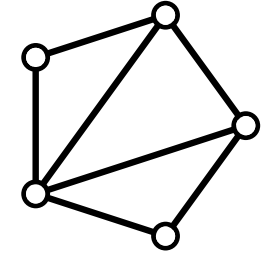
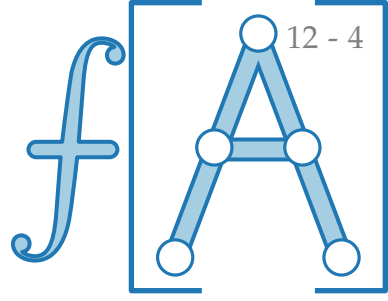


Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G



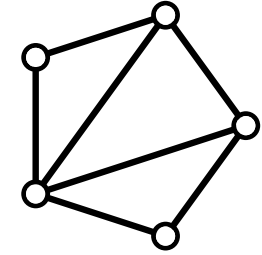
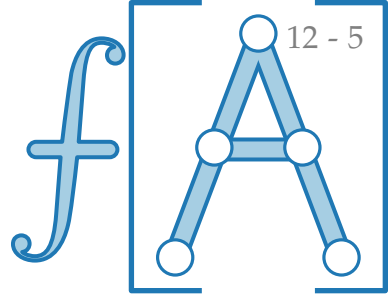
Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G ,

d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig



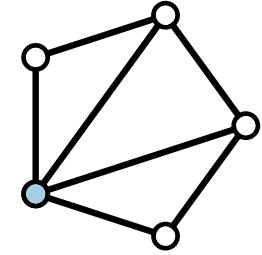
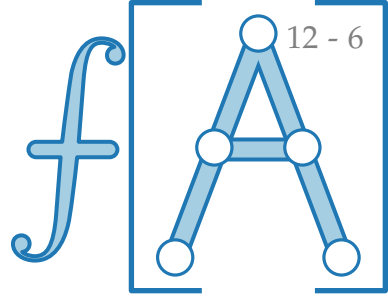
Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G ,

d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig

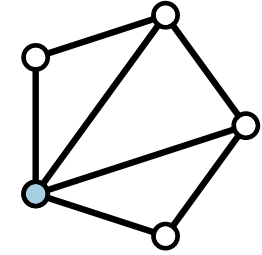
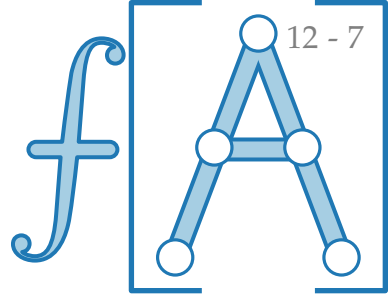


Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig

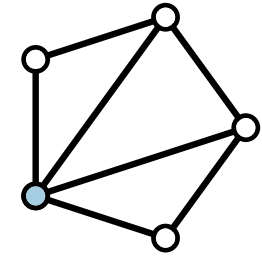
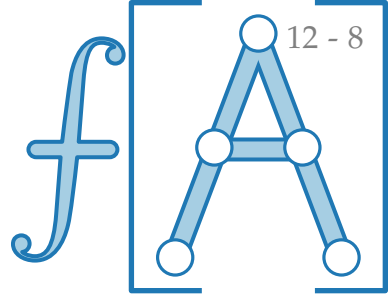


Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.



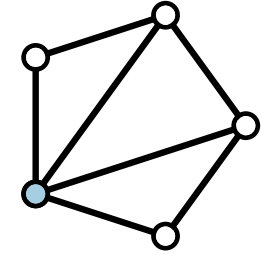
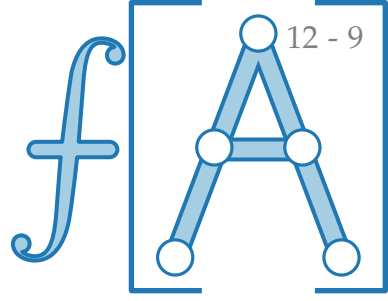
Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



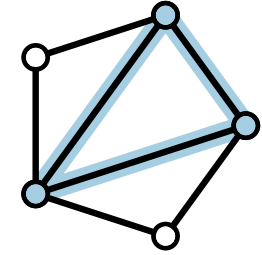
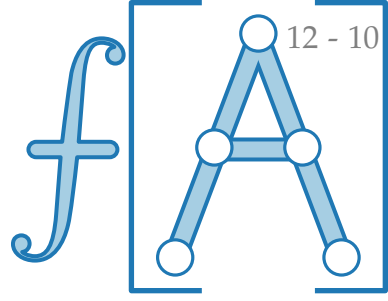
Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

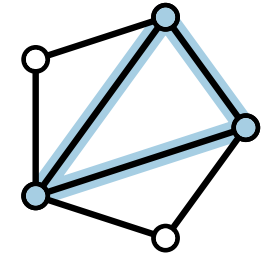
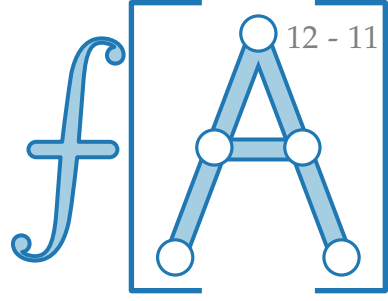
Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!

Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$.



Bsp. II: ILP-Formulierung für MaxClique

Problem. MaxCLIQUE

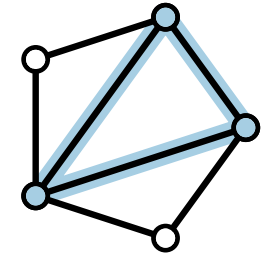
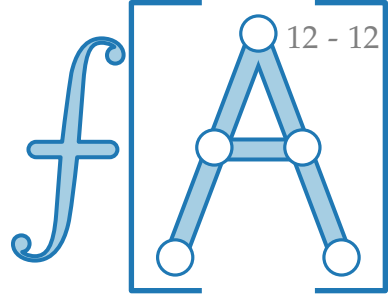
Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

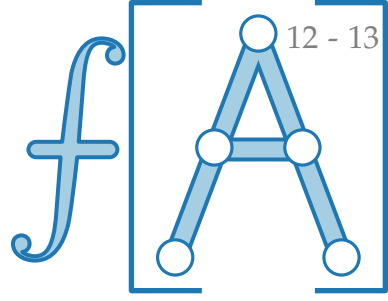
Ziel. $|V'|$ maximal!

Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)



Bsp. II: ILP-Formulierung für MaxClique

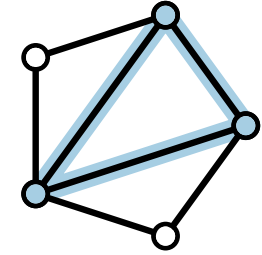


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!

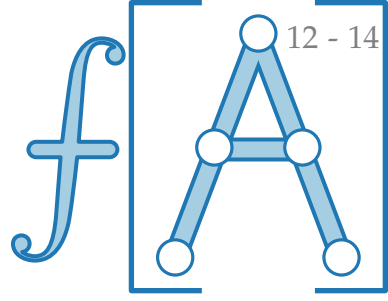


Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Bsp. II: ILP-Formulierung für MaxClique

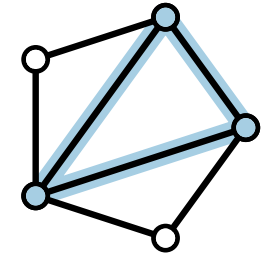


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



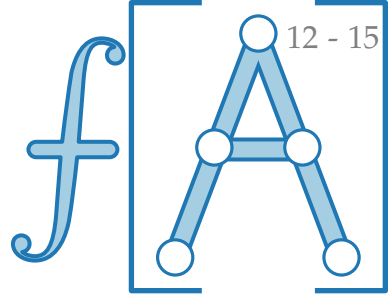
Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

Bsp. II: ILP-Formulierung für MaxClique

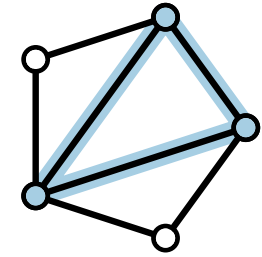


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

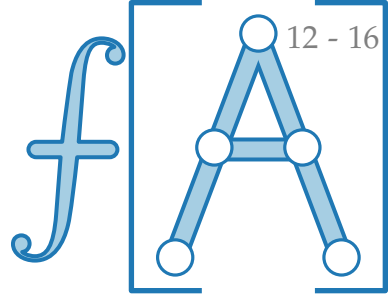
Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

Bsp. II: ILP-Formulierung für MaxClique

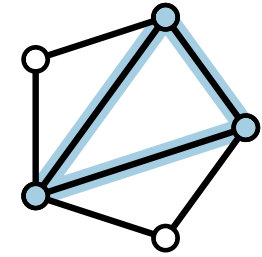


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

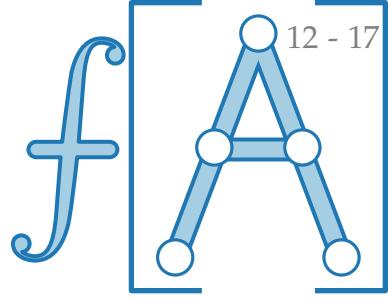
Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$:

Bsp. II: ILP-Formulierung für MaxClique

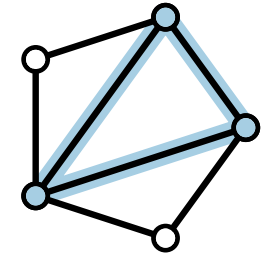


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

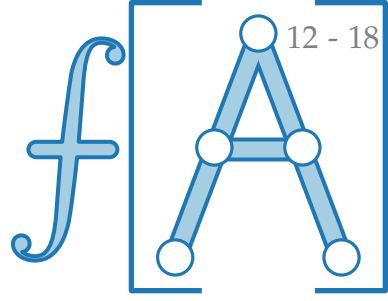
Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

Bsp. II: ILP-Formulierung für MaxClique

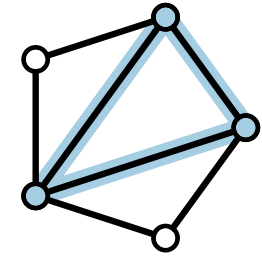


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

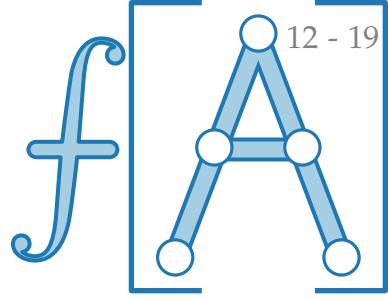
Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

Bsp. II: ILP-Formulierung für MaxClique

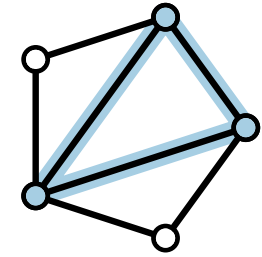


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



MaxSAT:

Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

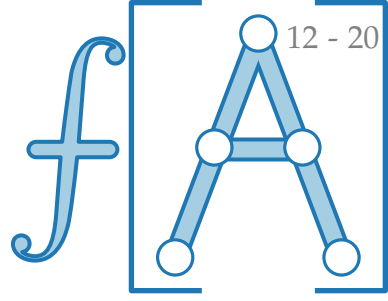
Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

Bsp. II: ILP-Formulierung für MaxClique

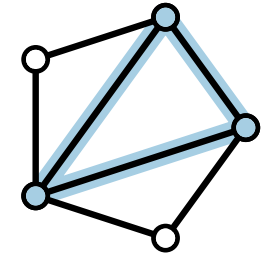


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

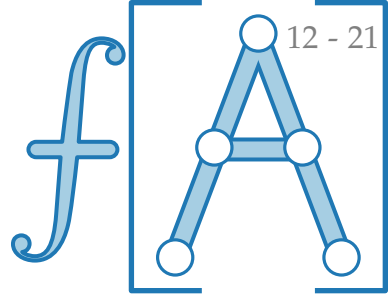
D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

MaxSAT:

Variablen:

x_v

Bsp. II: ILP-Formulierung für MaxClique

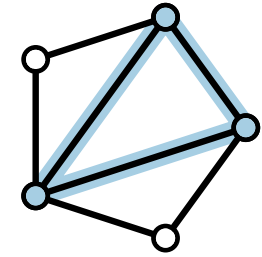


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

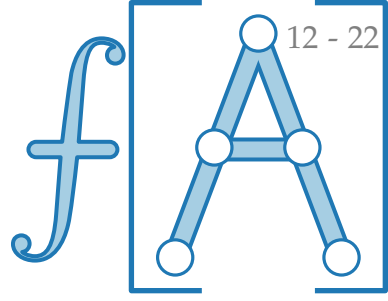
MaxSAT:

Variablen:

x_v

Klauseln:

Bsp. II: ILP-Formulierung für MaxClique

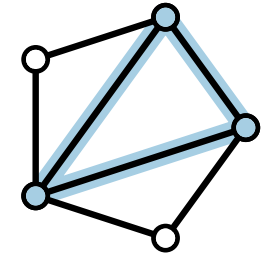


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

MaxSAT:

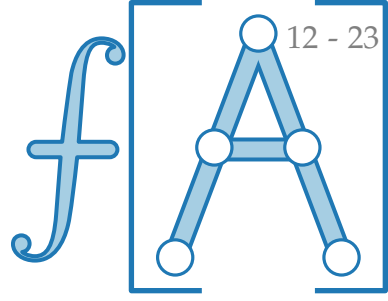
Variablen:

x_v

Klauseln:

x_v

Bsp. II: ILP-Formulierung für MaxClique

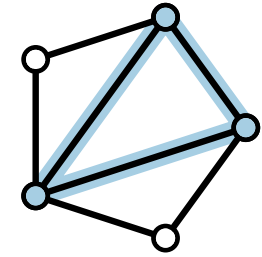


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

MaxSAT:

Variablen:

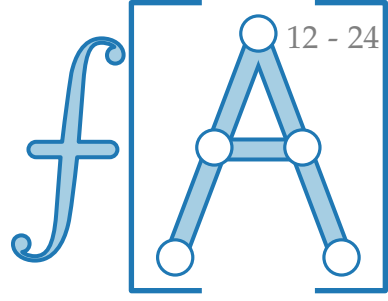
x_v

Klauseln:

x_v

Gewicht 1

Bsp. II: ILP-Formulierung für MaxClique

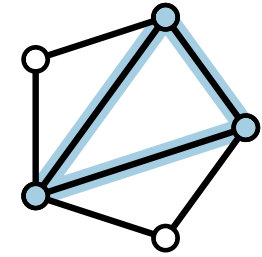


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

MaxSAT:

Variablen:

x_v

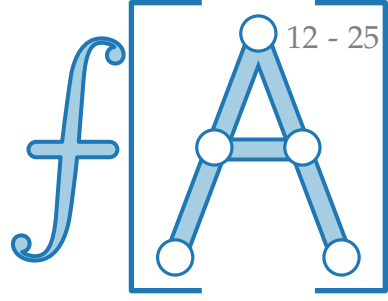
Klauseln:

x_v

Gewicht 1

$(\neg x_u \vee \neg x_v)$

Bsp. II: ILP-Formulierung für MaxClique

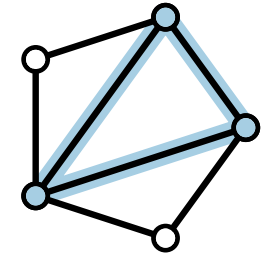


Problem. MaxCLIQUE

Geg. ungerichteter, ungewichteter Graph $G = (V, E)$

Ges. Clique in G , $G[V'] := (V', \{u'v' \in E : u' \in V', v' \in V'\})$
d.h. $V' \subseteq V$, sodass der von V' induzierte Graph $G[V']$ vollständig
m.a.W. $V' \subseteq V$, so dass für alle $\{u', v'\} \in \binom{V'}{2}$ gilt $u'v' \in E$.

Ziel. $|V'|$ maximal!



Variable:

Für $v \in V$ sei $x_v \in \{0, 1\}$. (Damit codieren wir die Menge V' !)

Zielfunktion:

Maximiere $\sum_{v \in V} x_v$

unter den Nebenbedingungen:

für jede Nicht-Kante $\{u, v\} \in \binom{V}{2} \setminus E$: $x_u + x_v \leq 1$.

D.h. wenn u und v nicht benachbart sind, darf höchstens einer in die Clique.

MaxSAT:

Variablen:

x_v

Klauseln:

x_v

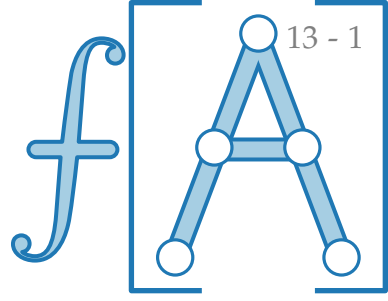
Gewicht 1

$(\neg x_u \vee \neg x_v)$

Gewicht $> n$

Bsp. III: ILP-Formulierung für TSP

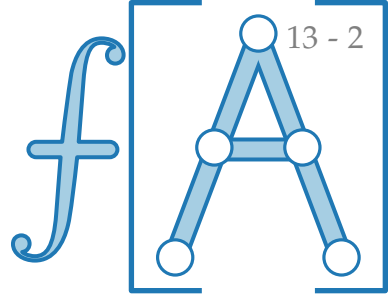
Problem. TRAVELINGSALESMANPROBLEM



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

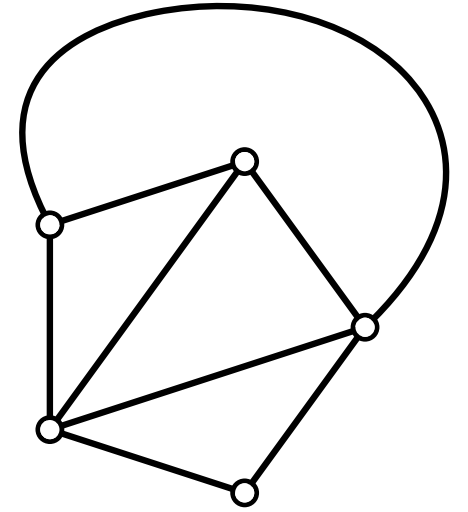
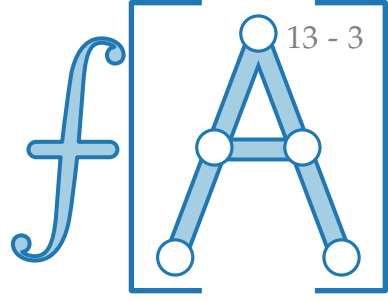
Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und
Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

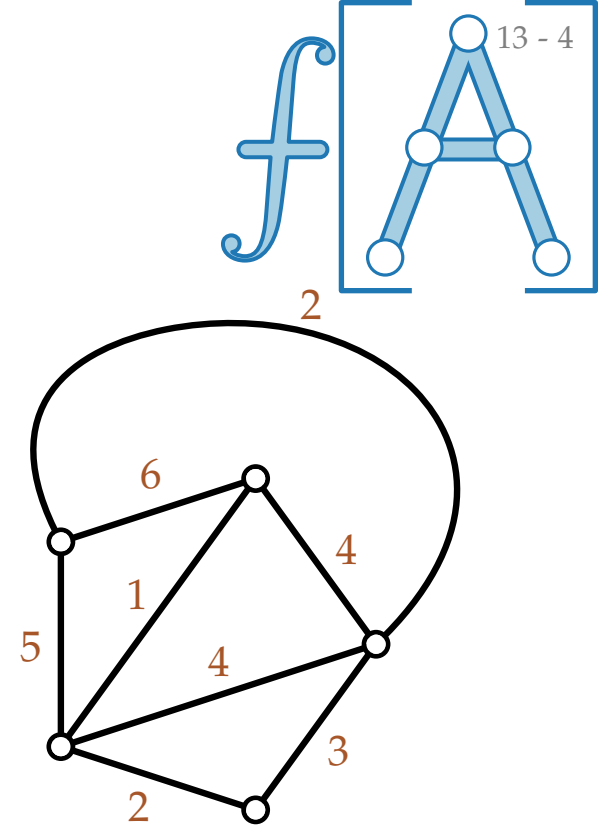
Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und
Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und
Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

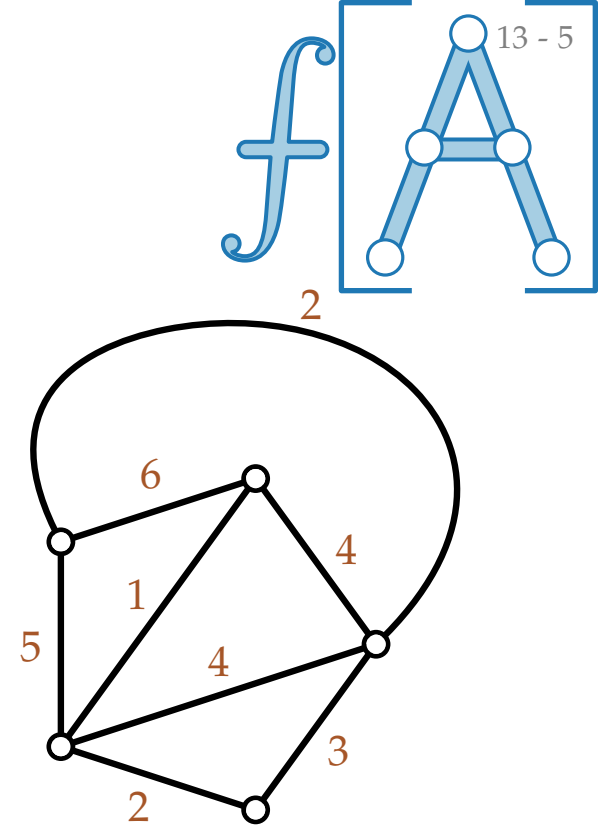


Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.

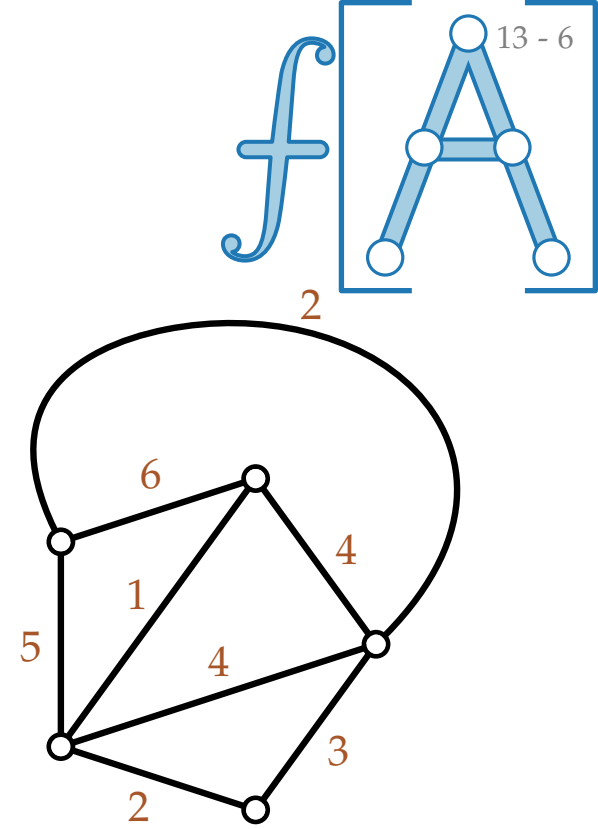


Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

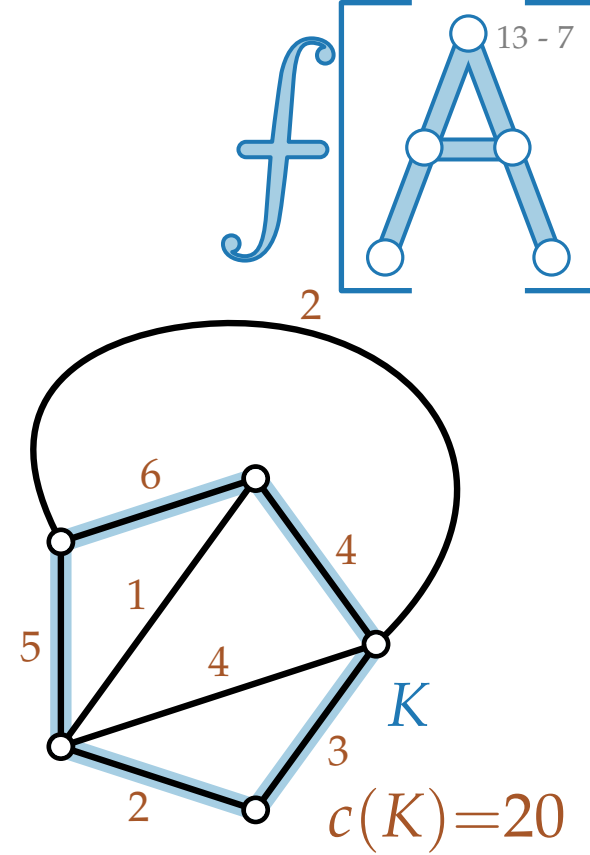


Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

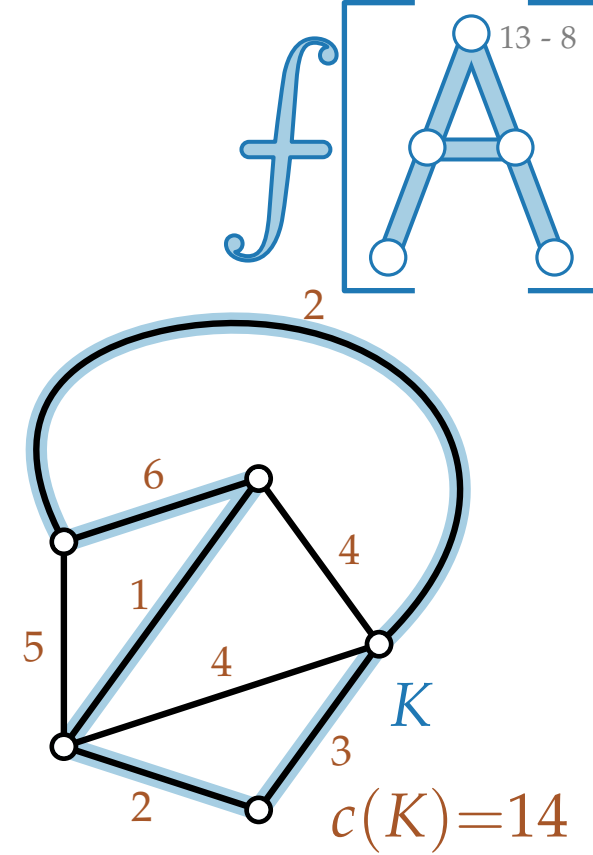


Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.



Bsp. III: ILP-Formulierung für TSP

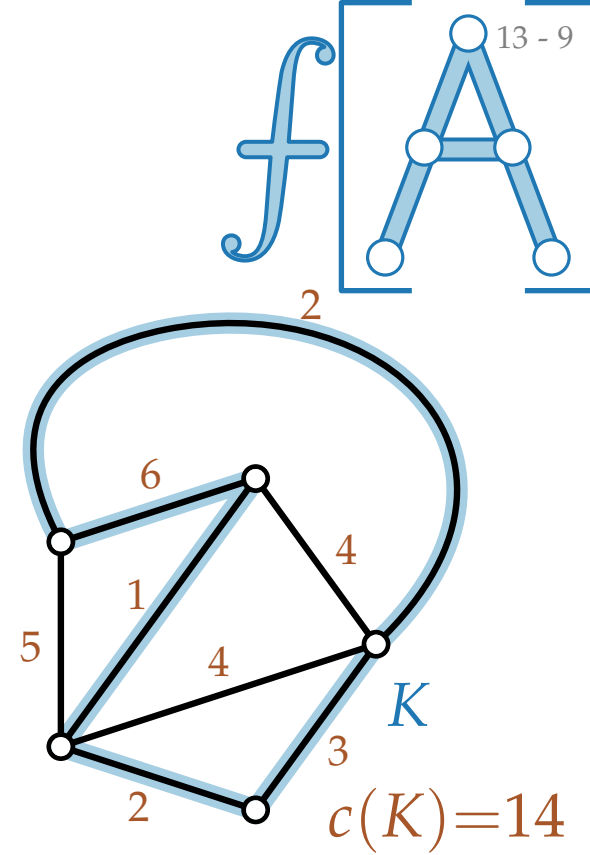
Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

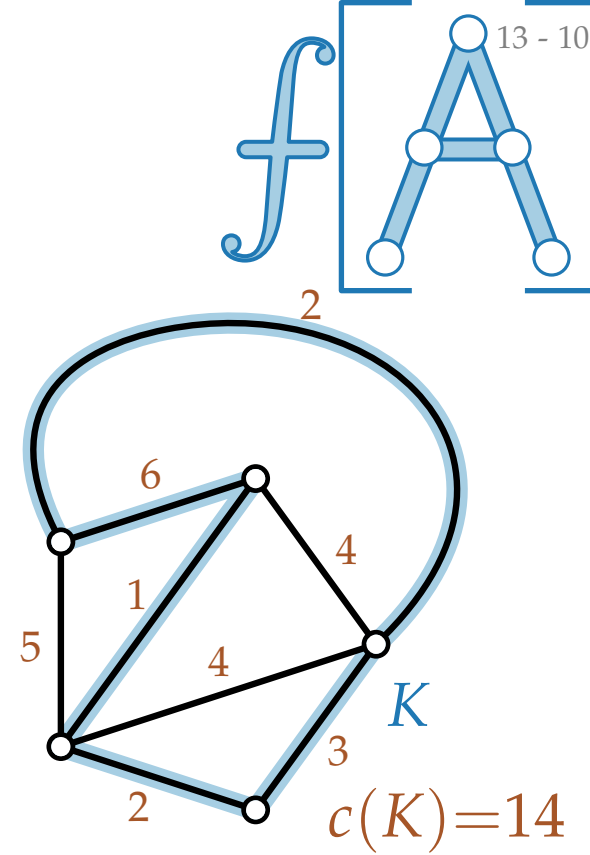
Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

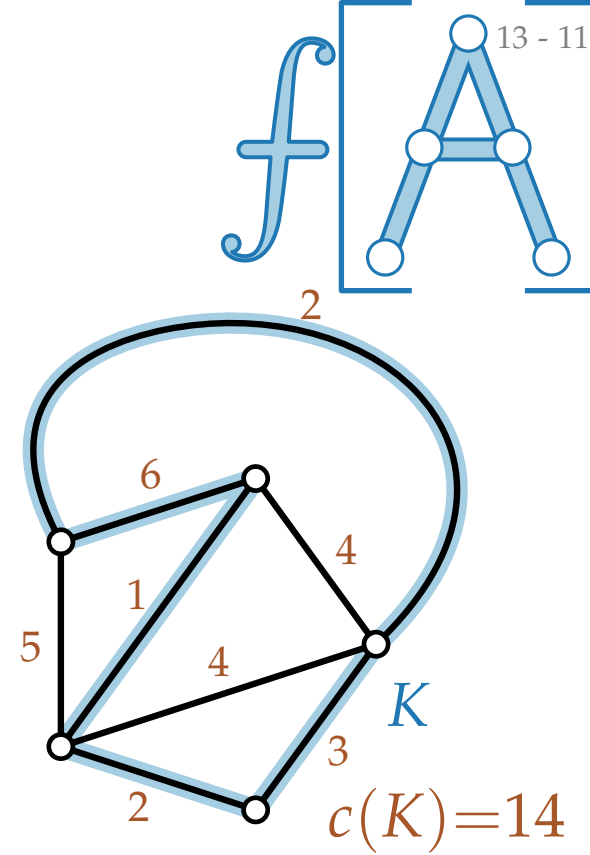
Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

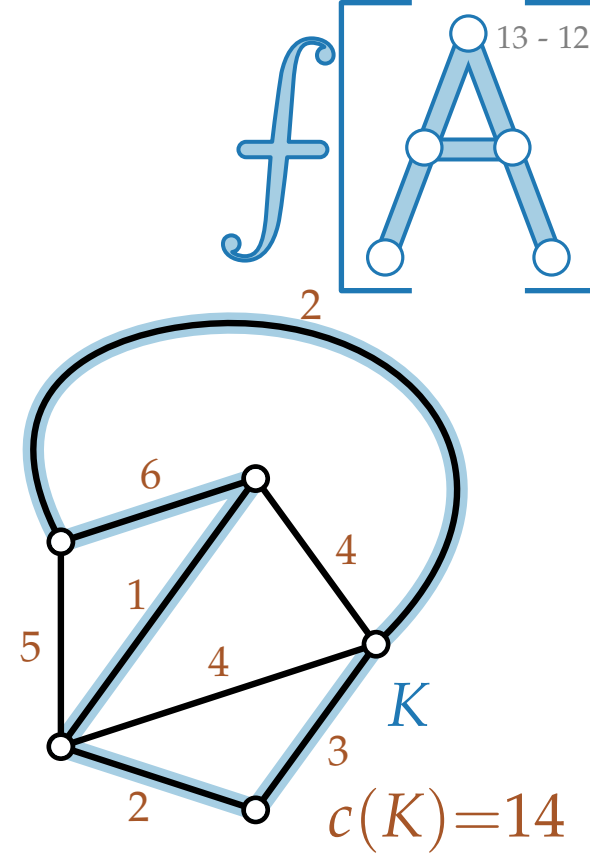
Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

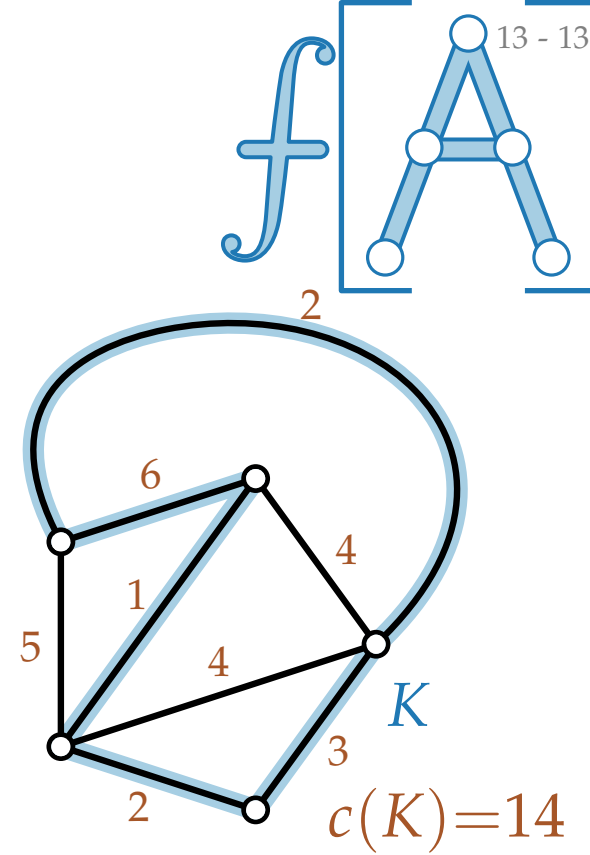
Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: für jeden Knoten $v \in V$:



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

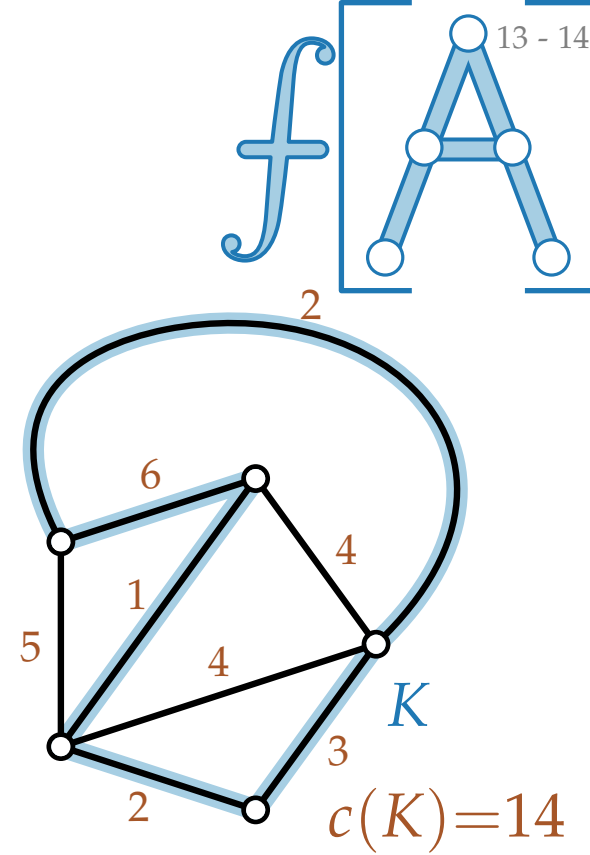
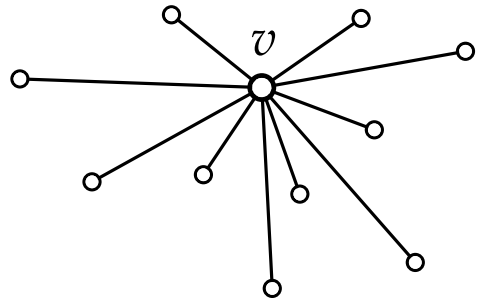
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} =$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

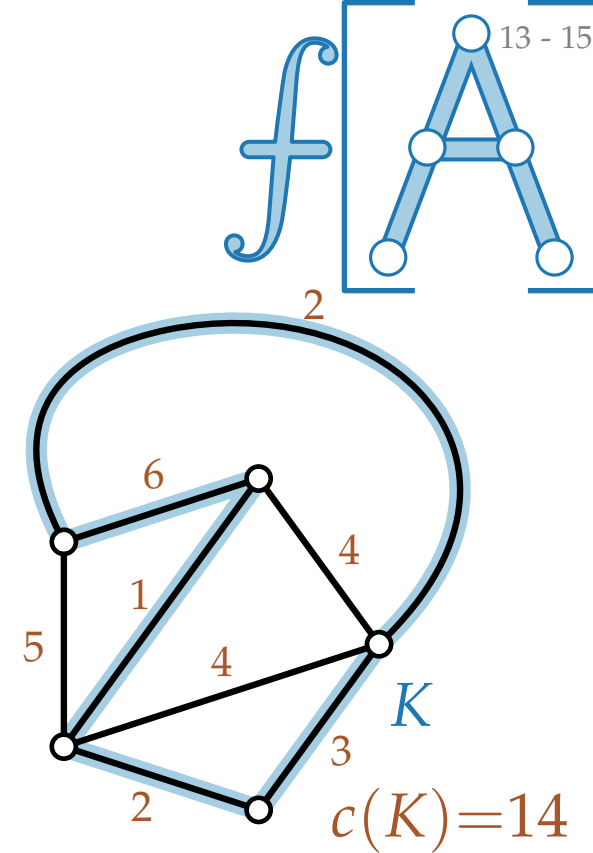
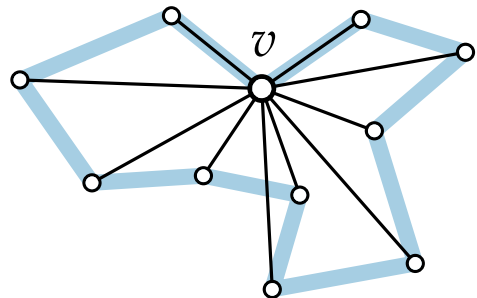
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} =$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

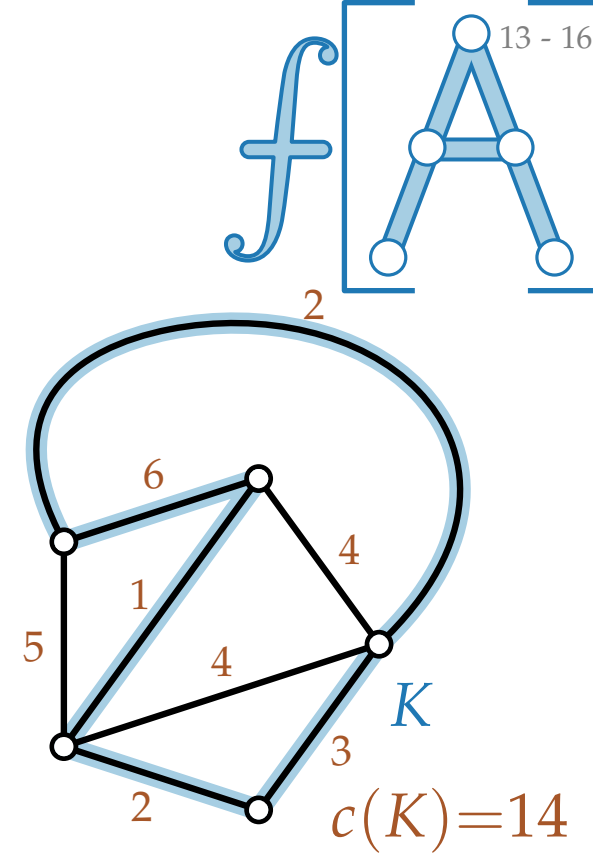
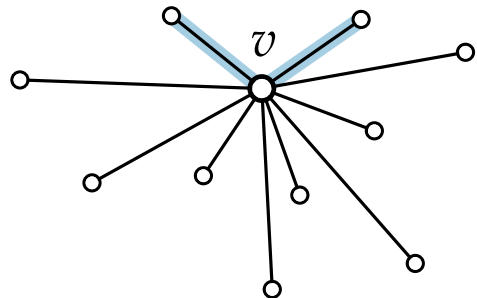
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} =$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

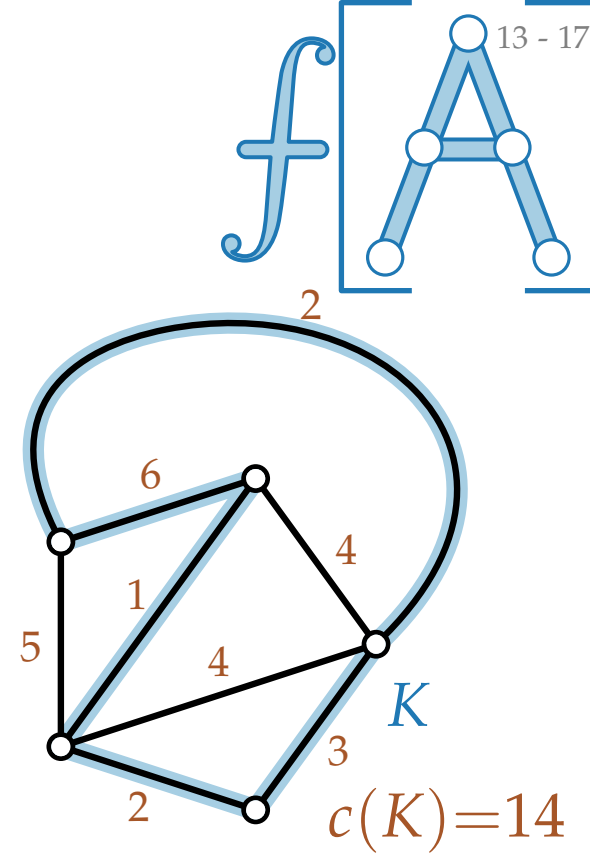
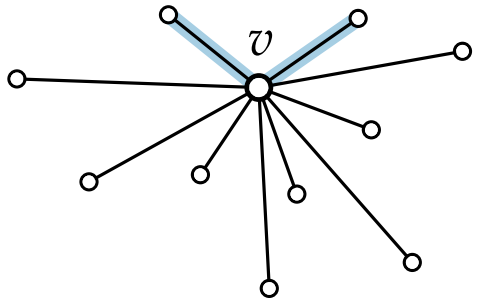
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

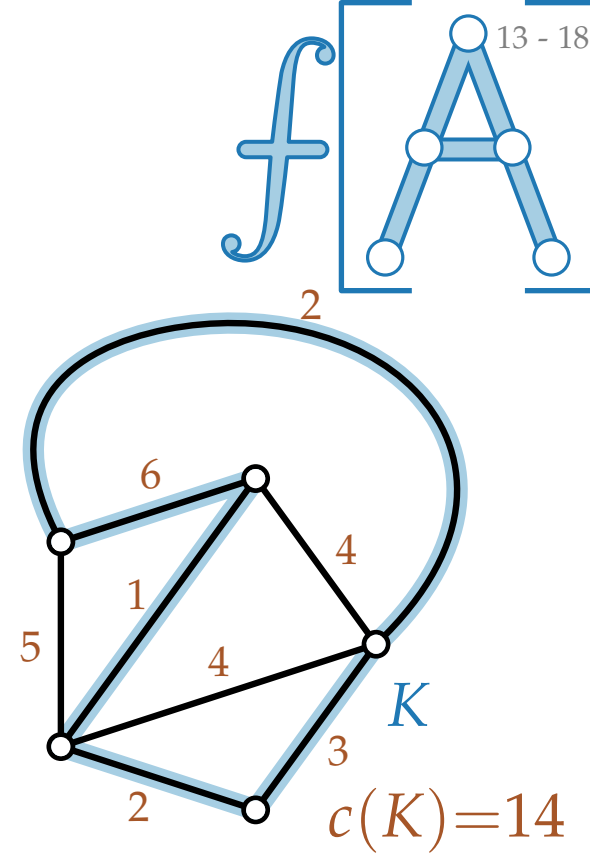
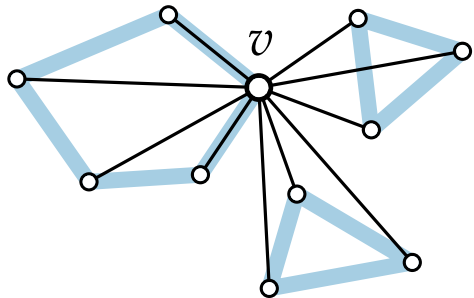
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

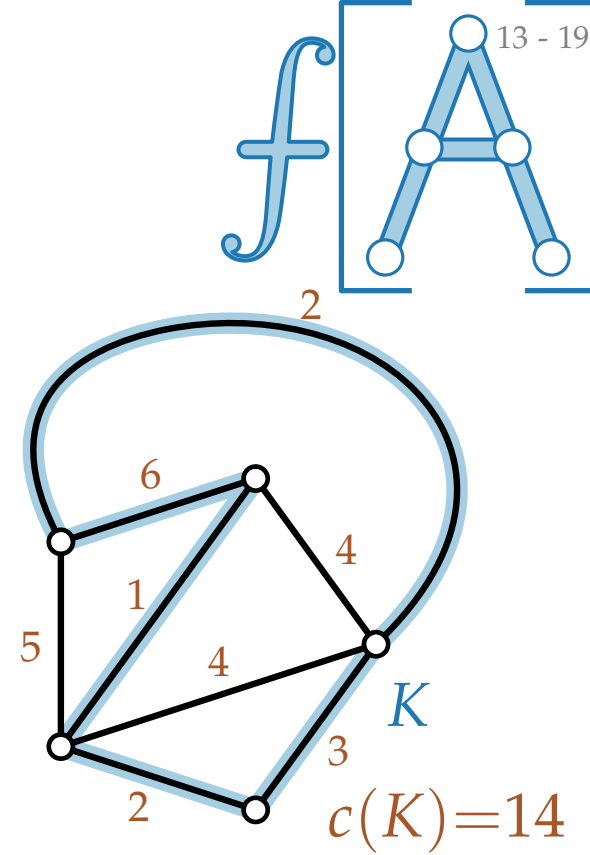
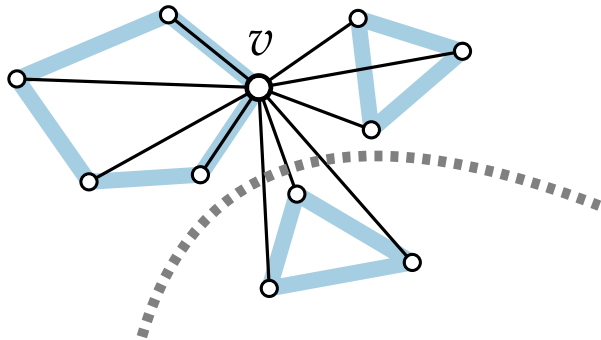
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

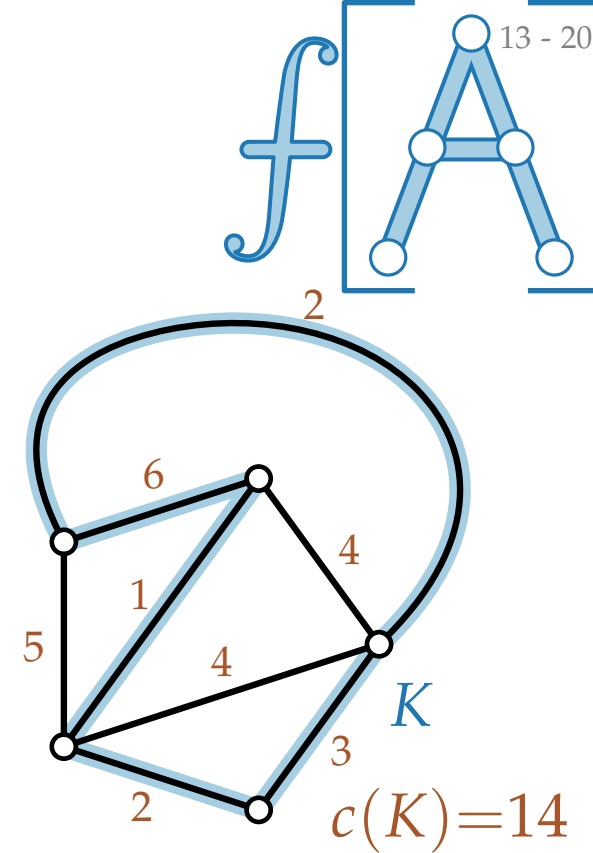
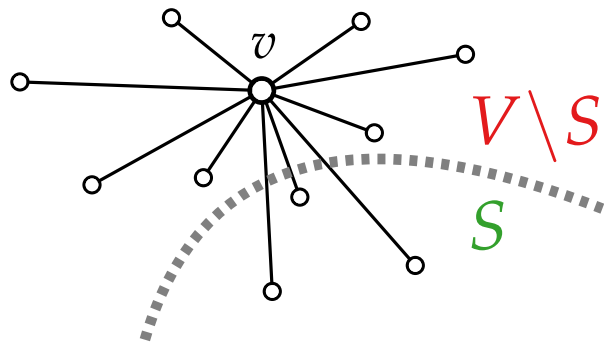
Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

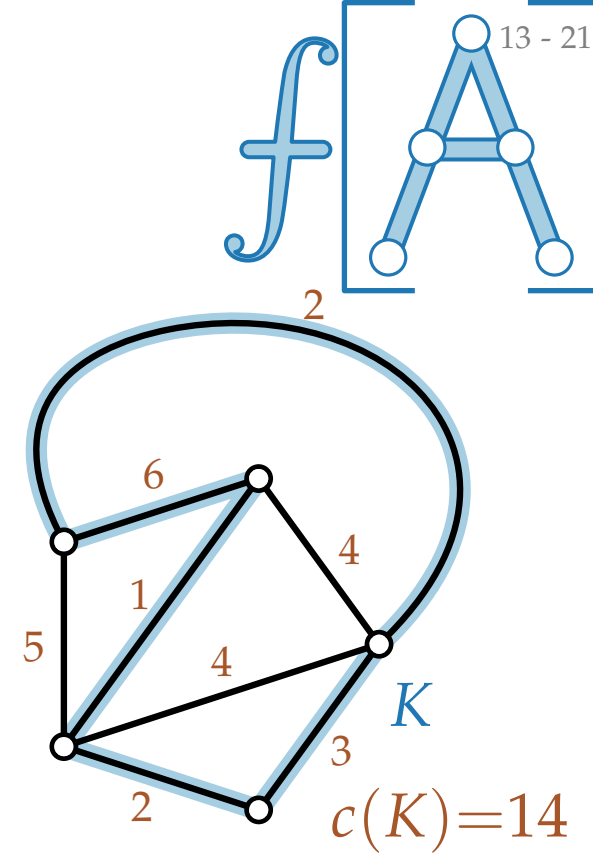
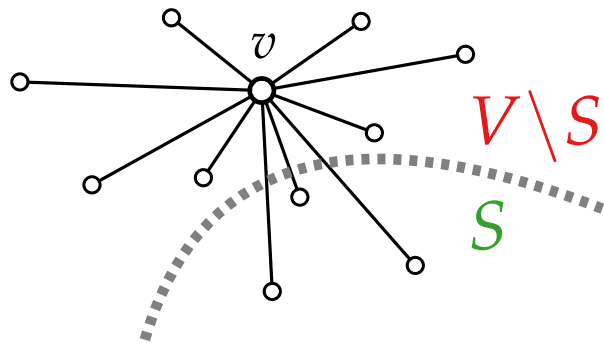
Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

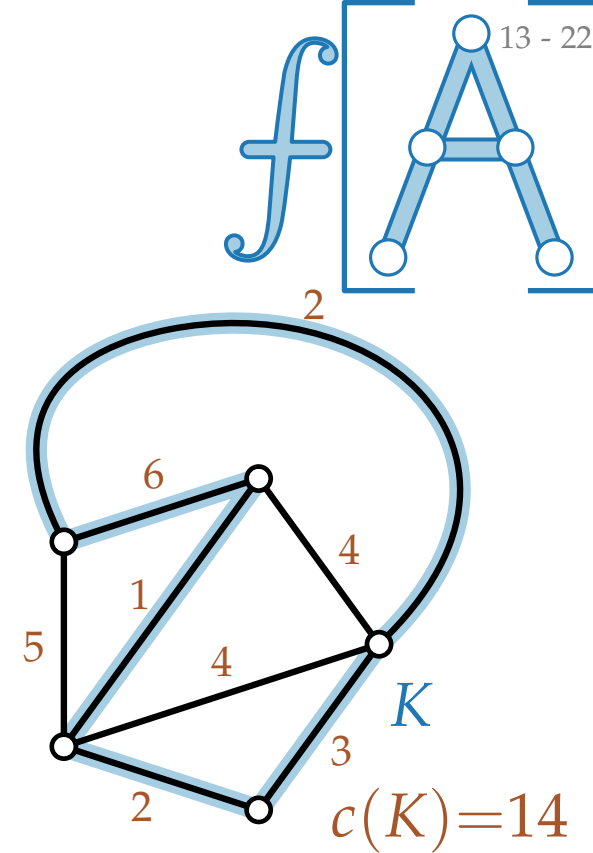
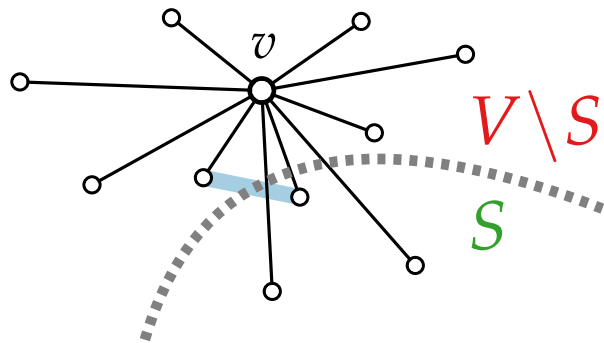
Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

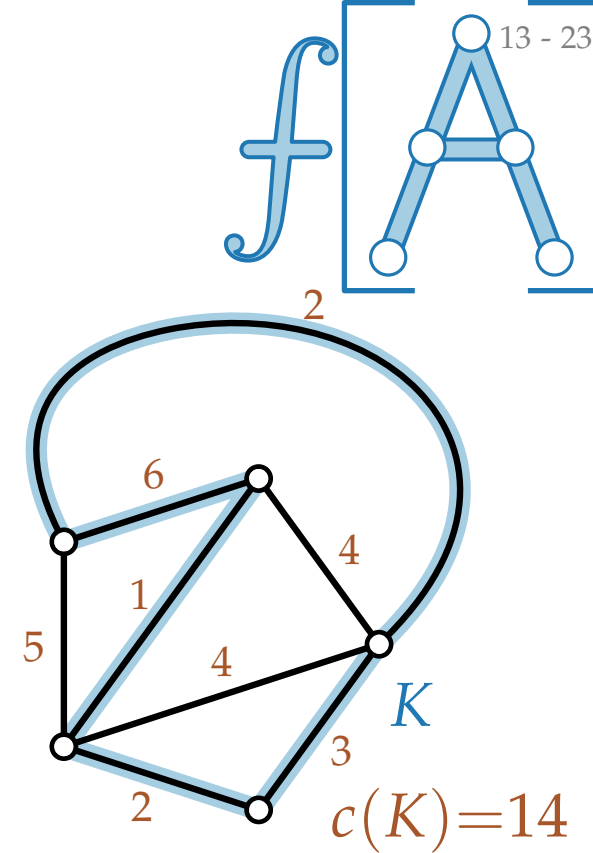
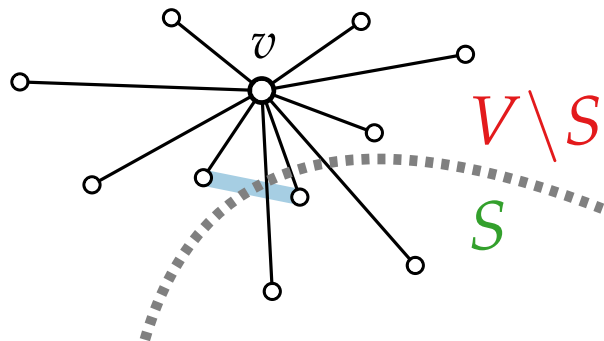
Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 1$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

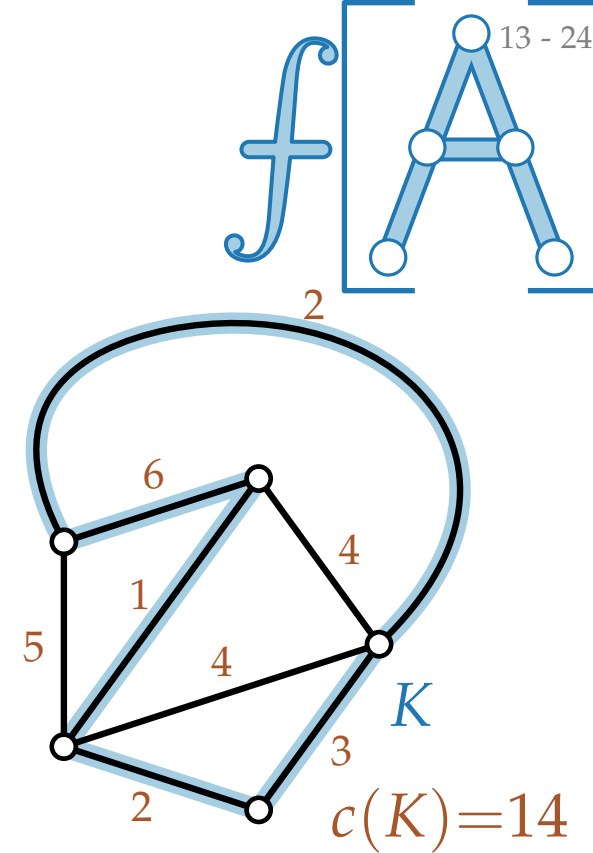
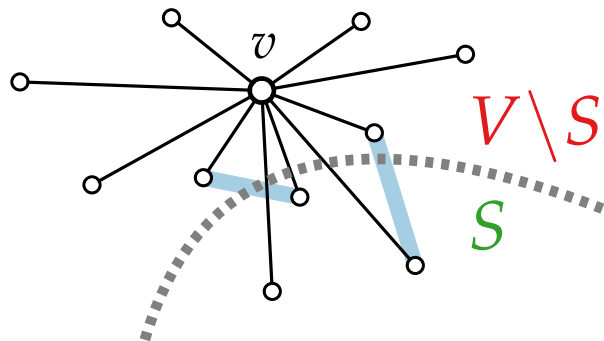
Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 1$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

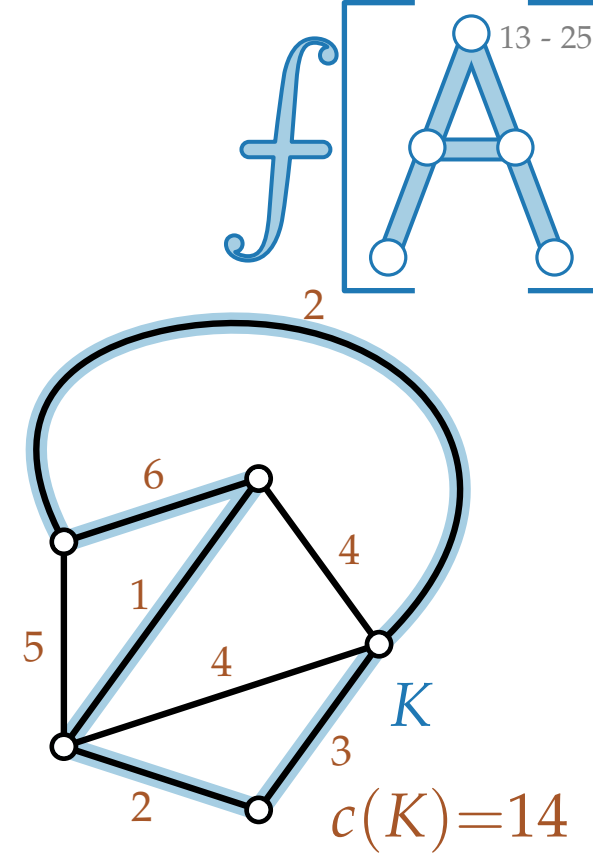
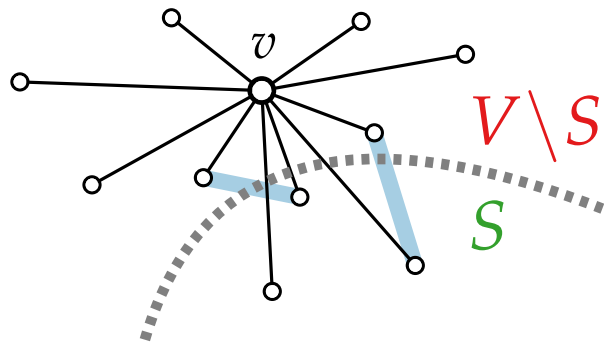
Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

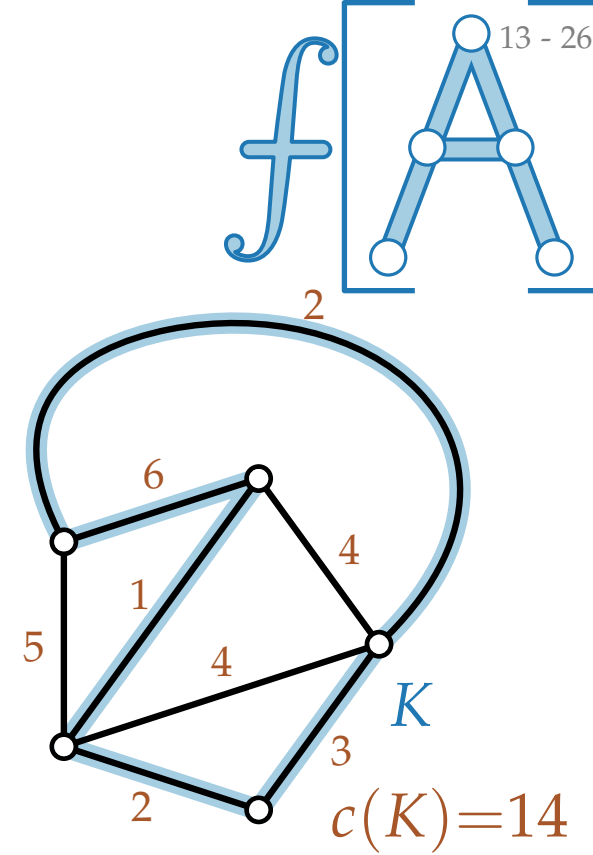
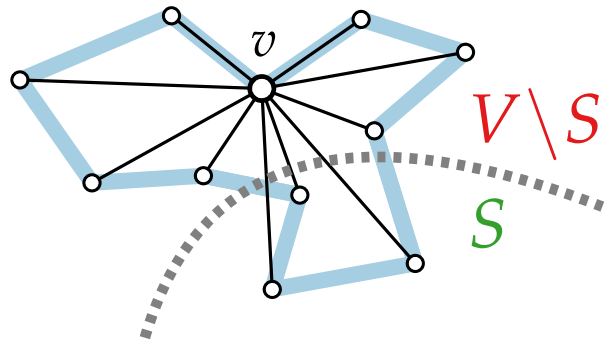
Nebenbedingungen:

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$



Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.

Variable:

Zielfunktion:

Nebenbedingungen:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

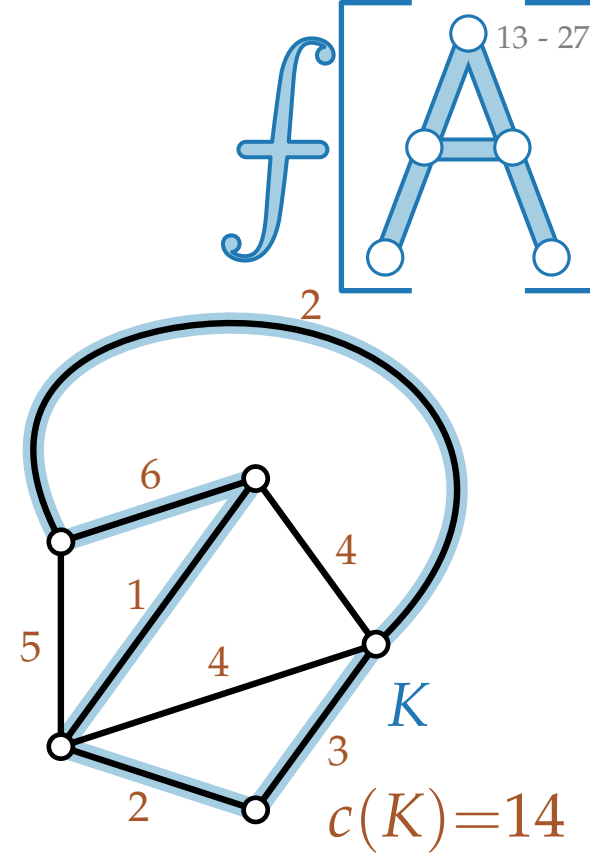
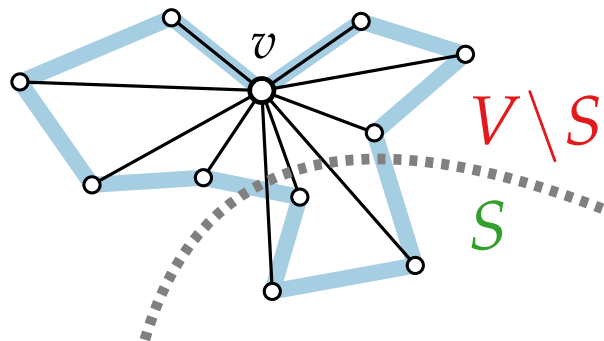
für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$

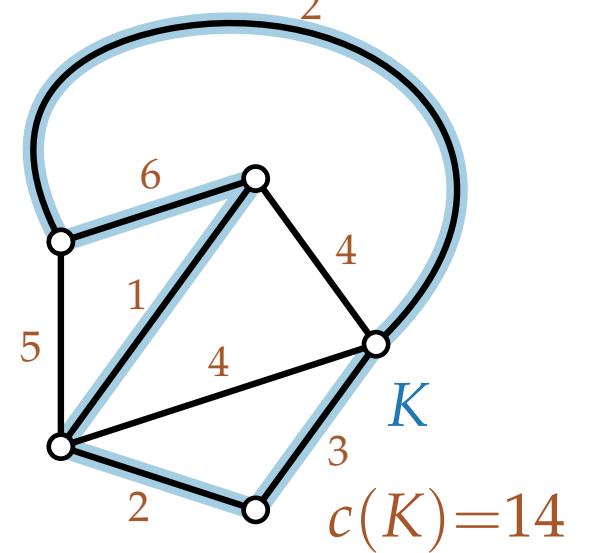
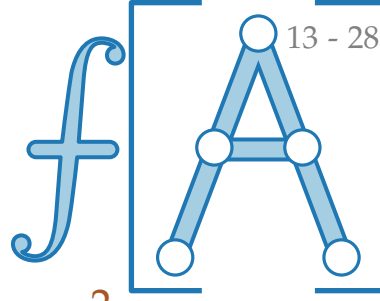


Bsp. III: ILP-Formulierung für TSP

Problem. TRAVELINGSALESMANPROBLEM

Geg. vollständiger Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ und Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$

Ges. Hamiltonkreis K in G mit minimalen Kosten $c(K)$.
d.h. Permutation σ von $\langle 1, \dots, n \rangle$, die
 $c(v_{\sigma(n)}, v_{\sigma(1)}) + \sum_{i=1}^{n-1} c(v_{\sigma(i)}, v_{\sigma(i+1)})$ minimiert.



Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

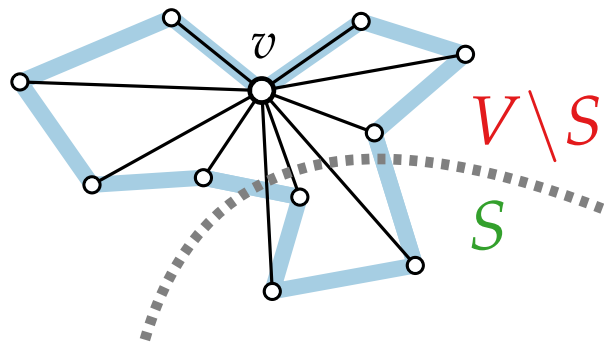
für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

Es gibt $2^{n-1} - 1$ solcher Schnitte!



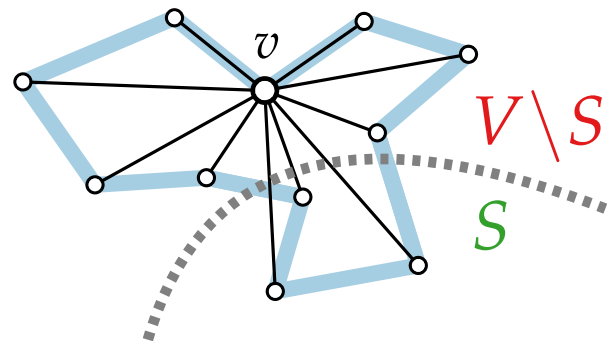
Bsp. III: ILP-Formulierung für TSP

Lösung:

Variable:

Zielfunktion:

Nebenbedingungen:



Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

für jeden Knoten $v \in V$:

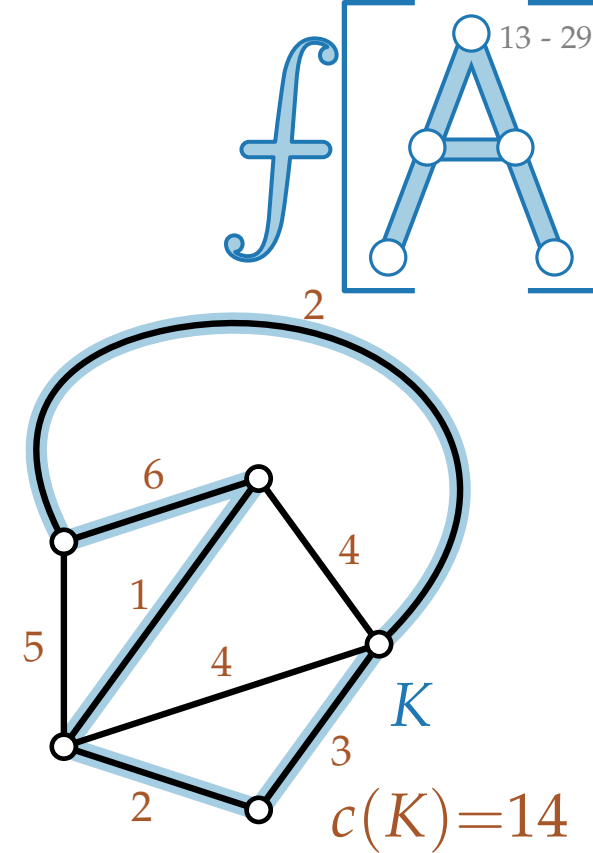
$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

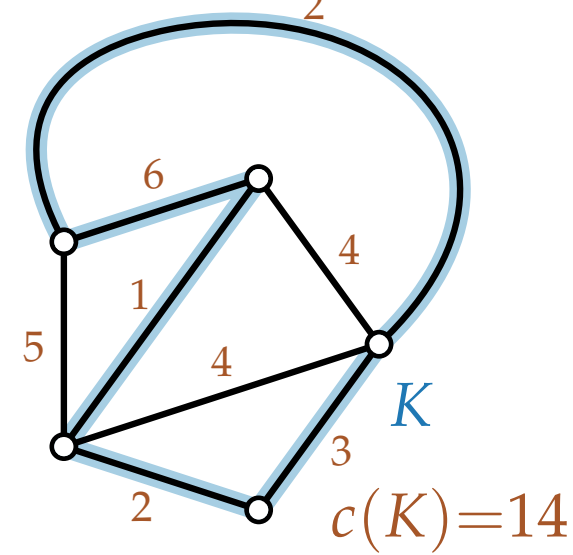
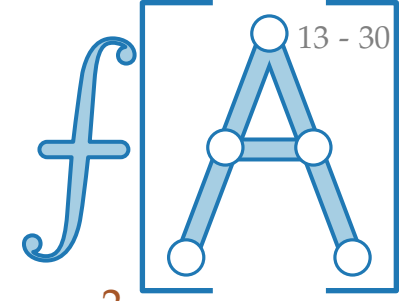
Es gibt $2^{n-1} - 1$ solcher Schnitte!

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$



Bsp. III: ILP-Formulierung für TSP

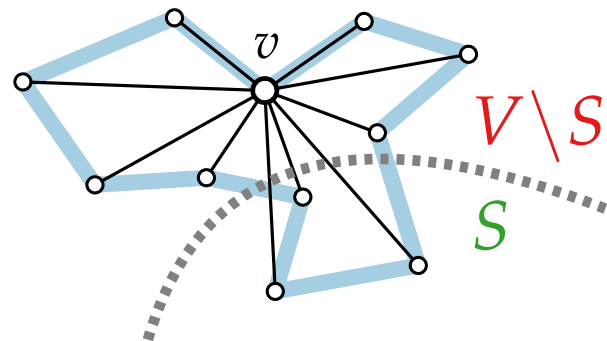
Lösung: ■ Stelle Formulierung nur mit Knotenbedingungen auf.



Variable:

Zielfunktion:

Nebenbedingungen:



Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

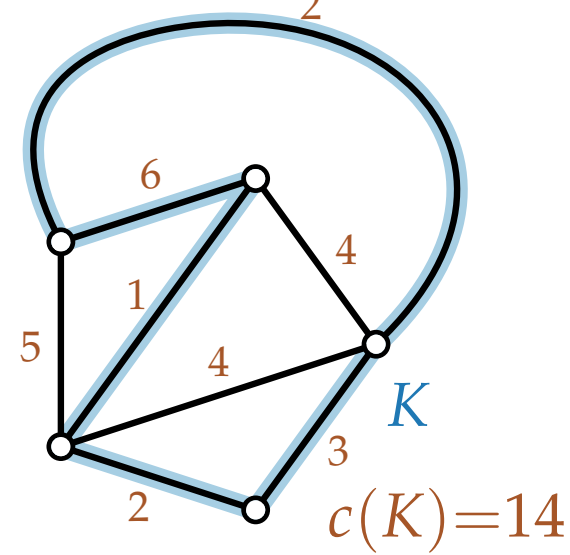
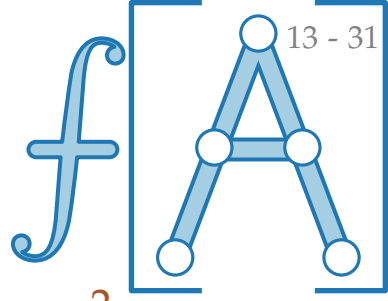
für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

Es gibt $2^{n-1} - 1$ solcher Schnitte!

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$

Bsp. III: ILP-Formulierung für TSP

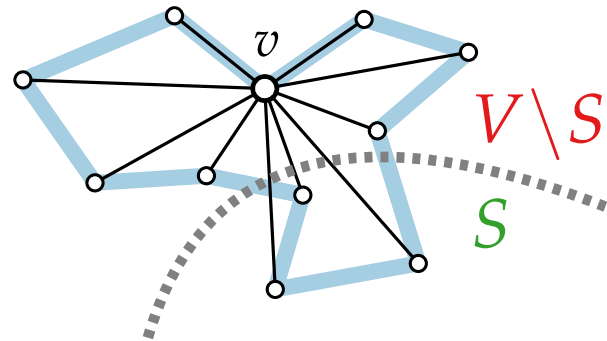
- Lösung:**
- Stelle Formulierung nur mit Knotenbedingungen auf.
 - Löse Formulierung F mit ILP-Solver (CPLEX o.ä.)



Variable:

Zielfunktion:

Nebenbedingungen:



Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

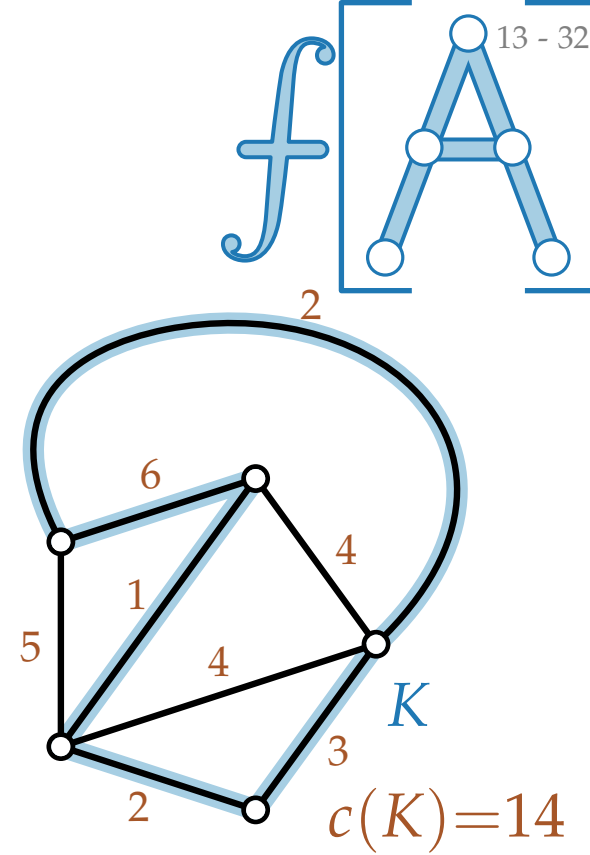
für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

Es gibt $2^{n-1} - 1$ solcher Schnitte!

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$

Bsp. III: ILP-Formulierung für TSP

- Lösung:**
- Stelle Formulierung nur mit Knotenbedingungen auf.
 - Löse Formulierung F mit ILP-Solver (CPLEX o.ä.)
 - Solange Lösung aus mehreren Kreisen besteht:



Variable:

Zielfunktion:

Nebenbedingungen:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

für jeden Knoten $v \in V$:

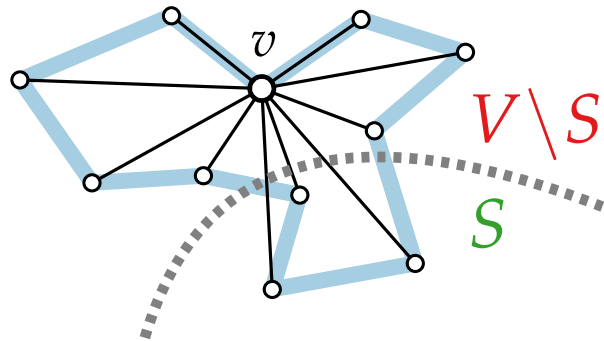
$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

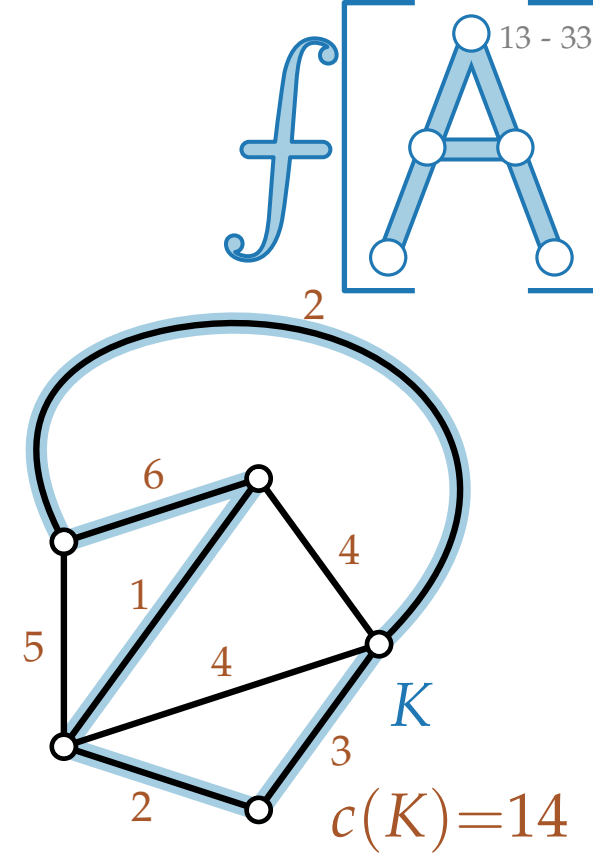
Es gibt $2^{n-1} - 1$ solcher Schnitte!

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$



Bsp. III: ILP-Formulierung für TSP

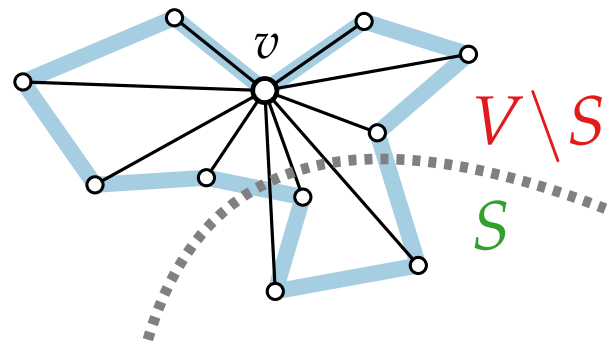
- Lösung:**
- Stelle Formulierung nur mit Knotenbedingungen auf.
 - Löse Formulierung F mit ILP-Solver (CPLEX o.ä.)
 - Solange Lösung aus mehreren Kreisen besteht:
 - Für jeden Kreis K



Variable:

Zielfunktion:

Nebenbedingungen:



Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

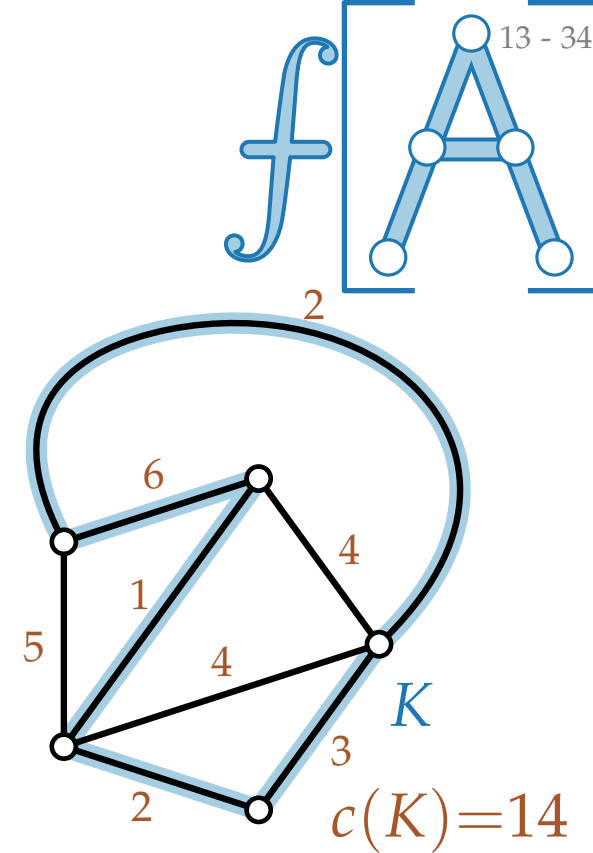
für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

Es gibt $2^{n-1} - 1$ solcher Schnitte!

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$

Bsp. III: ILP-Formulierung für TSP

- Lösung:**
- Stelle Formulierung nur mit Knotenbedingungen auf.
 - Löse Formulierung F mit ILP-Solver (CPLEX o.ä.)
 - Solange Lösung aus mehreren Kreisen besteht:
 - Für jeden Kreis K
füge die Schnittbed. für $(K, V \setminus K)$ zu F hinzu.



Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

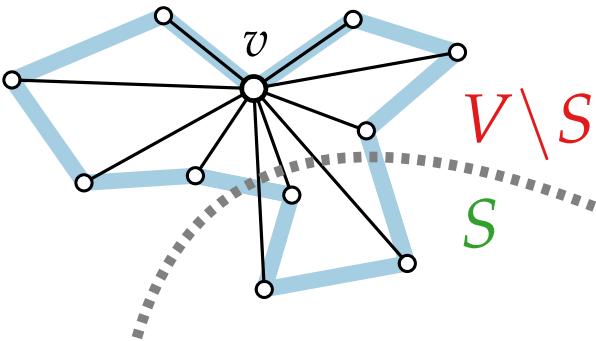
für jeden Knoten $v \in V$:

$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

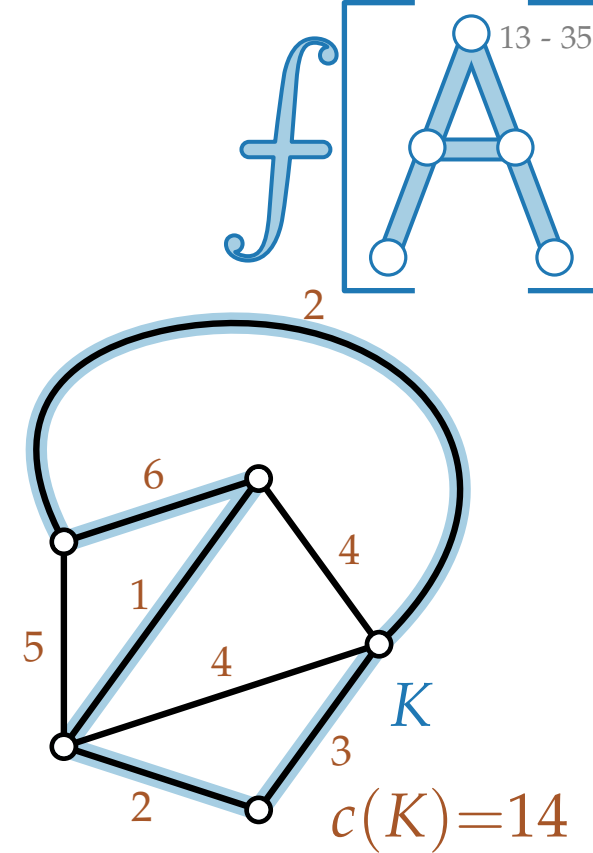
für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

Es gibt $2^{n-1} - 1$ solcher Schnitte!



Bsp. III: ILP-Formulierung für TSP

- Lösung:**
- Stelle Formulierung nur mit Knotenbedingungen auf.
 - Löse Formulierung F mit ILP-Solver (CPLEX o.ä.)
 - Solange Lösung aus mehreren Kreisen besteht:
 - Für jeden Kreis K
füge die Schnittbed. für $(K, V \setminus K)$ zu F hinzu.
 - Fahre mit der Lösung des ILPs fort („Warmstart“).



Variable:

Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$.

Zielfunktion:

Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:

für jeden Knoten $v \in V$:

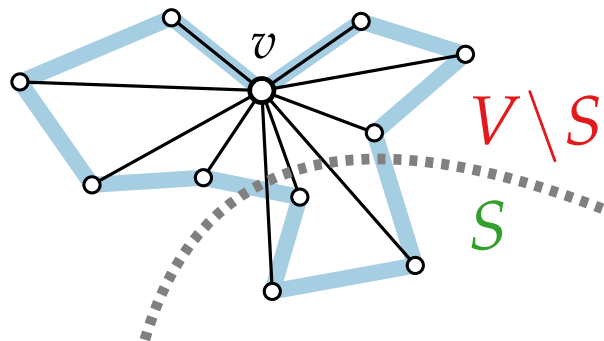
$$\sum_{u \in \text{Adj}[v]} x_{uv} = 2$$

Problem: Wie viele solcher Schnitte gibt es?

für jeden Schnitt $(S, V \setminus S)$ mit $S \neq V$

Es gibt $2^{n-1} - 1$ solcher Schnitte!

$$\sum_{u \in S, v \in V \setminus S} x_{uv} \geq 2$$

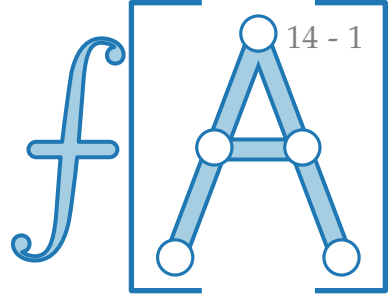


Bsp. IV: ILP-Formulierung für Paarungen

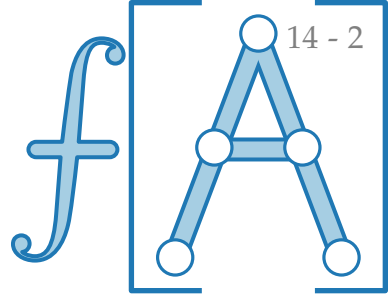
Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.:

Ges.:



Bsp. IV: ILP-Formulierung für Paarungen



Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit
Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

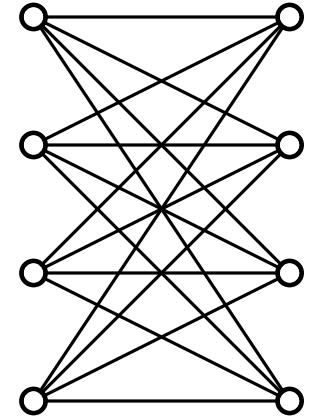
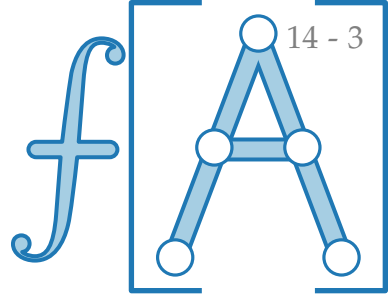
Ges.:

Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit
Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.:

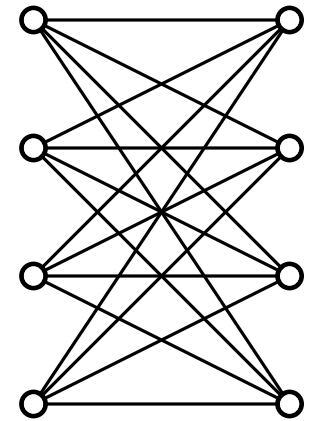
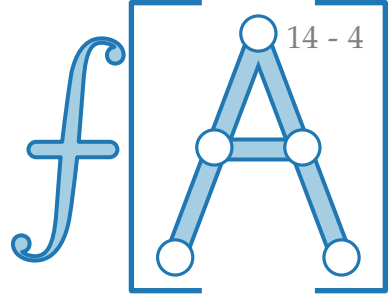


Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

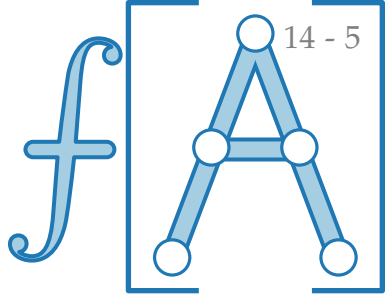
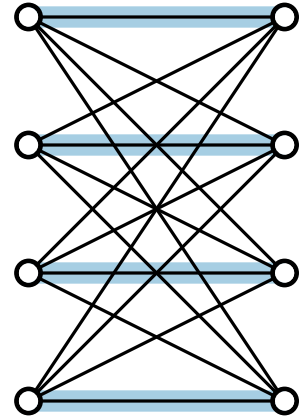


Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

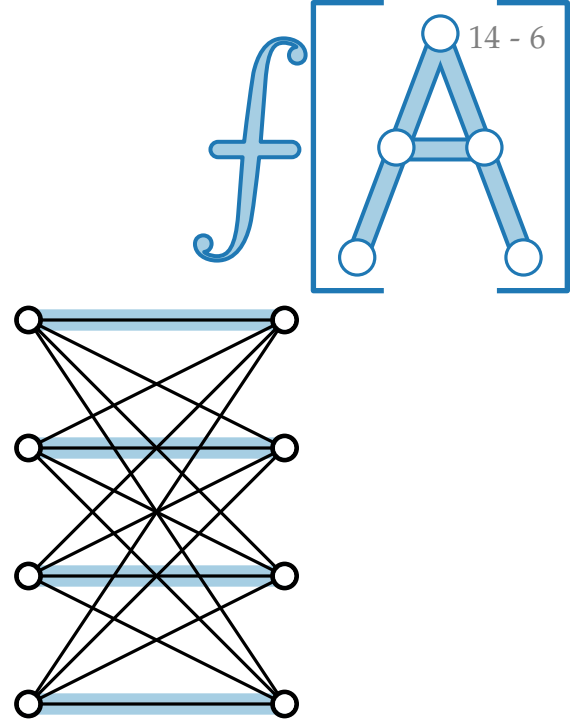
Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen:

Zielfunktion:

Nebenbedingungen:



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

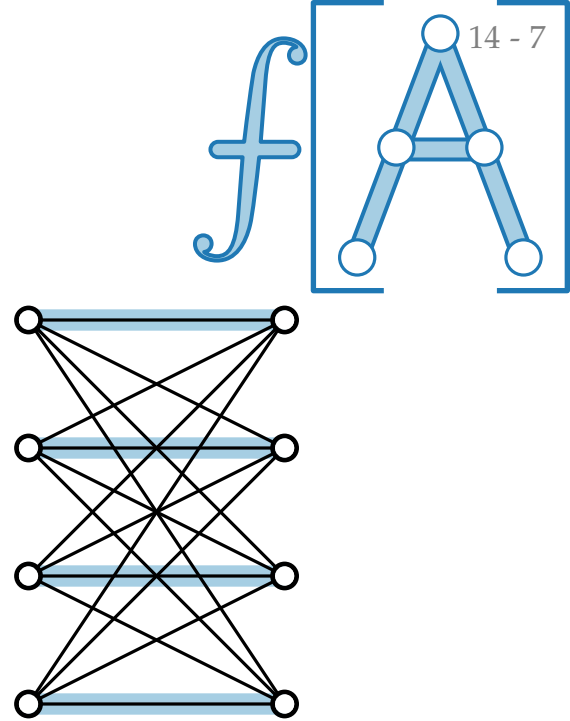
Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion:

Nebenbedingungen:



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

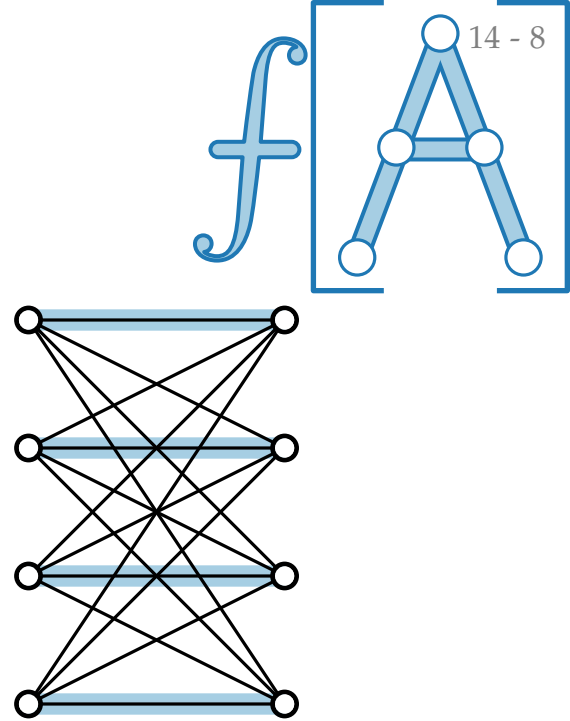
Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen:



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

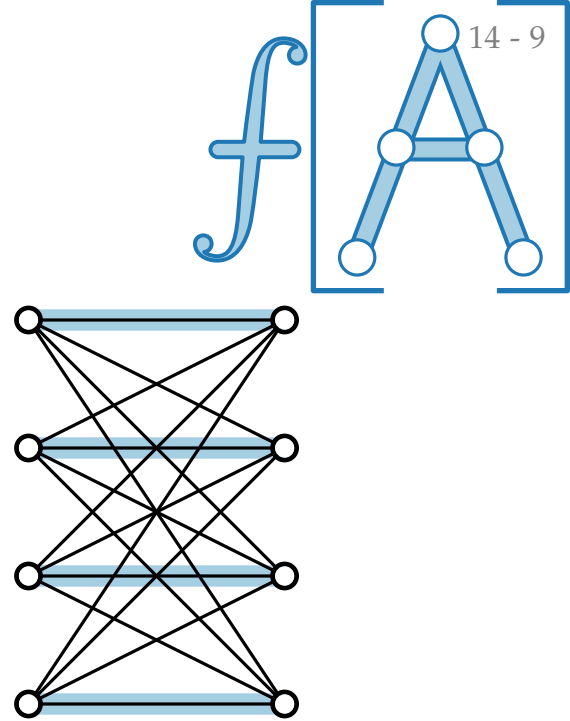
Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

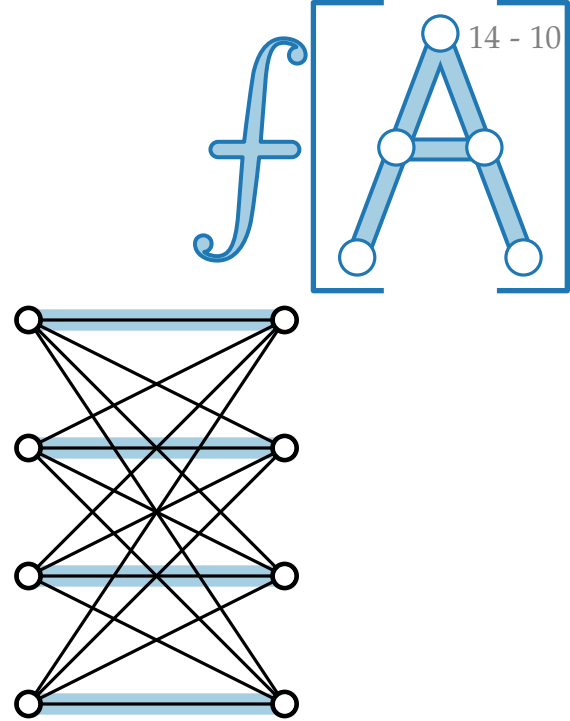
Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

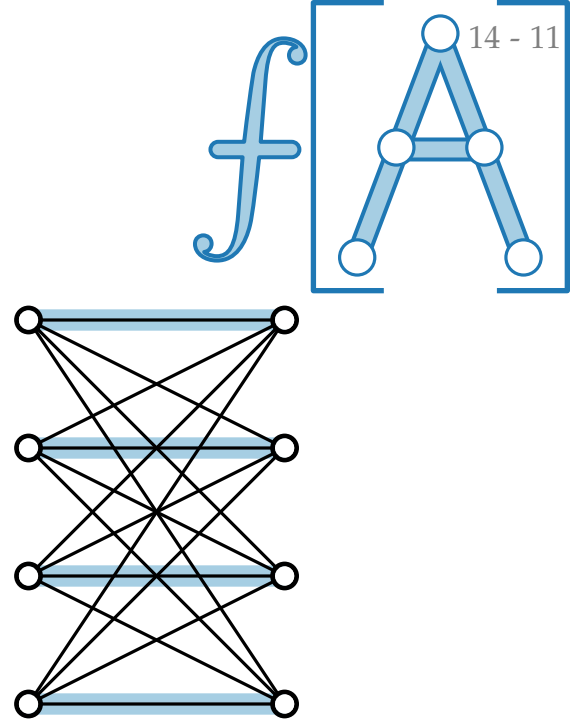
Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

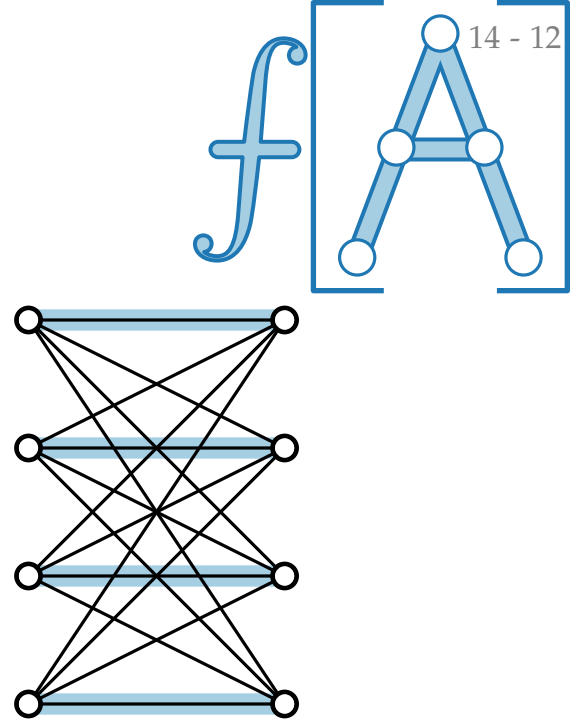
Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln:



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

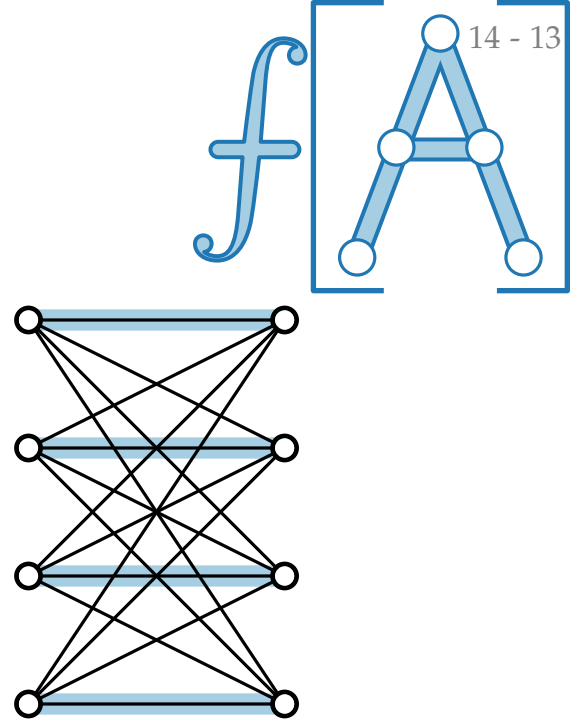
Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln:



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

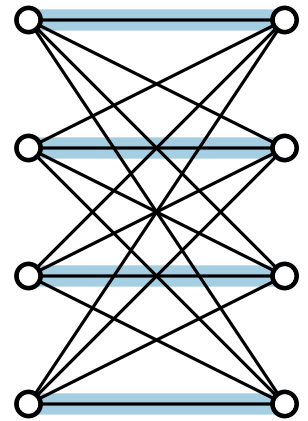
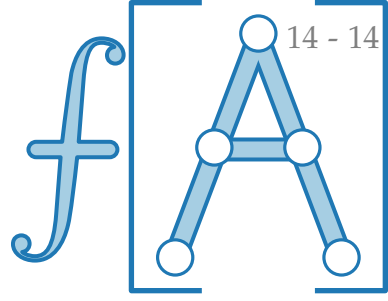
Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln:

sei $C = \max_{uv \in E} c(u, v)$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

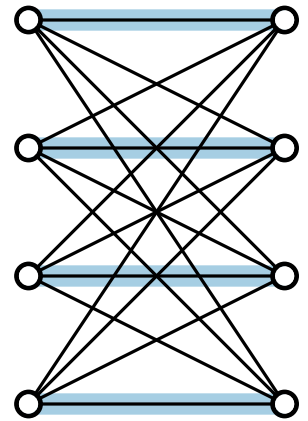
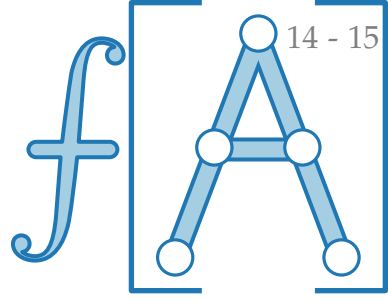
Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln: x_{uv} für jede Kante $uv \in E$

sei $C = \max_{uv \in E} c(u, v)$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

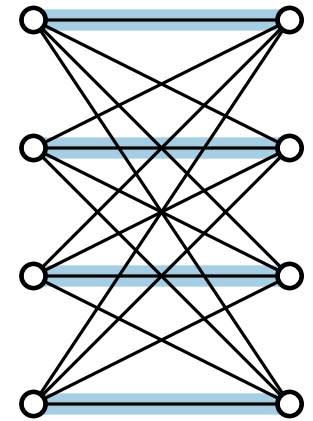
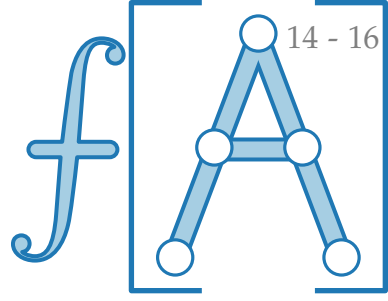
Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln: x_{uv} für jede Kante $uv \in E$

sei $C = \max_{uv \in E} c(u, v)$ Gewicht $C - c(u, v)$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

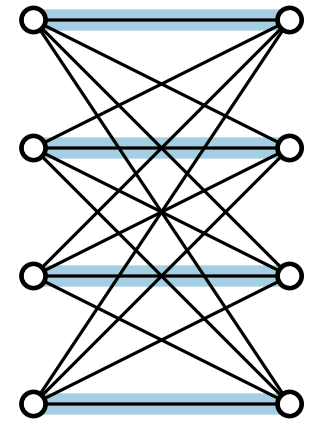
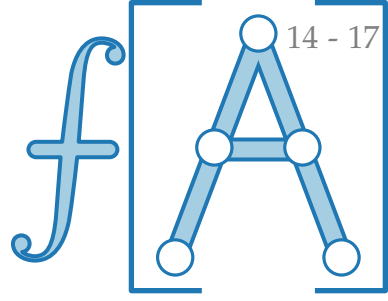
Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$?

MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln: x_{uv} für jede Kante $uv \in E$

sei $C = \max_{uv \in E} c(u, v)$ Gewicht $C - c(u, v)$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$?

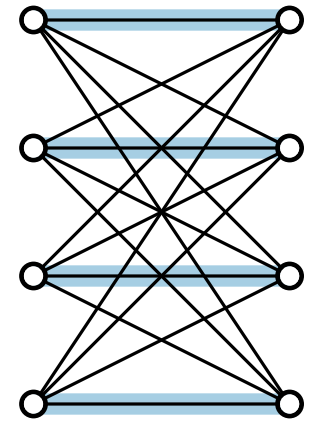
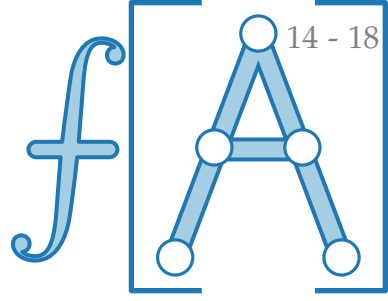
MaxSAT:

Variablen: x_{uv} für jede Kante $uv \in E$

Klauseln: x_{uv} für jede Kante $uv \in E$

sei $C = \max_{uv \in E} c(u, v)$ Gewicht $C - c(u, v)$

$\forall_{uv \in E} x_{uv}$ für alle $v \in V$



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$?

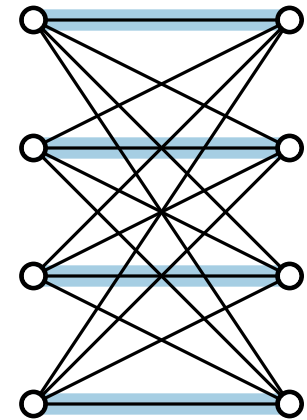
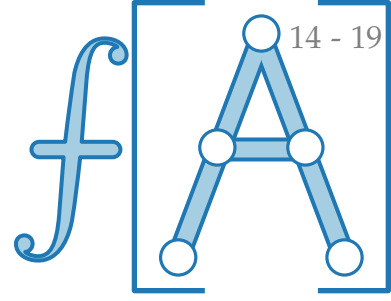
MaxSAT:

Variablen: X_{uv} für jede Kante $uv \in E$

Klauseln: X_{uv} für jede Kante $uv \in E$

sei $C = \max_{uv \in E} c(u, v)$ Gewicht $C - c(u, v)$

$\bigvee_{uv \in E} x_{uv}$ für alle $v \in V$
 $\neg X_{uv} \vee \neg X_{wv}$ für alle Nachbarn u, w von v



Bsp. IV: ILP-Formulierung für Paarungen

Problem. MINCOSTPERFECTBIPARTITEMATCHING

Geg.: bipartiter Graph $G = (V, E)$ mit Kantenkosten $c: E \rightarrow \mathbb{R}_{\geq 0}$.

Ges.: perfekte Paarung P mit minimalen Kosten $c(P) = \sum_{e \in P} c(e)$.

Variablen: Für $uv \in E$ sei $x_{uv} \in \{0, 1\}$

Zielfunktion: Minimiere $\sum_{uv \in E} c(u, v) \cdot x_{uv}$

Nebenbedingungen: $\sum_{uv \in E} x_{uv} = 1$ für alle $v \in V$?

MaxSAT:

Variablen: X_{uv} für jede Kante $uv \in E$

Klauseln: X_{uv} für jede Kante $uv \in E$

sei $C = \max_{uv \in E} c(u, v)$ Gewicht $C - c(u, v)$

$\bigvee_{uv \in E} x_{uv}$ für alle $v \in V$

$\neg X_{uv} \vee \neg X_{wv}$ für alle Nachbarn u, w von v

Gewicht $> Cn$

